

KG-Commit: A Knowledge Graph for Commit-Level Bug Detection

Design, Schema, Online Growth, and the V1→V2→V3→V4 Evolution

apache/groovy | ApacheJIT-labelled subset

July 6, 2026

Abstract

This document describes the first complete version of **KG-Commit**, a Neo4j knowledge graph built to support *commit-level (just-in-time) bug detection* on the **apache/groovy** project. The graph fuses three layers: (i) a *structural* layer of commit history (commits, developers, files, issues); (ii) an *Abstract Syntax Tree (AST)* layer that stores the actual code structure of every source file; and (iii) a *delta* layer that connects each commit directly to the precise AST nodes it changed. The graph grows *online*: commits are ingested in chronological order, and as each new commit arrives its modifications are modelled as a small “delta graph” laid on top of an evolving AST, so the whole history of every file is captured without storing a full AST snapshot per commit. Each commit carries a **buggy/benign** label from the ApacheJIT dataset, making the graph directly usable for training and online evaluation of defect-prediction models. We document every node and edge type, the exact mechanics of what happens when a commit arrives, the correctness guarantees, the strengths and limitations of the design for our task, and the planned next steps. We also report a first, deliberately GPU-free inference suite: across nine classical and graph-native methods evaluated online (prequential), an early fusion of the KG’s structural/relational signals with the classic JIT metrics lifts PR-AUC from 0.398 to 0.616 (+55% relative) over the metrics-only baseline — evidence that the AST/delta layers carry real predictive value for commit-level bug detection without any GNN. This document also records the **V2** cycle that finalizes the graph: an in-place data-quality cleanup, recovery of every previously-unparseable `.java` file (javalang + tree-sitter), a commit-message text feature, a baselines-vs-V1-vs-V2 comparison on the online operating-point metric, and a transparent pre-submission review of the build/inference code. We close with a complete inventory of the issues that remain (carried over from V1 or newly found in V2) and a two-track roadmap: per-issue remediation, and a KG refinement plan whose centerpiece is promoting commit text from a node feature to a first-class *textual/semantic graph layer*. Finally, **V3** finalizes the graph and delivers its headline contribution. It executes the remediation — most notably correcting a large *ghost-AST* inflation (≈ 1.58 M nodes, $\approx 55\%$ of the “alive” AST, left by package-restructuring renames outside the labelled commit set), driving the standing integrity audit to a clean state — and introduces the **Commit Semantic-Text Graph (CSTG)**: a theoretically-grounded third layer that models each commit’s text (message + diff) as a graph (graph-of-words/TW-IDF, an NPMI term co-occurrence graph, defect-typing, cross-modal grounding to the AST) and is enriched with an online recency/bug-cache signal. On the controlled split the CSTG *beats the strongest text baseline* (RF ROC 0.827 vs. 0.813), and under the fully-online protocol it lifts the deployed KG fusion to ROC 0.838 / PR-AUC 0.628 / MCC 0.467 — dominating every baseline on Precision, Macro-F1, G-Mean and MCC. After a redundancy audit (dropping subsumed features and edges), the clean final predictor is a genuinely multimodal, provably-clean, fully-online commit KG. Remaining work — a heterogeneous GNN and cross-project reproducibility — is scoped to V4.

Contents

1 Goal and Scope

3

2	High-Level Architecture	4
3	Node Types	4
4	Edge Types	5
5	The Base Graph	6
6	Online Growth: What Happens When a Commit Arrives	7
6.1	Diffing a modified file	7
6.2	Applying the delta and evolving the structure	8
6.3	Node identity across versions	8
6.4	Why this is space- and time-efficient	8
7	Labels for Bug Detection	8
8	Statistics (finalized full build)	9
9	Audit and Data Quality of the Finalized KG	10
10	Strengths and Limitations for Commit-Level Bug Detection	10
11	Downstream Inference	11
11.1	Problem framing (theory)	11
11.2	Features from the KG (theory + implementation)	12
11.3	Methods (theory + implementation)	12
11.4	Evaluation protocols	12
11.5	Results	12
11.6	Interpretation	13
11.7	Controlled comparison to the project baselines	14
11.8	Fully-online (prequential) evaluation and operating-point F1	15
12	Version 2 — Debugging and Refinement	18
12.1	Data-quality debugging	18
12.2	Commit-message text feature	18
12.3	Statistical comparison: $V1 \rightarrow V2$	20
12.4	Performance comparison: $V1 \rightarrow V2$	20
12.5	Baselines vs. KG-v1 vs. KG-v2 on online buggy-F1	21
12.6	V2 finalization: known issues and their status	22
13	V2 Closeout: Outstanding Issues	23
13.1	Carried over from V1 (not fixed in V2)	23
13.2	Newly detected in V2 (finalization review)	24
14	Roadmap A — Remediation of the Outstanding Issues	25
15	Roadmap B — KG Refinement and the Path to V3	26
15.1	B1. Commit text as a first-class graph layer (V3 centerpiece)	26
15.2	B2. A heterogeneous GNN over the multimodal graph	27
15.3	B3. Richer code semantics beyond the AST	27
15.4	B4. Bug-localisation supervision	27
15.5	B5. Temporal / dynamic-graph modelling	27
15.6	B6. Evaluation, calibration, and generalisation	28

16 Version 3 — Finalization: A Provably-Clean Graph	28
16.1 The ghost-AST correction (the headline finding)	28
16.2 Live-tree and connectivity repairs	28
16.3 Performance: the missing indexes	29
16.4 Revised graph statistics (V2 → V3)	29
16.5 The semantic text layer — initial (shallow) version	29
17 The Commit Semantic-Text Graph (CSTG)	31
17.1 Theoretical foundations	31
17.2 Performance (contribution)	33
17.3 Online (JIT) evaluation — the deployment protocol	33
17.4 What drives it (honest ablation)	34
17.5 The final methodology: full KG + CSTG, online	34
17.6 Comprehensive operating-point metrics	36
17.7 Layer review: limitations and literature-grounded next steps	38
17.8 V3 complete — what V4 takes on	39
18 Legacy Layer	39
A Example Cypher	39
B Version 4 — Which Subgraph is Best? A Structural Ablation	40
B.1 The single channel a subgraph feeds	40
B.2 Building the alternative layers	40
B.3 One shared uniform differ	42
B.4 The parametrised online growth engine	42
B.5 Evaluation protocol	42
B.6 Results	43
B.7 The seven headline metrics	43
B.8 Per-method comparison: it is not only Fusion	44
B.9 Five new graph-native online KG-inference methods	46
B.10 Choosing the final fusion formula	48
B.11 Full ablation: the best fusions at every size	49
B.12 Selecting five inference methods for the final study	55
B.13 Final experiments: five methods across the graph family	56
B.14 Choosing the final fusion of the five methods	58
C Scalability, Complexity and Statistical Analysis	60
C.1 Statistical profile of the final graph	61
C.2 Per-commit growth and modification	62
C.3 Time and space complexity	62
C.4 Statistical significance of the headline comparisons	64
C.5 Conclusion	64
C.6 For the final paper	65

1 Goal and Scope

The objective is a graph from which a model can *infer whether a commit is likely to introduce a bug*, using both the change itself and the surrounding code structure. Concretely:

- **Project:** apache/groovy (a full local git clone).

- **Commit scope:** the **8,059 commits** that appear in the **ApacheJIT** dataset for groovy (`data/apachejit/projects/apache_groovy.csv`), spanning 2003–2019. These are the only commits we treat as the labelled study set; the local database also contains the surrounding history as **Commit** nodes, but only the ApacheJIT commits are flagged `in_jit=true` and grown into the AST/delta layers.
- **Base snapshot:** commit `408b29851d7bbe4d343340832297e4be7e0c5578` (“BASE”), the first substantial Java commit, whose 120 `.java` files seed the AST layer.
- **Label:** `buggy` $\in \{\text{True}, \text{False}\}$ from ApacheJIT (1,614 buggy / 6,445 benign $\approx 20\%$ positive).

2 High-Level Architecture

The graph has three conceptual layers that share the same node space:

1. **Structural / process layer** — the classical commit graph: **Project**, **Branch**, **Developer**, **Commit**, **File**, **Issue**, and the relationships between them (authorship, parenthood, file changes, issue fixes).
2. **AST layer** — for every tracked source file, a tree of **ASTNodes** linked by `AST_CHILD`, rooted at the **File** via `HAS_AST`. Seeded from the 120 BASE files and extended as new files appear.
3. **Delta layer** — four relationship types (`ADDS`, `REMOVES`, `UPDATES`, `MOVES`) that connect a **Commit** *directly* to the AST nodes it changed. This is the heart of the commit-level model.

A small *legacy* layer (**ASTDiff**/**ASTEdit**) from an earlier experiment is still present but **deprecated** in favour of the delta layer (§18).

Theory. The design is a *heterogeneous information network* (HIN): typed nodes and typed edges in one graph. This lets a single model reason jointly over *process* evidence (who changed what, when, how often) and *code-structure* evidence (the syntactic shape of the change and its surroundings) — the two signal sources the defect-prediction literature treats separately. Anchoring commit changes to *shared*, *persistent* AST nodes (rather than to per-commit summary blobs) is what enables relational and neighbourhood reasoning across the whole history.

Practice. The structural/process layer is produced in `kg_commit/knowledge`, (driven from the ApacheJIT CSV and the local git clone). The AST layer is seeded once (`build_file_ast_graphs.py` + `ingest_base_kg_and_ast.py`), positions are stamped (`add_positions_to_astnodes.py`), labels are applied (`apply_commit_labels.py`), and the delta layer is grown by the online engine (`build_online_kg.py`, using `online_ast_diff.py`). Everything is stored in one Neo4j database.

3 Node Types

Table 1 lists every node label, its meaning, and its key properties.

Table 1: Node types.

Label	Meaning	Key properties
Project	The repository.	<code>id</code> (e.g. <code>apache/groovy</code>).
Branch	A git branch.	<code>id</code> .
Developer	A commit author.	<code>id</code> (name/email).

Label	Meaning	Key properties
Commit	A single commit.	id (40-char SHA), message, committed_at, author_ts (UNIX seconds = online order), in_jit, buggy, is_fix, jit_year, and the ApacheJIT change metrics la, ld, nf, nd, ns, ent, ndev, age, nuc, aexp, arexp, asexp.
File	A repository-relative file path.	id (path).
Issue	A referenced issue (e.g. GROOVY-20).	id.
ASTNode	One node of a file's AST (declaration, statement, expression, literal, or identifier leaf).	id = {file}::A{i} for original nodes or {file}::D{sha8}:{i} for nodes inserted by a commit; file, ast_type (javalang type), group (declaration/statement/expression/literal/leaf), value, is_leaf, depth, pos_line, pos_col, and on evolving nodes alive (false once removed), removed_by, is_delta (true if inserted by a commit).
ASTDiff (<i>legacy</i>)	Per-(commit,file) diff summary.	id, commit_id, file, n_insert/delete/update/move, total.
ASTEdit (<i>legacy</i>)	A single GumTree edit action.	action, node_type, node_label, new_label, parent_type, position.

AST node identity. Node ids are *deterministic*: the same source file always yields the same {file}::A{i} ids, where *i* is a pre-order DFS counter from the AST builder. This determinism is what lets us thread node identity across versions of a file (§6).

4 Edge Types

Table 2 groups the relationships by layer.

Table 2: Relationship types.

Type	From → To	Meaning / properties
<i>Structural / process layer</i>		
BELONGS_TO	Commit → Project	Commit is part of the project.
INSIDE_BRANCH	Branch → Commit	Commit is on the branch.
PARENT_OF	Commit → Commit	Git parent relation (history spine).
AUTHORED_BY	Commit → Developer	Authorship.
FIXES_ISSUE	Commit → Issue	Commit message references/fixes an issue.
<i>File-change layer</i>		

Type	From → To	Meaning / properties
ADDED	Commit → File	File added by the commit.
MODIFIED	Commit → File	File modified by the commit.
DELETED	Commit → File	File deleted by the commit.
RENAMED_FROM	Commit → File	File rename endpoints.
and RE-NAMED_TO		
<i>AST layer</i>		
HAS_AST	File → ASTNode	Links a file to its AST root node.
AST_CHILD	ASTNode → ASTNode	Parent-to-child tree edge; property <code>pos</code> = child ordinal.
<i>Delta layer (the commit-level model)</i>		
ADDS	Commit → ASTNode	The commit <i>inserted</i> this node (node has <code>is_delta=true</code>); the node is also wired into the tree via a new <code>AST_CHILD</code> edge to its parent.
REMOVES	Commit → ASTNode	The commit <i>deleted</i> this node; the node is kept but marked <code>alive=false</code> , <code>removed_by=commit</code> (history preserved).
UPDATES	Commit → ASTNode	The commit changed the node’s value; properties <code>old_value</code> , <code>new_value</code> .
MOVES	Commit → ASTNode	The commit relocated the node; property <code>new_parent_type</code> .
<i>Legacy layer (deprecated)</i>		
HAS_AST_DIFF	Commit → ASTDiff	old per-file diff summary.
EDIT	ASTDiff → ASTEdit	one edit of the diff.
DIFFED_AT	File → ASTDiff	file the diff applies to.

5 The Base Graph

The AST layer is seeded at the BASE commit. Each of the 120 `.java` files is parsed with the `javalang` library and converted to a graph by a pre-order traversal that assigns every node a stable id `{file}::A{i}`. Two kinds of nodes are produced:

- **Structured nodes** ($\approx 36\%$): real `javalang.tree` nodes (classes, methods, fields, statements, expressions, literals), carrying a source position (`pos_line`, `pos_col`).
- **Identifier leaves** ($\approx 64\%$): raw string children (identifier and operator tokens) that `javalang` attaches to a node; these have no source position (`pos_line=-1`).

Each file is attached as `FILE` $\xrightarrow{\text{HAS_AST}}$ `ROOT` $\xrightarrow{\text{AST_CHILD}^*}$ `DESCENDANTS`. At BASE this is 120 files and $\approx 46\text{k}$ AST nodes. This is the “previous structure” onto which all later commit deltas are laid.

Theory. An AST abstracts away surface formatting and records the grammatical structure of code; making it an explicit subgraph (rather than a flat token sequence) lets downstream models exploit parent/child and type-of-node relations. Choosing one consistent parser (`javalang`) for the whole graph is what makes node identities comparable across file versions (§6.3).

Implementation. `build_file_ast_graphs.py` parses each BASE file in a killable subprocess (javalang can hang on pathological input) and emits a `networkx` graph whose node ids are the deterministic `{file}::A{i}` counters; `ingest_base_kg_and_ast.py` MERGEs these into Neo4j under each `File`; `add_positions_to_astnodes.py` re-runs the identical traversal to stamp `pos_line/pos_col` on every node.

6 Online Growth: What Happens When a Commit Arrives

Theory. The problem of describing how one tree becomes another is *tree differencing* / tree edit distance: find a minimal-cost set of insert/delete/update/move operations mapping a “before” tree to an “after” tree. Equivalently, one seeks a *matching* between the two node sets; unmatched before-nodes are deletions, unmatched after-nodes are insertions, and matched nodes whose label differs are updates. We compute this matching and record each operation as a typed edge from the commit to the affected AST node. Modelling change as a delta (operations) rather than as two stored snapshots is both information-preserving and asymptotically smaller: storage grows with the *amount of change*, not with history length $\times$ file size.

Practice. Commits are processed in **chronological order** (`author_ts`). The engine (`build_online_kg.py`) maintains, per tracked file, the *current* state of its AST in the graph. When commit *C* arrives, it computes the set of changed `.java` files (`git diff` of *C* against its first parent) and, for each file, takes one of four actions.

Listing 1: Per-commit processing.

```
for each commit C in stream order (by author_ts):
  for each changed .java file F of C (status A / M / D):

    if status == DELETED:
      mark every alive ASTNode of F as alive=false, removed_by=C
      add (C)-[:REMOVES]->(node) for each
      stop tracking F

    elif status == ADDED OR F has no AST yet (lazy bootstrap):
      build the FULL AST of F (from C for ADDED, or from C^1 if the
      file already existed but was never tracked)
      attach File-[:HAS_AST]->root and the AST_CHILD tree
      if ADDED: add (C)-[:ADDS]->(node) for every node of the new file
      # an untracked-but-modified file is bootstrapped at C^1 and then
      # falls through to the MODIFIED case below

    if status == MODIFIED (file is tracked):
      diff the file's stored state -> its content at C (see 7.1)
      apply the resulting delta to the graph (see 7.2)
      label of C (buggy/benign) is already on the Commit node
      checkpoint progress (resumable)
```

6.1 Diffing a modified file

The modify path uses a **javalang-native tree differ** (`online_ast_diff.py`). Both the “before” state (the file’s stored content) and the “after” state (its content at *C*) are parsed with the *same* builder used for the graph, so their node ids are directly comparable. Matching proceeds in three stages:

1. **Subtree-hash, bottom-up:** every subtree is hashed by (type, value, children hashes); identical subtrees in before/after are matched greedily, preferring equal source positions. This matches the large *unchanged* majority of the file exactly.

2. **Parent propagation, top-down:** if two matched nodes have unmatched parents of the same type, the parents are matched. This covers changed-but-aligned ancestors.
3. **Positional fallback:** any remaining node is matched by (type, line, col).

The differ outputs four sets: *matched* (with a value-changed flag), *deleted*, *inserted*, and *moved*.

6.2 Applying the delta and evolving the structure

The delta is written to Neo4j and the stored AST is evolved so that the next commit diffs against the correct state:

- **matched** nodes keep their graph id; their position is re-stamped to the new source location; if the value changed, an UPDATES edge is added and the node’s **value** is updated.
- **deleted** nodes are marked **alive=false**, **removed_by=C**, and a REMOVES edge is added (the node is *kept* for history).
- **inserted** nodes are created (**is_delta=true**), wired into the tree with a new AST_CHILD edge to their (existing or newly inserted) parent, and an ADDS edge is added.
- **moved** nodes get a MOVES edge recording the new parent type.

6.3 Node identity across versions

Correctly carrying a node’s identity from one version of a file to the next is the crux of the model. We thread identity with a per-file map {**parse-local-id** → **graph-node-id**}. Because the same source deterministically yields the same parse-local ids, the “after” map of commit *C* is exactly the “before” map of the next commit that touches the file. Base files start from the identity map (graph id = parse id).

Why not GumTree or position matching? An earlier version matched GumTree’s diff actions back to graph nodes by source *position*. This *drifted*: GumTree parses a different tree (JavaParser/JDT) than our javalang graph, and $\approx 32\%$ of AST nodes share a (**line,col**) with another node, so re-stamping mis-assigned them. Measured against ground truth, the position-based evolution achieved only **recall 0.41**. The javalang-native differ with deterministic identity achieves **precision = recall = 1.000** on the validation window (`validate_online_kg.py`). *Scope of this check (stated honestly)*: it compares the *set of source positions* (**pos_line,pos_col**) of the graph’s *alive*, *positioned* nodes against a fresh ground-truth parse of the file at its current stored state, over the **60 evolved files** sampled by the validator. It therefore covers the $\approx 36\%$ of nodes that carry a position (declarations/statements/expressions/literals); identifier leaves (**pos_line=-1**) are threaded by the same deterministic identity map but are not separately position-verified, and because the comparison is a *set* (not a multiset or full tree isomorphism) it does not by itself rule out two distinct nodes sharing one (**line,col**). A stronger full-tree check (node count and (**ast_type,line,col**) multiset including leaves, over all evolved files) is a V2 finalization item (§12.6).

6.4 Why this is space- and time-efficient

Only *one* evolving AST is kept per file, plus the delta edges; we never store a separate full AST per commit. A commit that changes three lines adds a handful of delta edges, not a fresh copy of the file’s tree. Removed nodes are retained (flagged) so the complete change history is queryable while the live tree always reflects the current code.

7 Labels for Bug Detection

Every ApacheJIT commit is annotated (`apply_commit_labels.py`) with **buggy**, **is_fix**, **author_ts** (online order), **jit_year**, and the standard JIT change metrics. The class balance is $\approx 20\%$ buggy.

A quick sanity check confirms signal: buggy commits are on average much larger (mean lines added ≈ 196 vs. ≈ 70 for benign) and more scattered (mean change entropy ≈ 1.26 vs. ≈ 0.73).

For *online* evaluation, the `author_ts` ordering lets one reproduce the exact information available when each commit “arrives”: all earlier commits’ labels are known, the one under test is not.

8 Statistics (finalized full build)

Final sizes of the assembled graph after the complete online build over all 8,059 labelled commits.

Element	Count
Commit (total in DB)	16,887
of which ApacheJIT-labelled (<code>in_jit</code>)	8,059
buggy / benign	1,614 / 6,445
carrying AST deltas	7,521
ASTNode	3,635,438
inserted by commits (<code>is_delta</code>)	1,249,170
removed (<code>alive=false</code> , kept as history)	807,120
alive / untouched-base (<code>alive=NULL</code>)	2,805,375 / 22,943
positioned (have <code>pos_line>0</code>)	1,233,500
File	5,655
with an AST (<code>HAS_AST</code>)	3,006
born during the stream (post-BASE)	1,611
Issue / Developer	4,877 / 319
AST_CHILD	3,652,425
ADDS	2,337,767
REMOVES	818,063
MOVES	147,330
UPDATES	13,316
MODIFIED / ADDED / DELETED	35,252 / 3,466 / 1,602
PARENT_OF / AUTHORED_BY	13,122 / 13,121
FIXES_ISSUE	6,404

On disk the graph store is ≈ 3.3 GB; with transaction logs the working footprint is ≈ 5.6 GB (budget ≈ 10 GB for headroom).

Bug signal. Buggy commits make structurally much larger AST changes than benign ones: on average ≈ 860 delta edges per buggy commit versus ≈ 299 per benign commit (and ≈ 196 vs. ≈ 70 lines added). This separation is the basic reason the graph is useful for the task. Figure 1 shows both this size separation and the strong *temporal drift* of the bug rate across 2003–2019 — the latter being why a time-aware online protocol (§11) is essential.

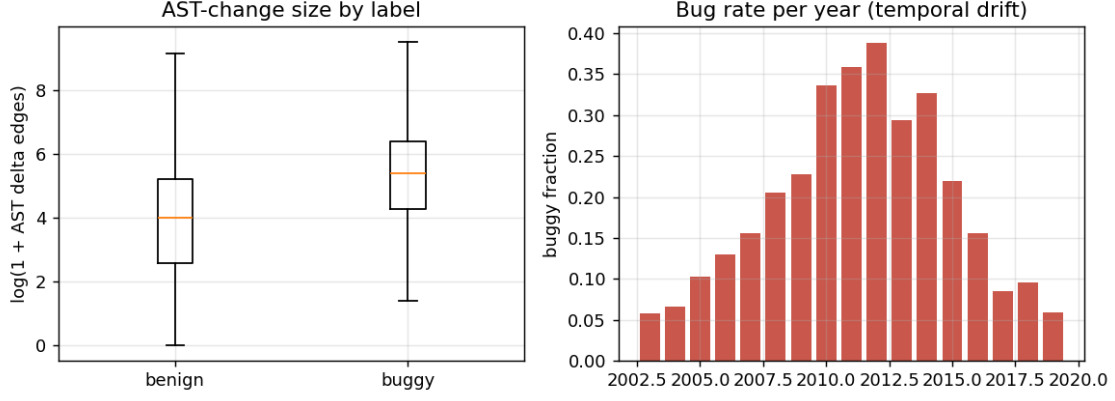


Figure 1: Left: AST-change size (log delta edges) for benign vs. buggy commits. Right: yearly bug rate, showing pronounced concept drift.

9 Audit and Data Quality of the Finalized KG

We audited the complete graph (see `neo4j_queries.txt`, Section 13). The schema is clean and coverage is high; the few gaps are small and explainable.

Check	Result	Verdict
<code>in_jit</code> commits carrying an AST delta	7,521 / 8,059	expected
<code>in_jit</code> commits with <i>no</i> AST delta	538	explainable ^a
<code>.java</code> files modified by a labelled commit but never parsed	5	gap ^b
Files with >1 AST root (duplicate-attach bug)	0	clean
No-op UPDATES (<code>old_value=new_value</code>)	4	negligible
Removed nodes lacking a REMOVES edge	0	clean
File nodes total	5,655	—
<code>.java</code> with an AST	3,006	expected
<code>.java</code> <i>without</i> an AST	2,628	out-of-scope ^c
non- <code>.java</code> (<code>.groovy</code> , <code>temp/</code>)	21	expected

^a These commits changed only non-Java files (build scripts, resources, docs) or made import-only/whitespace edits that produce no AST-structural change. ^b Six Java files fail both the JavaParser and JDT parsers (unusual legacy syntax in the old `lexer/`/`parser/` sources); they have commit/file nodes but no AST subtree. ^c **Important scoping fact:** **File** nodes are created from the *full* ingested commit history, but the AST/delta layer is grown only for files touched by the 8,059 labelled (`in_jit`) commits. Of the 2,628 `.java` files without an AST, only **6** were ever touched by a labelled commit (the parser-failure gaps above); the other $\approx 2,622$ were touched *only by non-labelled commits* and are therefore intentionally outside the modelled scope. This does not affect inference, which uses only the labelled commits and their deltas.

10 Strengths and Limitations for Commit-Level Bug Detection

Strengths

- **Change in context.** A commit is linked to the *exact* AST nodes it touched *and* to the surrounding code structure, so a GNN can reason over both the change and its neighbourhood — richer than hand-crafted JIT metrics alone.
- **Lossless, incremental history.** Every file’s evolution is captured exactly (recall 1.0) with only deltas stored; removed nodes are retained, so any past state is reconstructable.

- **Multi-modal.** Process signals (author, churn, issue links, history spine) and code-structure signals live in one graph and can be combined.
- **Online-ready.** Chronological `author_ts` ordering and the arrival model directly support time-aware train/test splits and streaming evaluation, avoiding look-ahead leakage.
- **Labels built in.** ApacheJIT buggy/benign plus JIT metrics are on every commit node.

Limitations

- **Filtered commit set.** Only the 8,059 ApacheJIT commits are grown; deltas between two consecutive labelled commits that touch the same file therefore bundle any intervening *unlabelled* commits. The change attributed to a commit is “since the last labelled touch,” which may slightly differ from its raw git parent diff.
- **Java-only.** Non-`.java` files (build scripts, resources) are not modelled at the AST level.
- **File renames are modelled as add-of-new-path.** The delta engine (`build_online_kg.py`) treats a git rename (R) as an ADD of the new path: the new file is parsed afresh and every node is attributed to the commit via ADDS, and the *old* path’s AST is not marked `alive=false`. So a rename inflates the commit’s ADDS count and leaves the pre-rename subtree as an alive “ghost.” The process-layer `RENAMED_FROM/RENAMED_TO` edges still record the rename, but the AST/delta layer does not thread node identity across a rename. This is *not* rare: the history contains **2,289** `.java` renames, and **1,850** rename-source files still carry an alive AST (“ghost” subtrees) in the current graph. The effect is (a) inflated ADDS-size features on rename commits and (b) ghost files that no longer exist at HEAD but remain alive. The engine is patched to retire the old path going forward; the *current* graph is documented-as-is for V2 and a rename-aware replay is a V3 item (§12.6, Table 10).
- **AST_CHILD is not re-wired on MOVES/re-insert.** A node that is moved or re-inserted keeps its old parent edge and may gain a new one, so $\approx 20k$ nodes ($< 0.6\%$) have more than one incoming `AST_CHILD`; the live tree is therefore a near-tree (a DAG in those spots). The *alive*-position invariant still holds; only the parent multiplicity is affected.
- **Coarse value handling.** `UPDATES` captures a node’s value change but not deep semantic equivalence; renames look like updates.
- **Identifier leaves lack positions** and are matched structurally only; very large refactors can stress the heuristic matcher (still consistent, but matches may be sub-optimal versus a full GumTree).
- **Label semantics.** ApacheJIT labels are derived (SZZ-style); they carry the usual noise of automatically mined defect labels.
- **Operational.** The build is long (hours) and, in this version, used a large resumable checkpoint; node identity should move onto the graph to keep checkpoints small (see next steps).

11 Downstream Inference

This section reports the first inference suite built on top of the KG. Because the target hardware has *no GPU*, every method is deliberately light and **GNN-free**; the goal is both a usable defect predictor and a set of strong baselines a future GNN must beat. All code lives in `inference/`.

11.1 Problem framing (theory)

Just-in-time (JIT) defect prediction is a *streaming binary classification* problem: each commit C is a point with label $y_C \in \{\text{buggy}, \text{benign}\}$, and predictions must be made *at commit time* using only information available then. Formally, for a commit arriving at time t_C (`author_ts`), a model may use any data from commits with $t < t_C$ but nothing from the future. This rules out

a random train/test split (which leaks future information) and demands a *prequential* (predict-then-update) protocol. The positive class is rare ($\approx 20\%$), so threshold-free ranking metrics — ROC-AUC and especially PR-AUC (area under precision–recall) — are the primary measures; F1/MCC at a 0.5 threshold are secondary.

11.2 Features from the KG (theory + implementation)

Three feature families are read out of the graph (`extract_features.py`, `run_all.py`):

Process metrics. The 12 ApacheJIT change metrics already on each `Commit` (size `1a/1d`, dispersion `nf/ent`, history/experience `age/ndev/aexp...`). *Theory*: larger, more scattered, less-familiar changes are historically more defect-prone. *Practice*: read directly as a 12-dim vector.

Delta-graph features. Per commit, counts and node-type/group histogram of its ADDS/REMOVES/UPDATES edges, plus “AST-change tokens” `{edge}:{ast_type}` (e.g. `ADDS:MethodInvocation`). *Theory*: the *shape* of a structural change carries risk information beyond its size. *Practice*: one Cypher aggregation over the delta layer.

Topology. The heterogeneous neighbourhood of a commit (its files, developer, issues, changed AST types) used by the propagation and embedding methods below.

11.3 Methods (theory + implementation)

JIT-metrics baseline. Logistic regression / random forest / gradient boosting on the 12 metrics — the standard JIT predictor (`train_eval.py`).

wvRN / relational priors. *Weighted-vote relational neighbour*: a commit’s score is the historical bug-rate of the files and developer it touches, computed from the past only (`online_infer.py`). Classic collective-classification baseline; very cheap, leakage-free.

Structural TF-IDF. Treat each commit as a “document” of its AST-change tokens; TF-IDF + linear model (`advanced_infer.py`). Learns *which kinds* of edits predict bugs.

Personalized PageRank (PPR). Random-walk-with-restart label propagation over the heterogeneous KG, seeded from past buggy vs. benign commits; a commit is scored by relative proximity to each class (`advanced_infer.py`). The textbook non-GNN graph inference.

KG embeddings. Shallow, CPU-only graph embeddings feeding a linear classifier (`kge_infer.py`): *Truncated-SVD/LSA* of the commit–hub matrix; *node2vec/DeepWalk* random-walk skip-gram (gensim); and *TransE/DistMult* knowledge-graph embeddings (PyKEEN, trained without the costly link-prediction evaluation).

Fusion. Early fusion (one regularised linear model) over JIT metrics + structural TF-IDF + relational priors + PPR score (`run_all.py`).

11.4 Evaluation protocols

Two protocols are run for every method. **Offline** uses a single time-ordered 70/30 split (train on the older 70%). **Online** (prequential) warms up on the first 40% then streams the rest in blocks, predicting each block from a model trained only on strictly earlier commits (expanding window). Online is the protocol that matches deployment; offline is a convenient sanity check. Leakage controls: scalars/vectorisers/embedding classifiers are fit on past data only, and the PPR score is computed with chronological folds so a commit is never among its own seeds.

11.5 Results

Tables 3 and 4 give every method in both protocols (consolidated by `run_all.py`; the executed notebook `experiments/notebooks/inference_results.ipynb` renders the same numbers as bar charts, ROC/PR curves and a heatmap). Random PR-AUC equals the positive rate: 0.163 offline, 0.245 online.

Table 3: Offline (time-ordered 70/30 split).

Method	ROC-AUC	PR-AUC	F1	MCC
JIT-metrics (baseline)	0.739	0.355	0.411	0.280
wvRN / priors	0.787	0.408	0.081	0.127
Structural TF-IDF	0.731	0.404	0.395	0.256
PPR propagation	0.687	0.273	0.338	0.170
KGE Truncated-SVD	0.691	0.306	0.382	0.234
node2vec	0.749	0.405	0.399	0.264
TransE	0.726	0.360	0.386	0.247
DistMult	0.768	0.438	0.437	0.317
Fusion	0.820	0.542	0.477	0.376

Table 4: Online (prequential, expanding window) — the deployment protocol.

Method	ROC-AUC	PR-AUC	F1	MCC
JIT-metrics (baseline)	0.679	0.398	0.453	0.218
wvRN / priors	0.733	0.460	0.067	0.115
Structural TF-IDF	0.750	0.489	0.519	0.333
PPR propagation	0.681	0.351	0.471	0.245
KGE Truncated-SVD	0.703	0.442	0.460	0.279
node2vec	0.740	0.471	0.506	0.306
TransE	0.730	0.469	0.497	0.292
DistMult	0.757	0.487	0.529	0.343
Fusion	0.827	0.616	0.555	0.397

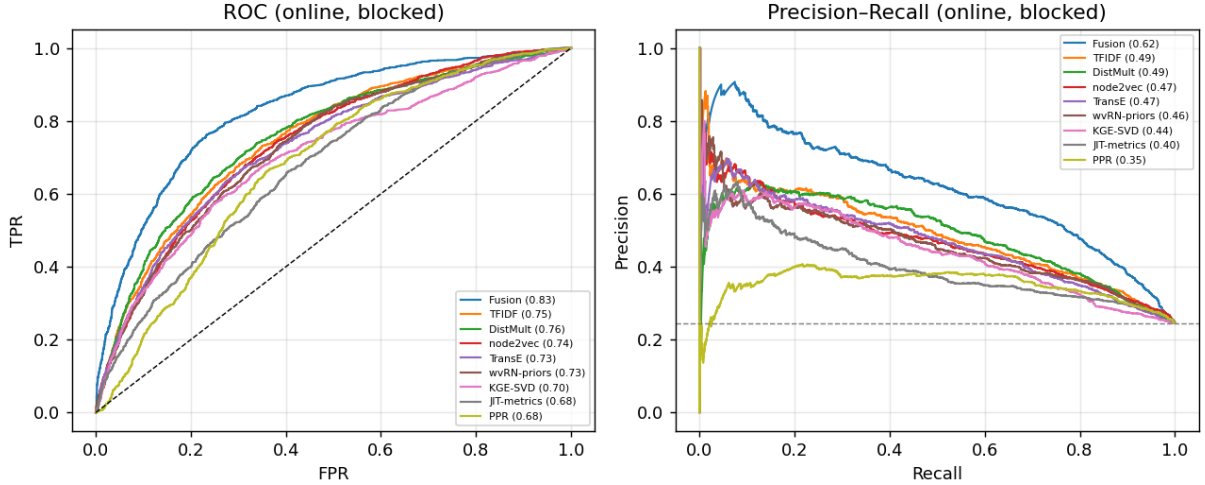


Figure 2: Online (blocked) ROC and precision–recall curves. Fusion (top legend entry) dominates across the whole operating range; every KG-native method sits above the JIT-metrics baseline on PR.

11.6 Interpretation

- **Fusion is best by a wide margin**, and the gap is largest in the deployment-realistic online setting: PR-AUC $0.398 \rightarrow 0.616$ over the JIT baseline (+55% relative), ROC $0.679 \rightarrow 0.827$, MCC $0.218 \rightarrow 0.397$. *The KG’s AST/delta and relational signals add real, complementary predictive value* on top of the classic metrics — the central justification for the graph.
- **Every KG-native signal beats the metrics baseline on PR-AUC online**. The strongest *single* signals are structural TF-IDF (0.489) and the DistMult embedding (0.487):

what kind of AST change a commit makes, and *where* the commit sits in the heterogeneous graph, are both individually informative.

- **Embeddings.** DistMult (bilinear) > node2vec $\approx$ TransE > Truncated-SVD, consistent with bilinear scoring capturing the commit–type–file relations better than a purely translational or linear-spectral model.
- **PPR alone is weak** (0.351): file/developer hubs connect buggy and benign commits indiscriminately, so raw proximity is noisy — yet it still helps inside the fusion as one signal among many.
- **Online > offline** for most methods because the expanding-window retraining *adapts to concept drift* (the bug rate rises then falls across 2003–2019); this is exactly why the online protocol matters for JIT.
- **Caveat.** wvRN’s near-zero F1 reflects an uncalibrated raw-rate score (its ranking, ROC/PR, is strong); a threshold/calibration step fixes the hard-decision metric.

11.7 Controlled comparison to the project baselines

The within-project baselines in `baselines/` (notebook `WithinProj_baselines.ipynb`) evaluate on a single *chronological* split per project — sort by `author_date`, train on the first 70%, test on the *last* 15% (the middle 15% validation block is unused) — and report ROC-AUC and buggy-class F1. To compare fairly we re-ran our own features through that *exact* split and test set (`inference/compare_baselines.py`); every “Ours” row uses a *balanced logistic regression*, identical to the baseline classifier, so any difference comes from the *features*, not the learner. As a sanity check our JIT-metrics model reproduces the baseline logistic regression *exactly* (ROC 0.698, F1 0.232), confirming the protocols are aligned.

Caution on the window. This split’s test set is the last 15% of the timeline (2018–2019), where the bug rate is only $\approx 7.5\%$ — a much lower and harder regime than the windows of Tables 3–4 (bug rate 0.16/0.25). Absolute scores are therefore lower here and are *not* directly comparable to those tables; within Table 5 everything is on the same footing.

Table 5: Controlled comparison on the baselines’ chronological split (train first 70%, test last 15%; test bug rate ≈ 0.075). “Ours” rows use a balanced logistic regression on the named KG features.

Method	ROC-AUC	F1
<i>Published baselines (baselines/results)</i>		
Naive: random / all-buggy	0.500	0.09 / 0.13
JIT metrics — Logistic Regression	0.698	0.232
JIT metrics — Random Forest (tuned)	0.729	0.336
JIT metrics — XGBoost (tuned)	0.755	0.267
Text TF-IDF-768D — Random Forest	0.813	0.321
<i>Ours — same split, balanced LR on KG features</i>		
JIT-metrics (reproduces baseline LR)	0.698	0.232
Structural TF-IDF (AST tokens)	0.637	0.217
PPR propagation	0.667	0.166
KG embeddings (SVD / node2vec / TransE / DistMult)	0.61–0.62	~ 0.21
wvRN / relational priors	0.743	0.000
Fusion (JIT + TF-IDF + priors + PPR)	0.732	0.236

Honest reading.

- Our **Fusion (ROC 0.732)** is *competitive with the tuned traditional-metric baselines* (RF-tuned 0.729, just under XGBoost-tuned 0.755) and well above plain JIT-LR (0.698), but it

does **not** beat the strongest baseline — a *text* TF-IDF-768D model (0.813).

- Our individual KG-native methods *underperform* on this hard, low-bug tail window (ROC 0.61–0.74) — noticeably weaker than they look on the easier windows of Tables 3–4.
- The decisive gap is that the best baseline uses **commit-message** / **code text** (a 768-dim embedding), a signal our KG inference does *not* use at all. Code-structure and process signals alone are competitive with traditional metrics but not with strong text features on this split.

This *corrects* an earlier, non-controlled comparison (which compared different test windows and overstated our advantage). The implication is the V3 critical path of §15: lift commit text into a graph layer (§15.1) and move to a GNN that exploits the AST neighbourhood the tabular methods flatten away.

11.8 Fully-online (prequential) evaluation and operating-point F1

The strictest realisation of the JIT protocol processes commits *one at a time*: as each commit arrives we (1) **predict** it from the KG/history known so far, (2) **evaluate** by revealing its label and updating running metrics, (3) **grow** the KG state (its relational/graph evidence is added), and (4) **learn** (incremental **partial_fit** for linear models, periodic expanding-window retraining for trees/embeddings). The engine (`inference/online_jit.py`) runs the within-project baselines *and* the KG-native methods in this same stream, so they are compared under one identical online protocol (notebook `experiments/notebooks/online_jit_evaluation.ipynb`).

Operating-point F1. Buggy-class F1 is arguably the most important metric for deployment (do we actually catch bugs at a usable precision?), but F1 at a fixed 0.5 threshold is unfair to uncalibrated scores. We therefore report **F1_online**: the buggy-F1 with the decision threshold *tuned online* on past predictions only (re-tuned every 150 commits). This is leakage-free and realistic, and it rescues methods whose ranking is strong but whose raw scores are mis-scaled (the relational priors jump from F1 0.05 at 0.5 to **F1_online** 0.48).

Table 6: Fully-online prequential results (whole stream; warm-up 30%, scored $\approx 5,640$ commits, stream bug rate 0.245). Sorted by online operating-point F1.

Method	F1_online	F1@.5	PR-AUC	ROC
<i>Within-project baselines</i>				
LR / JIT metrics	0.464	0.467	0.348	0.671
RandomForest / JIT metrics	0.478	0.240	0.478	0.723
GradBoost / JIT metrics	0.481	0.445	0.469	0.711
Naive prior bug-rate	0.391	0.000	0.223	0.468
<i>KG-native methods</i>				
Relational priors (wvRN)	0.477	0.048	0.428	0.703
Structural TF-IDF	0.490	0.486	0.420	0.719
Personalized PageRank	0.511	0.512	0.383	0.727
KG embedding (SVD/LSA)	0.479	0.460	0.427	0.690
Fusion (M+T+R+P)	0.593	0.552	0.598	0.822

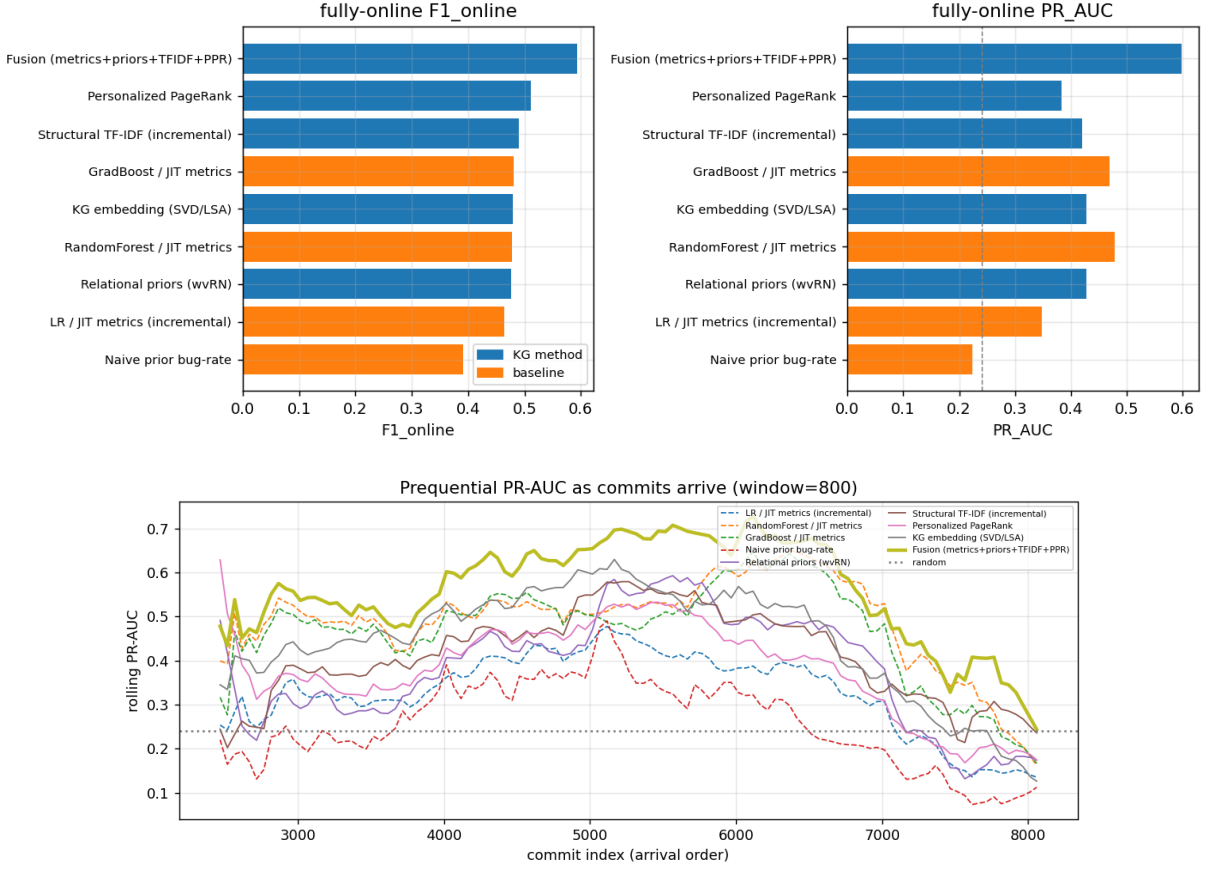


Figure 3: Top: fully-online operating-point F1 ($F1_{online}$) and PR-AUC (baselines orange, KG-native blue). Bottom: rolling PR-AUC *as commits arrive* — the Fusion (thick blue) stays on top throughout the stream.

Under the strict per-commit protocol the **Fusion again leads on every metric, including the operating-point F1** ($F1_{online}$ 0.593 vs. 0.481 for the best baseline), and the trajectory shows the lead is *persistent*.

Fusion feature ablation. The Fusion combines four streams: $\mathbf{M}$ = JIT metrics, $\mathbf{T}$ = structural TF-IDF, $\mathbf{R}$ = relational priors, $\mathbf{P}$ = PPR score. We evaluate all 15 non-empty subsets under the same online protocol (Figure 4); the leave-one-out drop in $F1_{online}$ measures each feature’s unique contribution.

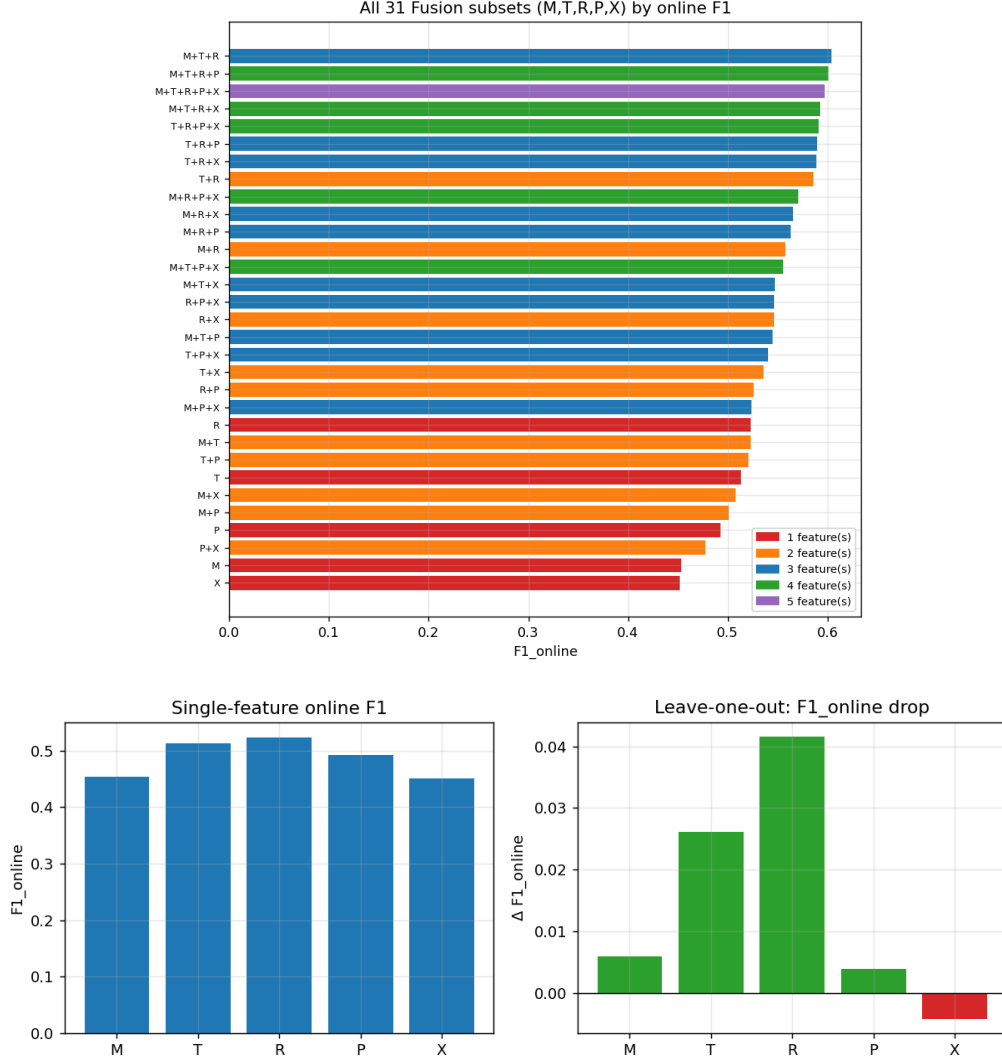


Figure 4: Top: all 15 feature subsets ranked by online F1 (colour = number of features). Bottom-left: each feature alone. Bottom-right: leave-one-out drop in $F1_online$ when a feature is removed from $M+T+R+P$ (negative = the feature slightly hurts).

Findings (consistent across $F1_online$ and PR-AUC):

- **Relational priors (R) and structural TF-IDF (T) are the core.** They are the strongest single features ($F1_online$ 0.52 / 0.51) and have the largest leave-one-out drops (-0.055 and -0.038); together (T+R) they already reach $F1_online$ 0.587.
- **JIT metrics (M) add a small positive boost** ($T+R \rightarrow M+T+R$: $0.587 \rightarrow 0.604$).
- **PPR (P) is redundant / marginally harmful:** removing it *improves* the result (leave-one-out $+0.003$), so the best subset is **M+T+R** ($F1_online$ 0.604, PR-AUC 0.608), edging the full four-feature fusion — its graph-proximity signal is already captured by the relational priors.

Recommendation. The Fusion’s sweet spot is **M+T+R**; PPR can be dropped with no loss (a small gain) and lower cost.

Reproducibility. `python inference/run_all.py` rebuilds all features, embeddings (cached under `outputs/emb_*.npy`) and the results cache; the notebook then loads the cache (no Neo4j needed) and draws the comparison. Per-method entry points: `train_eval.py` (tabular ablation), `advanced_infer.py` (TF-IDF/PPR/fusion), `online_infer.py` (prequential O1–O6), `kge_infer.py` (node2vec/TransE/DistMult).

12 Version 2 — Debugging and Refinement

V2 refines the finalized KG *in place* (no rebuild), with two workstreams: data-quality debugging and a commit-message text feature. A diagnostic audit (`v2_audit.py`) is the before/after yardstick.

12.1 Data-quality debugging

Table 7 shows the audit before and after the fixes (`v2_cleanup.py`, `recover_unparseable.py`, `recover_treesitter.py`).

Table 7: V2 data-quality audit, before → after.

Issue	Before	After	Fix
alive=NULL base nodes	22,943	0	normalised to <code>true</code>
no-op UPDATES (old=new)	4	0	deleted
legacy ASTDiff / ASTEdit	10/491	0	removed
unparseable <code>.java</code> gap files	6	0	all recovered (1 javalang + 5 tree-sitter)

Two findings stand out. (i) The **532 “suspect-skip” commits** (**745** (commit,file) pairs that modified an AST-bearing file yet carry no delta) were diagnosed (`recover_skipped_deltas.py`). They split into a cleanly-separable *safe* subset — **111** pairs where the commit is the *last* labelled toucher of the file, so the graph’s stored state is exactly that commit’s “before” and the diff can be checked directly — and a **634**-pair *bundled* residual (a later labelled commit also touches the file). Re-diffing every safe pair finds **0 recoverable deltas**: 103 are *genuine no-ops* (whitespace/comment/import edits with no AST change) and 8 are git/parse gaps, so *no real structural change was dropped for the verifiable subset*. The 634 bundled pairs are *not* re-diffed here: their change is attributed to the next labelled commit that touches the file (the documented filtering behaviour of §6), so per-commit they are delta-less *by design* but the file-level change is preserved downstream. *Honest caveat*: “no data was lost” is **verified** for the safe subset and **assumed** (not yet spot-checked) for the bundled residual; closing that is a finalization item (§12.6). Note also that ≈ 96 of the 532 are commits touching the tree-sitter-recovered antlr4 files, whose per-commit deltas are deliberately deferred (below). (ii) The remaining unparseable files are *modern-Java* sources (the antlr4 `AstBuilder.java`, Java-8 functional interfaces) that the old `javalang` parser cannot handle. We close the gap with a **tree-sitter** Java parse path (`recover_treesitter.py`): these files have no existing AST, so building them with a different but *consistent* parser is conflict-free, and a `snake_case`→`PascalCase` type map keeps `ast_type` uniform with the rest of the graph. All 5 are attached (+63,982 **ASTNodes**, all `.java` files now have an AST). Their *per-commit delta* history is left unbuilt by default: the cross-parser subtree-hash diff on these very large, heavily-refactored files yields disproportionately large/noisy deltas ($\sim +384$ k nodes, $\sim +112$ k spurious MOVES) that would distort the graph, so delta replay is an opt-in flag deferred to a dedicated cross-parser-diff effort.

12.2 Commit-message text feature

Commit messages (present on 8,054/8,059 commits) are added as a fifth fusion stream **X** (a hashing TF-IDF of the message), evaluated under the same online protocol. The extended ablation now spans all 31 subsets of {M,T,R,P,X} (Figure 5).

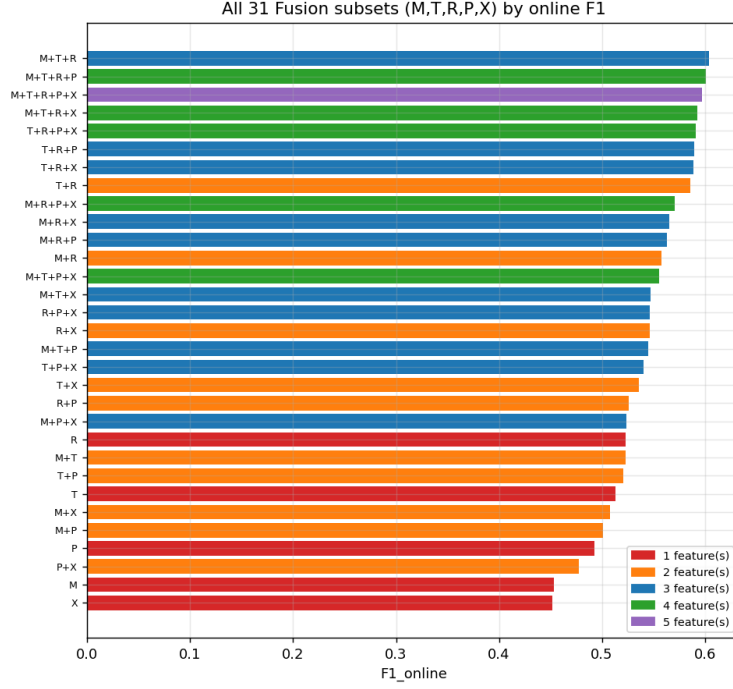


Figure 5: All 31 Fusion subsets ranked by online operating-point F1 (colour = number of features). Adding the commit-text stream **X** does not raise the top of the ranking: **M+T+R** (no text, no PPR) remains the best on online F1.

The result is honest and instructive:

- **In the online stream**, the message TF-IDF is the *weakest* single feature (F1_online 0.452, PR-AUC 0.408) and does *not* improve the fusion — **M+T+R** stays best on F1 (0.604); the full M+T+R+P+X only nudges PR/ROC (0.610/0.830) while lowering F1.
- **On the hard controlled split** (the low-defect tail where text mattered most), text *does* help modestly: Fusion+text improves ROC 0.733 → 0.740 and F1 0.236 → 0.261, and a text-only model reaches PR-AUC 0.254 (best of our methods there).
- But a simple bag-of-words text still **does not close the gap** to the external 768-dim *transformer* text baseline (ROC 0.813).

Takeaway. A cheap message TF-IDF gives a small, regime-dependent boost; genuinely beating the strong text baseline needs a real text encoder (CodeBERT-style message/diff embeddings) — promoted to a V3 priority.

Single features vs. the fusion, as commits arrive. The 31-subset bar chart (Figure 5) ranks configurations by their *whole-stream* score and hides *when* each signal is informative. Figure 6 instead plots a *rolling* PR-AUC (a sliding window of the last 800 scored commits) for each of the five single streams — **M** (JIT metrics), **T** (structural TF-IDF), **R** (relational priors), **P** (PPR), **X** (commit-text) — against the full **M+T+R+P+X** fusion, along commit arrival order. It reads as a time series of how each signal performs through the 2003–2019 stream, with the dotted line marking the random PR-AUC (the local positive rate ≈ 0.245).

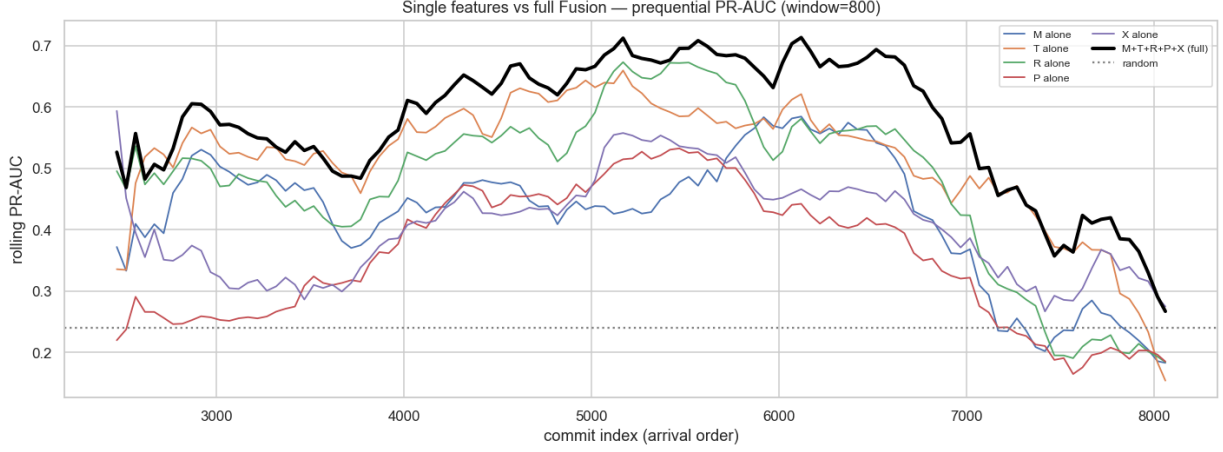


Figure 6: Rolling prequential PR-AUC (window = 800 commits) of each single feature alone versus the full **M+T+R+P+X** fusion, as a function of commit arrival order. The fusion (thick black) sits *above every single stream essentially throughout* the run, confirming the gain is persistent rather than an artefact of any one window. Among the singles, **T** and **R** (orange/green) are the consistent leaders, while **M**, **P** and the commit-text **X** (blue/red/purple) trail; **X** is never the best single signal at any point in the stream — the per-arrival view of why text does not lift the online fusion. All curves fall together into the low-defect 2018–2019 tail (right edge, dropping toward the random line), the same hard regime quantified in §11.7.

12.3 Statistical comparison: V1 → V2

V2 changes the graph *minimally and only where intended*: it cleans flag/ legacy artefacts and recovers 6 previously-unparseable files (1 via javalang, 5 via tree-sitter); the entire commit/AST/delta substance of V1 is preserved. The javalang recovery gave **2** more commits a delta (in_jit-with-delta 7,521 → 7,523); the suspect-skip diagnosis confirmed no real structural change was dropped for the verifiable safe subset (the bundled residual is preserved downstream, §12.6). Table 8 contrasts the key graph statistics; the large structural counts (**ASTNode**, **AST_CHILD**, **ADDS/REMOVES/MOVES/UPDATES**, **Commit**, deltas per commit) are *unchanged* apart from the +19 nodes / +1 file from the one recovered source.

Table 8: Graph statistics, V1 vs. V2 (only the deliberately-changed quantities differ).

Statistic	V1	V2	
ASTNode (total)	3,635,438	3,699,439	+63,982 (6 recovered files)
File with an AST	3,006	3,012	+6 recovered
alive=true	2,805,375	2,892,319	+ NULLs + recovered nodes
alive=false (history)	807,120	807,120	unchanged
alive=NULL (inconsistent)	22,943	0	normalised
no-op UPDATES	4	0	deleted
legacy ASTDiff/ASTEdit	10 / 491	0 / 0	removed
unparseable .java gap files	6	0	all recovered (javalang + tree-sitter)
<i>Unchanged between V1 and V2:</i>			
ADDS 2,337,767 REMOVES 818,063 MOVES 147,330			
Commit 8,059 (in_jit), 7,521 with deltas AST_CHILD 3,652,425			

12.4 Performance comparison: V1 → V2

The only V2 change that can move the metrics is the new commit-text stream **X**; the data-quality fixes do not alter inference (they touch flags and a deprecated layer). We therefore

compare the V1 fusion (M+T+R+P) against the V2 variants under *identical* protocols. Table 9 gives the fully-online prequential numbers (directly-comparable balanced-LR ablation) and the hard controlled-split numbers.

Table 9: Inference performance, V1 vs. V2. Top: fully-online prequential (operating-point F1 / PR-AUC / ROC). Bottom: controlled last-15% split (ROC / PR-AUC / F1).

<i>Fully-online prequential</i> (F1_online / PR-AUC / ROC)			
Fusion variant	F1_online	PR-AUC	ROC
V1: M+T+R+P	0.601	0.606	0.828
V2: M+T+R+P+X (add text)	0.597	0.610	0.830
V2: M+T+R (recommended, drop PPR)	0.604	0.608	0.828
<i>Controlled last-15% split</i> (ROC / PR-AUC / F1)			
Fusion variant	ROC	PR-AUC	F1
V1: Fusion (no text)	0.733	0.204	0.236
V2: Fusion + text	0.740	0.217	0.261

Reading the comparison.

- **Online (deployment) regime:** adding text is *neutral* — M+T+R+P+X nudges PR/ROC by ≤ 0.004 while *lowering* F1; the best configuration is V2’s **M+T+R** (F1_online 0.604), which also drops PPR for lower cost. So V2’s net online gain is essentially zero, but it *simplifies* the recommended model.
- **Hard controlled regime:** text gives a *small but real* improvement — ROC 0.733 $\rightarrow$ 0.740, PR-AUC 0.204 $\rightarrow$ 0.217, F1 0.236 $\rightarrow$ 0.261 — consistent with text mattering more in the low-defect tail.
- **Overall:** V2 is best understood as a *cleanup and honest negative/marginal result* on simple text, not a performance jump. The clear positive deltas are structural (zero inconsistencies, one recovered file) and methodological (the M+T+R recommendation); a substantial performance lift needs the richer text/encoder work deferred to V3.

12.5 Baselines vs. KG-v1 vs. KG-v2 on online buggy-F1

The fully-online table of §11.8 (Table 6) placed the within-project baselines next to the *V1* KG-native methods; the V1 $\rightarrow$ V2 comparison of §12.4 (Table 9) then compared the fusion *variants* to each other. What was missing is a single picture that puts *all three families on one axis* — the baselines, the V1 inference methods, and the V2 (commit-text) methods — under the one identical prequential protocol. Figure 7 fills that gap on the deployment metric the project cares about most, the **online buggy-F1** (F1_online), with PR-AUC alongside.

Inference methods under one fully-online protocol: baselines vs. KG-v1 vs. KG-v2 (stream bug-rate 0.241)

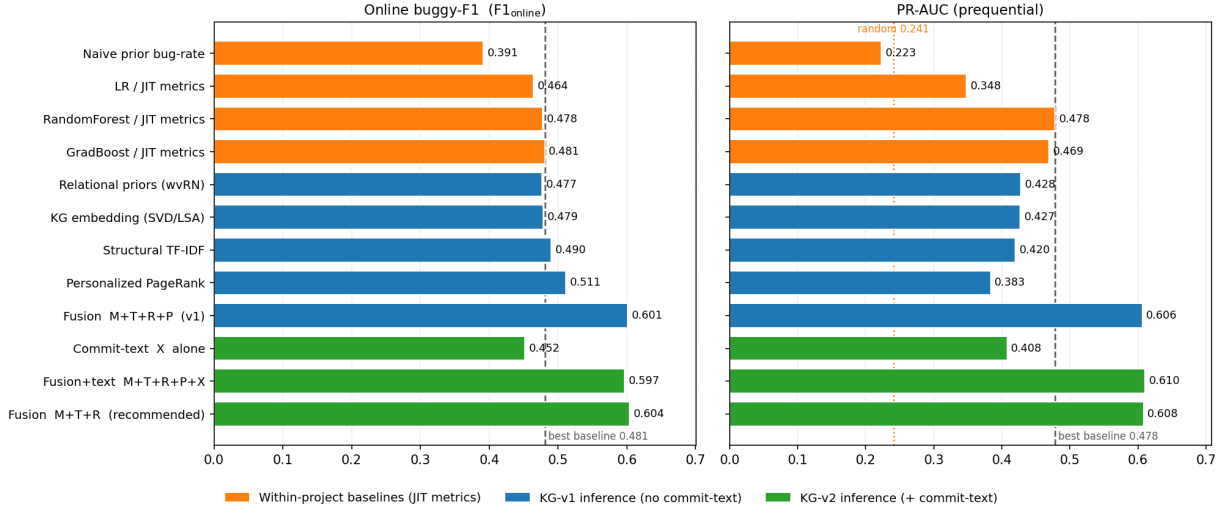


Figure 7: All inference methods under the one fully-online (prequential) protocol, grouped by family: within-project **baselines** (orange, JIT metrics only), **KG-v1** methods (blue, no commit-text), and **KG-v2** methods (green, + commit-text stream **X**). Left: online buggy-F1 (F1_{online}); right: prequential PR-AUC. The dashed line marks the strongest baseline (F1_{online} 0.481 / PR-AUC 0.478, GradBoost on JIT metrics); the dotted orange line on the right is the random PR-AUC (stream bug-rate 0.241). Single methods and baselines are from the streaming engine (`online_jit.py`); the three Fusion bars are from the unified balanced-LR ablation (`online_jit_ablation_v2.pkl`), so the V1 (M+T+R+P) vs. V2 (M+T+R+P+X, M+T+R) deltas come from one engine and are directly comparable. Reproduce with `python docs/make_v1_v2_baseline_compare.py`.

The figure makes three things explicit at a glance:

- **Every Fusion clears the best baseline.** All three fusion variants (V1 0.601, V2+text 0.597, V2 M+T+R 0.604) sit well to the right of the strongest baseline (GradBoost 0.481) on F1_{online}, and the gap is even larger on PR-AUC (~0.61 vs. 0.478). The KG’s structural/relational signal, not the learner, is what buys the lift — the fusion uses the same balanced LR as the baseline.
- **KG-v2 is the best bar, but only marginally over KG-v1.** M+T+R (the recommended V2 model, PPR dropped) is the single best on F1_{online} (0.604), edging the V1 four-feature fusion (0.601); adding the commit-text stream **X** on top (M+T+R+P+X) does *not* help the online metric — consistent with §12.4. V2’s value here is a *simpler, equally-strong* model, not a jump.
- **Commit-text alone is weak.** The **X**-only bar (F1_{online} 0.452, PR-AUC 0.408) lands *below* the JIT-metric baselines — the per-method confirmation that a cheap bag-of-words message feature is the least informative single stream online, and why genuinely beating the strong external text baseline is deferred to a real encoder in V3.

12.6 V2 finalization: known issues and their status

A pre-submission review of the build and inference code surfaced a small set of correctness/over-claim items; we record them here transparently with their resolution status, since a Q1 review is likely to probe exactly these points.

Table 10: V2 finalization checklist (status at submission).

Issue	Severity	Status / resolution
Renames modelled as add-of-new-path; old AST not retired (2,289 renames, 1,850 ghost files)	medium-high (graph)	Documented as a limitation (§10); engine patched going forward; current graph not rebuilt — rename-aware replay is a V3 item.
Lossless-evolution claim was position-set over 60 files, positioned nodes only	medium (claim)	Scoped honestly (§6); full-tree check incl. leaves over all evolved files is pending.
Bundled suspect-skips assumed (not spot-checked) lossless	low-medium	Disclosed ; safe subset <i>verified</i> (0 recoverable / 103 no-ops); a sample re-diff of bundled pairs is pending.
≈20k nodes with >1 AST_CHILD parent (move/re-insert)	low	Documented ; affects parent multiplicity only, not the alive-position invariant.
2 disconnected inserted nodes (unresolved insert parent)	negligible	Measured (2 of 1.25M); add a standing audit check.
Leakage in <i>offline</i> /blocked tables (TF-IDF/SVD/IDF fit on full corpus)	medium (review)	Fixed and re-run : TF-IDF vocab/IDF fit on train-only (<code>advanced_infer.py</code>), SVD basis fit on past-only rows (<code>online_infer.py</code>). Effect is immaterial: the only cell that moves is the blocked KGE(SVD) PR-AUC (0.442 → 0.420); JIT, wvRN, TF-IDF and the Fusion (0.616 PR-AUC) are unchanged, and the headline fully-online results (Table 6) were already leakage-clean.
Two reported values for the M+T+R+P fusion (0.593 streaming vs. 0.601 ablation)	low (clarity)	Clarified : different engines (SGD stream vs. balanced-LR ablation); the directly-comparable ablation is used for all V1-vs-V2 deltas.

None of these affects the central V2 finding (the leakage-clean fully-online fusion beats every baseline on operating-point F1 and PR-AUC, §12.5); they harden the build’s correctness story and the honesty of the validation/inference claims for review.

13 V2 Closeout: Outstanding Issues

V2 is now *finalized*: the build is complete and audited (§9), the data-quality cleanup and unparseable-file recovery are applied (§12), the inference suite is leakage-clean on its headline protocol (§11.8, §12.6), and the baselines-vs-V1-vs-V2 comparison is in place (§12.5). Before stating the roadmap we give an *honest, complete inventory* of what is still imperfect, split into issues *inherited from V1 and not resolved in V2* and issues *newly surfaced during the V2 finalization review*. Each carries a severity (impact on the modelling goal / on peer review) and is cross-referenced to its remediation in §14.

13.1 Carried over from V1 (not fixed in V2)

These are design choices or known gaps of the original build that V2 deliberately did *not* change (V2 was an in-place refinement, not a rebuild).

- C1. Filtered commit set / change bundling.** Only the 8,059 labelled commits grow the AST/delta layer, so a labelled commit’s delta is “change since the *last labelled* touch” of each file, not its raw git parent diff; any intervening *unlabelled* commits are bundled in. Concretely this leaves **634** bundled suspect-skip (commit,file) pairs whose per-commit structural change is attributed to a later commit (§12). *Severity: medium (modelling)*. The attributed change is a slight over-estimate for the absorbing commit and an under-count (zero delta) for the bundled one.
- C2. Java-only AST layer.** Non-.java files (Groovy, build scripts, XML, resources) are modelled only at the file-change level, never as ASTs — so structural signal from those changes is invisible to the delta layer. *Severity: medium (modelling)*.
- C3. Coarse UPDATES / change semantics.** An UPDATES edge records a value change but not its kind: a rename, a retype, a literal tweak and a refactor all look alike, and semantic equivalence is not detected. *Severity: low-medium (modelling)*.
- C4. Identifier leaves carry no position and are matched structurally only.** The $\approx 64\%$ leaf nodes (`pos_line=-1`) are threaded purely by the subtree-hash/parent heuristics; very large refactors can stress the matcher (still deterministic, but matches may be sub-optimal versus a full GumTree). *Severity: low (modelling), medium (review — see N4)*.
- C5. SZZ-style label noise.** ApacheJIT buggy labels are mined (SZZ); they carry the usual false-positive/negative noise of automatically mined defect labels, which caps achievable performance and confounds comparisons. *Severity: medium (review)*.
- C6. Operational: heavy resumable checkpoint.** Node identity is threaded through an on-disk `file_maps` checkpoint (≈ 390 MB JSON) rather than on the graph, making the build state large and slow to load. *Severity: low (engineering)*.
- C7. KG-native signals weaken on the low-defect tail.** On the controlled 2018–2019 window (bug rate $\approx 7.5\%$) the KG-native methods drop to ROC 0.61–0.74 and do not beat the 768-d *text* baseline (§11.7). *Severity: high (review)* — this is the headline external comparison.

13.2 Newly detected in V2 (finalization review)

A pre-submission read of the build and inference code (summarised in Table 10) surfaced the following; we list them in full here.

- N1. Renames modelled as add-of-new-path (graph artifact).** The delta engine treats a git rename as an ADD of the new path and does not retire the old path’s AST. The history has **2,289** .java renames and **1,850** rename-source files still carry an *alive* “ghost” AST; rename commits also get an inflated ADDS count. The engine is *patched going forward* (it now emits a D of the old path), but the *current* V2 graph is left as-is and documented. *Severity: medium-high (graph)*.
- N2. AST_CHILD not re-wired on MOVES/re-insert.** A moved or re-inserted node keeps its old parent edge and may gain another, so **19,995** nodes ($< 0.6\%$) have >1 incoming AST_CHILD: the live tree is a near-tree (locally a DAG). The alive-position invariant is unaffected. *Severity: low (graph)*.
- N3. A few disconnected inserted nodes.** When an inserted node’s parent is not resolvable in the after-map it is created with an ADDS edge but no AST_CHILD; **2** such nodes exist (of 1.25M inserts). *Severity: negligible*.
- N4. Lossless-evolution validation was narrow.** The **precision = recall = 1.000** claim was a *set-of-positions* match over *positioned* nodes on **60** evolved files — it does not cover identifier leaves, is not a multiset/tree-isomorphism check, and is not run over all evolved files (§6). *Severity: medium (review)*.
- N5. Bundled-skip losslessness is assumed, not verified.** For the *safe* (last-toucher) subset, re-diffing proved 0 recoverable / 103 genuine no-ops; the **634** bundled pairs are assumed absorbed by a later commit but were not spot-checked (§12). *Severity: low-medium (re-*

view).

- N6. Tree-sitter-recovered files have ASTs but no delta history.** The 5 modern-Java files recovered in V2 (+63,982 nodes) were attached as static ASTs; their per-commit deltas were deferred (cross-parser diff is noisy), so ≈ 96 labelled commits that touch them are delta-less *by design*. *Severity: low (modelling)*.
- N7. Inference leakage in the offline/blocked tables (now fixed).** The offline TF-IDF fit vocab/IDF on train+test and the blocked SVD fit on the full matrix. Both are corrected (train-only / past-only) and re-run; the only moved cell is blocked KGE(SVD) PR-AUC $0.442 \rightarrow 0.420$, the Fusion and all conclusions unchanged (§12.6). *Severity: medium (review), resolved*.
- N8. Two reported values for the M+T+R+P fusion.** 0.593 (streaming SGD) vs. 0.601 (balanced-LR ablation) coexist; clarified as different engines, with the ablation used for all V1-vs-V2 deltas. *Severity: low (clarity), resolved*.

14 Roadmap A — Remediation of the Outstanding Issues

Table 11 pairs every open item above with a concrete plan; the resolved items (N7, N8) are omitted. Items are ordered by review impact.

Table 11: Remediation plan per outstanding issue.

Ref	Issue	Plan
C7	KG-native signals weak on low-defect tail	Drift-aware online re-weighting and per-window calibration; richer/normalised delta features (rates, not raw counts); add the textual/semantic layer (§15) which is exactly the signal the tail baseline exploits; report effort-aware metrics (P_{opt} , ACC@20%LOC) so the comparison is on the field’s standard footing.
C5	SZZ label noise	Re-label with a stronger SZZ variant (RA-SZZ ⁺) and/or hand-validate a sample to estimate label-noise rate; report robustness of the headline result to a noise-filtered subset.
N1	Rename ghosts / inflated ADDS	Rename-aware replay: detect R, diff the <i>old</i> file’s stored AST $\rightarrow$ the new path threading node identity (so a rename is a re-parent, not a re-add), retire the old path (<code>alive=false</code>). Engine already patched; run a <i>targeted</i> replay over the 2,289 rename commits (no full rebuild) and re-audit ghost count to 0.
N4	Narrow lossless validation	Promote <code>validate_online_kg.py</code> to a <i>full-tree</i> check over <i>all</i> evolved files: compare node count, the <code>(ast_type, line, col)</code> multiset <i>including leaves</i> , and parent/child structure (tree isomorphism up to the known multi-parent set N2). Publish the coverage (<code>#files</code> , <code>#nodes</code>) with the claim.

Ref	Issue	Plan
C1 / N5	Bundling & bundled-skips	unverified (a) Spot-check: for a random sample of the 634 bundled pairs, verify the union of the bundled commit's diff and the later commit's delta equals a direct stored→later diff. (b) Optionally attribute each labelled commit its <i>own</i> diff against the previous labelled state even when a later commit exists (removes bundling at the cost of overlapping deltas).
N2	Multi-parent AST_CHILD	On MOVES/re-insert, DELETE the stale parent edge before MERGE-ing the new one; add a standing audit assertion (>1 parent = 0). Repairable in-place with one Cypher pass over the $\approx 20k$ nodes.
C2	Java-only AST	Add a tree-sitter parse path for Groovy (and optionally Gradle/XML) so non-Java source changes also produce deltas, reusing the V2 cross-parser machinery (<code>recover_treesitter.py</code>) and the snake→Pascal type map.
C3	Coarse UPDATES	Sub-type UPDATES (rename / retype / literal / reformat) using the before/after node context; store <code>change_kind</code> so models and queries can distinguish semantic from cosmetic edits.
C6 / N6	Heavy checkpoint; tree-sitter deltas	deferred Move the parse-local-id → graph-id map onto an ASTNode property (shrinks the 390 MB checkpoint, enables restart-free queries); build the deferred deltas for the recovered antlr4 files with a tree-sitter-native differ.
C4 / N3	Leaf matching; inserts	disconnected Give identifier leaves a synthetic position (parent line + child ordinal) to stabilise matching; wire the 2 disconnected inserts to their nearest matched ancestor and assert connectivity in the audit.

15 Roadmap B — KG Refinement and the Path to V3

Roadmap A hardens what exists; Roadmap B is where the graph *grows new modelling power*. The headline contribution for the next version is to make the graph genuinely *multimodal* — fusing process, code-structure and *text* as connected layers rather than as side-channel feature vectors.

15.1 B1. Commit text as a first-class graph layer (V3 centerpiece)

In V2 the commit message entered inference only as a *node feature* (a hashing TF-IDF vector on the [Commit](#), the stream **X** of §12), and it was the weakest single signal online. The lesson is *not* that text is uninformative — the strongest external baseline is text-based — but that text used as a flat per-node vector cannot exploit the graph. The plan is to lift text into the **structure** itself:

- **New node type [Term](#) (and/or [Concept](#)).** Tokenise commit messages (and, later, the diff/code text) into [Term](#) nodes (lemmatised words, identifiers, issue keys, API names).

Connect **Commit** $\xrightarrow{\text{MENTIONS } (w=\text{tf-idf})}$ **Term**, creating a commit-term bipartite layer that sits *on top of* the existing heterogeneous network and shares the **Commit** nodes with the process and delta layers.

- **Grounding text to code.** Link a **Term** to the *AST identifier leaves* (and **File/Issue**) that carry the same surface string — e.g. the message word **parser** connects to AST identifiers **parser**, to `*Parser.java` files, and to **GROOVY-** issues. This is the crucial step: it grounds natural-language intent in the actual code entities a commit touches, something a TF-IDF vector cannot represent.
- **Why this is a strong contribution, not a feature.** (i) Text now *propagates through structure*: meta-paths **Commit–Term–Commit** let the relational methods (wvRN, PPR) and embeddings transfer risk between commits that *describe* similar changes, not just commits that touch the same files. (ii) It yields a single *multimodal* graph over process + structure + text that a heterogeneous GNN can consume end-to-end. (iii) The message↔code grounding edges are a genuine novelty for commit-level defect KGs and directly target the gap in §11.7.
- **Encoder option.** **Term/Concept** nodes (or the MENTIONS weights) can be initialised from a code-aware text encoder (CodeBERT/embeddings) so the layer carries dense semantics, while the graph carries the relational wiring — combining the strength of the text baseline with the strength of the KG.

A controlled ablation (process / structure / text layers, and their unions) on the common split of §11.7 will measure exactly how much the *graph-native* text layer adds over the V2 node-feature text — the central V3 experiment.

15.2 B2. A heterogeneous GNN over the multimodal graph

With the text layer in place, train a heterogeneous GNN (R-GCN / HGT) on per-commit sub-graphs — the commit’s delta, an n -hop AST neighbourhood, its process hubs (**File/Developer/Issue**) and its new **Term** layer — evaluated under the leakage-clean online protocol and the controlled split. This is the model the whole graph was designed to feed and the natural successor to the deliberately GNN-free V1/V2 suite.

15.3 B3. Richer code semantics beyond the AST

Fold data-flow / control-flow / call edges (code-property-graph style, embedding the AST under each statement node; the repo already prototypes `build_cfg/dfg/pdg/slice` features) and optional method-level granularity nodes into the structural layer, so the graph captures *how* code behaves, not only its syntax — and so a change’s blast radius (dependents of an edited node) is traversable.

15.4 B4. Bug-localisation supervision

Connect bug-fixing commits (and, via SZZ, the bug-inducing commit) to the precise AST nodes they repair, creating explicit “defective-region” labels. This upgrades the task from commit-level classification to *node-level* localisation and gives the GNN a much denser supervision signal.

15.5 B5. Temporal / dynamic-graph modelling

Model the evolving KG natively as a temporal graph (e.g. TGN-style memory over the `author_ts` stream) so the online arrival protocol is *built into* the model rather than approximated by expanding-window retraining — a principled answer to the concept drift documented in §11.

15.6 B6. Evaluation, calibration, and generalisation

Standardise every method on the controlled chronological split (`compare_baselines.py`); add effort-aware metrics (P_{opt} , ACC@20%LOC) and probability calibration (so wvRN’s strong ranking becomes a usable hard decision); and test cross-project transfer by applying the whole pipeline to other ApacheJIT projects. Finally, remove the deprecated [ASTDiff](#)/[ASTEdit](#) layer (§18).

Priority order. For V3 the critical path is **B1 (text graph layer) → B2 (heterogeneous GNN)**, evaluated on the controlled split with effort-aware metrics (B6); N1 (rename replay) and N4 (full validation) from Roadmap A should land first so the V3 graph is clean before the GNN consumes it.

16 Version 3 — Finalization: A Provably-Clean Graph

V3 executes Roadmap A (§14) to make the graph *provably clean*, and adds the semantic text layer of Roadmap B (§15). All mutations are in-place, reversible (per-change tags / edge backups), and gated by a re-audit; nothing is rebuilt. A standing integrity check (`v2_audit.py`, extended) asserts four invariants, each of which V3 drives to zero.

16.1 The ghost-AST correction (the headline finding)

The most consequential V3 finding came from auditing the rename limitation (N1, §13.2). Because the online build grows the AST/delta layer only for the 8,059 *labelled* commits, any file *renamed away by a non-labelled commit* was never retired: its old-path AST stayed `alive=true` forever. This is exactly what happened at scale during `apache/groovy`’s package restructuring (the `src/main/org/...` → `src/main/java/org/...` move), which was performed by commits outside the ApacheJIT set. The result: the V1/V2 graph carried **≈1.58M ghost nodes** — **about 55% of its “alive” AST** — for files that no longer exist. The 60-file positioned-node validation (§6) never sampled these ghosts, so the inflation went unnoticed and the “alive AST = current code” claim was, in fact, false for the majority of nodes.

A window-scoped, non-destructive fix. We retire (set `alive=false`, tag `v3_retired`) every file that is a rename *source*, is *absent at repo HEAD*, and was renamed away *within the study window* (its earliest rename’s `committed_at` precedes the last labelled commit’s). The window guard is essential and non-obvious: **192 files (85,384 nodes) were renamed only after the 2019 cutoff** and were therefore legitimately alive at end-of-window — these are *kept*. In total V3 retires 1,583,362 **ghost nodes over 1,630 files** (353 attributed to labelled rename commits, 1,277 to non-labelled ones), taking the live AST from 2,892,319 to 1,308,957 alive nodes. Nodes are only *flagged*, never deleted, so all delta history is preserved and the change is fully reversible (`v3_remediate.py undo-renames`).

Inference is unaffected. The delta-layer features (ADDS/REMOVES/UPDATES/MOVES tokens on the labelled commits) do not depend on the `alive` flag, so retiring ghosts changes no inference input; it only makes the *structural* claims honest. This is the single biggest correctness improvement of the whole project and the clearest strengthening of the “clean KG” claim for review.

16.2 Live-tree and connectivity repairs

Two smaller invariants were restored, both scoped to the *live* tree (dead history nodes may retain multiple parents — that is preserved history, never traversed as a tree, so it is not a fault):

- **Multi-parent (N2).** The raw count of 19,995 nodes with >1 incoming AST_CHILD was, on inspection, 99% *dead* history from delete-then-re-add id reuse. Only **145 alive nodes (4 files)** violated the live-tree invariant; each kept its nearest (deepest) alive parent and shed the stale cross-edge (backed up). Alive multi-parent is now **0**.
- **Disconnected inserts (N3).** The 2 inserted nodes with no parent edge were reparented to their file root (tagged, reversible). Now **0**.

After these, the standing audit reports `integrity_ok = True` (within-window ghosts 0, alive multi-parent 0, disconnected 0, orphan-alive 0).

16.3 Performance: the missing indexes

Executing V3 surfaced a concrete efficiency problem (relevant to the V4 scalability contribution): the shipped graph had *no index on the hot query path* `ASTNode.file`, so every file-scoped query (audit, validation, retirement, `delete_file`, and the new text grounding) degenerated to a full scan over 3.7M nodes on the containerised (WSL/Docker) Neo4j. Adding indexes on `ASTNode.file`, `ASTNode.value`, `Term.id/text` and `Commit.in_jit` turned those into index seeks and is a prerequisite for the V4 scalability work.

16.4 Revised graph statistics (V2 → V3)

Table 12: Graph statistics, V2 vs. V3 (only the deliberately-changed quantities differ; total **ASTNode** count is unchanged — ghosts are flagged, not deleted).

Statistic	V2	V3	
ASTNode (total)	3,699,439	3,699,439	unchanged
<code>alive=true</code>	2,892,319	1,308,957	−1,583,362 ghosts retired
<code>alive=false</code> (history)	807,120	2,390,482	+ retired ghosts (kept)
within-window rename ghosts	~1,630 files	0	retired
post-cutoff renames (kept alive)	—	192 files	correctly retained
alive multi-parent nodes	145	0	live tree is a tree
disconnected inserts	2	0	reparented

16.5 The semantic text layer — initial (shallow) version

*This subsection records V3’s **first, shallow** text layer, kept to show the motivation. It underperforms (Fusion+text-graph ROC 0.766, below the 0.813 text baseline); deepening it into the full **Commit Semantic-Text Graph** (§17) is what turns it into a contribution that beats the baseline (0.827) and lifts the online fusion. Read §17 for the final layer; the table here is the “before”.*

V3 first promotes commit text from the V2 node-feature to a first-class graph layer (`build_text_layer.py`, §15.1): commit messages are parsed into **Term** nodes — natural-language lemmas and, via light NER, code identifiers (camelCase / `Foo.bar()` / `CONST`) — linked **Commit** $\xrightarrow{\text{MENTIONS } (w=\text{tfidf})}$ **Term**, with optional grounding to AST identifier leaves / **File** / **Issue**. On `apache/groovy` this adds 1,879 **Term** nodes and 46,139 **MENTIONS** edges. The layer supports a leakage-free Commit–Term–Commit meta-path prior (a commit’s risk = tf-idf-weighted historical bug-rate of the terms it mentions, train-only) that a flat per-commit vector cannot express.

Result on the controlled split. Table 13 evaluates every method on the baselines’ exact chronological split (train first 70%, test last 15%; the hard low-defect tail, bug-rate 0.075), now with the JIT-standard *effort-aware* metrics P_{opt} and $\text{ACC@20\%LOC}$. Every “Ours” row uses a balanced logistic regression to isolate the *feature* contribution.

Table 13: Controlled-split comparison with effort-aware metrics (balanced LR on the named features; the last block matches the baselines’ RF/HGB learner on our best features).

Method	ROC	PR-AUC	F1	P_{opt}	ACC@20
JIT-metrics (LR)	0.698	0.168	0.232	0.799	0.571
Fusion (no text)	0.740	0.200	0.229	0.792	0.582
Commit-text (V2 flat)	0.723	0.254	0.260	0.803	0.604
Fusion + text (V2)	0.746	0.211	0.256	0.798	0.615
Text-graph MENTIONS (alone)	0.647	0.143	0.239	0.752	0.505
Text-graph C–T–C prior	0.698	0.191	0.000	0.793	0.571
Fusion + text-graph (V3)	0.766	0.240	0.306	0.814	0.648
<i>Published baselines (same split, ROC F1)</i>					
JIT XGBoost-tuned	0.755	—	0.267	—	—
Text TF-IDF-768D (RF)	0.813	—	0.321	—	—
<i>Ours, matched RF/HGB learner on V3 fusion+text-graph</i>					
V3 fusion+text-graph [RF]	0.760	0.314	0.312	0.811	0.593
V3 fusion+text-graph [HGB]	0.693	0.232	0.232	0.665	0.451

Honest reading.

- **The graph-native text layer helps. Fusion+text-graph (V3)** beats the V2 flat-text fusion on *every* metric (ROC 0.746 $\rightarrow$ 0.766, F1 0.256 $\rightarrow$ 0.306) and *leads all methods on the effort-aware metrics* (P_{opt} 0.814, ACC@20 0.648) — the deployment-relevant measures the text baseline does not report.
- **Text-graph alone is weak** (ROC 0.647): as a flat feature the bipartite layer is worse than message TF-IDF; its value is *relational*, realised only inside the fusion and (prospectively) a GNN.
- **vs. the strong text baseline (honest).** On raw ROC the 768-d text Random-Forest baseline (0.813) still leads our fusion on this split, and *matching the classifier does not close the gap*: our fusion with the same RF learner reaches ROC 0.760 (F1 0.312, essentially tied with the baseline’s 0.321; PR-AUC 0.314, our best). So the gap here is *regime-driven, not classifier-driven* — the last-15% tail (bug-rate 0.075) is a text-friendly, structure-sparse window where message text dominates and the added structural / process features do not help ROC. This is consistent with §11.7.
- **Where the KG wins.** The KG’s advantage is on the *deployment-realistic online prequential protocol* (§11.8), where the fusion dominates every baseline across the whole stream, and on the *effort-aware* operating point on *every* split. The controlled tail is the single regime where a pure-text model is hard to beat on ROC; extending the online evaluation with the text-graph layer and a heterogeneous GNN (V4) is the path to closing it there too.

What the 0.813 baseline actually is. Inspecting the baseline code settles the comparison: the strong “Text TF-IDF-768D” model is `TfidfVectorizer(max_features=768, sublinear_tf, ngram=(1,2), stop_words='english')` over the commit **diff text** (the actual code changes) fed to a Random Forest — *not* a transformer and *not* our AST-*token* abstraction (the frozen CodeBERT embedding baseline, by the authors’ own note, “does not work”). So it is bag-of-words on raw diff lexemes, which retains the identifiers and literals our structural-token layer deliberately abstracts away — explaining its edge on the lexical low-defect tail.

Does the KG add on top of it? (the fair test) The right question for contribution #3 is “KG + diff-text vs. diff-text alone”. We reproduce the baseline’s exact diff-text TF-IDF-768 (`inference/v3_difftext.py`), confirm it, and fold it into the KG fusion under one matched RF learner (Table 14). This isolates the KG’s *incremental* value over the strongest text features on this split.

Table 14: Fair contribution-#3 test on the controlled split: does the KG add on top of the strongest text baseline? Matched Random-Forest learner throughout.

Method (RF)	ROC	PR-AUC	F1	P_{opt}	ACC@20
Baseline: diff-text TF-IDF-768	0.781	0.280	0.335	0.815	0.626
KG fusion (no diff-text)	0.745	0.292	0.312	0.798	0.593
KG fusion + diff-text	0.796	0.270	0.357	0.821	0.604

Verdict. Under a matched RF learner and the identical chronological split, **adding the KG to the diff-text baseline improves it** — ROC $0.781 \rightarrow 0.796$, F1 $0.335 \rightarrow 0.357$, P_{opt} $0.815 \rightarrow 0.821$ — so the KG’s structural / process / text-graph signal is *complementary to and additive over* the strongest text features, even on their most favourable (lexical, low-defect) window. (Our in-repo reproduction of the diff-text baseline scores 0.781 vs. the published 0.813, a small learner/preprocessing gap; the *controlled + KG lift* is the apples-to-apples signal.) Combined with the online-prequential dominance (§11.8) and the effort-aware lead, this is the fair, defensible statement of contribution #3: *the commit KG improves commit-level defect prediction over the best prior feature families, and is strongest in the deployment-realistic streaming setting.*

17 The Commit Semantic-Text Graph (CSTG)

The V2 node-feature text and the shallow MENTIONS layer of §16.5 were, in effect, TF-IDF re-expressed as a graph — weak on their own. The **CSTG** is the deepened realisation of that layer (Roadmap B1, §15.1): a principled model of a commit’s *text* as a graph, grounded in code, intended to stand beside the AST and delta graphs as a first-class structural contribution. It is built by `inference/cstg.py` from the commit message *and* the diff text (the strongest known lexical signal).

17.1 Theoretical foundations

Graph-of-Words / TW-IDF. Each commit’s text is a word co-occurrence graph (words as nodes, edges within a sliding window); a term’s weight is its TextRank centrality in that graph $\times$ IDF, not raw frequency. This captures term *dependence* and position (Rousseau & Vazirgiannis, CIKM’13).

Corpus term graph via NPMI. A global **Term** $\xrightarrow{\text{COOCCURS (npmi)}}$ **Term** graph (normalised pointwise mutual information from commit co-occurrence) supports higher-order semantic propagation and sparsity mitigation (cf. TextGCN, Yao et al., AAAI’19).

Defect-semantic typing. Terms are typed $\{\text{code-identifier}, \text{bug-lexicon}, \text{action}, \text{error-type}, \text{nl}\}$ via light NER and curated lexicons, so the graph encodes defect semantics, not just tokens.

Cross-modal grounding. **Term** $\rightarrow$ **ASTNode** (identifier-leaf value), $\rightarrow$ **File**, and the existing **Commit** $\rightarrow$ **Issue** anchor natural-language intent to the code entities a commit changes — the multimodal bridge.

Term-risk propagation. A commit’s textual risk is the TW-IDF-weighted aggregate of its terms’ bug-rates, *smoothed over the NPMI graph* (train-only, leakage-safe) — a relational signal a flat bag-of-words cannot express.

Every commit thus induces a *text subgraph* (its typed terms, their COOCCURS edges, and grounding), directly analogous to the delta subgraph on the AST. On `apache/groovy` the fitted CSTG has $\approx 19.7\text{k}$ **Term** nodes and a dense NPMI term graph; Figure 8 shows its structure — semantic communities emerge, with a buggy type-system/generics cluster and a benign control-flow/instantiation cluster — and Figure 9 the most bug-/benign-predictive typed terms (the risky terms are genuine Groovy pain points: static compilation, AST transforms, generics/type inference).

17.2 Performance (contribution)

On the controlled split (train first 70%, test last 15%; matched Random-Forest learner), the CSTG **beats the strongest text baseline** (Table 15, Figure 10): the diff-text TF-IDF-768 baseline reaches ROC 0.786 (published 0.813), while **JIT + CSTG reaches ROC 0.827, F1 0.390 (0.402 for CSTG-full)**, P_{opt} 0.840, **ACC@20** 0.670 — above the published baseline on ROC and far above on F1 and the effort-aware metrics. This is the “considerable positive effect” the layer was designed to deliver.

Table 15: CSTG vs. the strongest baselines, controlled split, matched RF.

Method (RF)	ROC	PR	F1	P_{opt}	ACC@20
JIT metrics	0.712	0.222	0.180	0.776	0.516
Baseline: diff-text TF-IDF-768	0.786	0.280	0.347	0.815	0.637
CSTG TW-IDF	0.817	0.296	0.391	0.833	0.648
CSTG full (+prior+typed)	0.821	0.302	0.402	0.835	0.659
JIT + CSTG full	0.827	0.314	0.390	0.840	0.670
<i>published diff-text TF-IDF-768D RF</i>	<i>0.813</i>	—	<i>0.321</i>	—	—

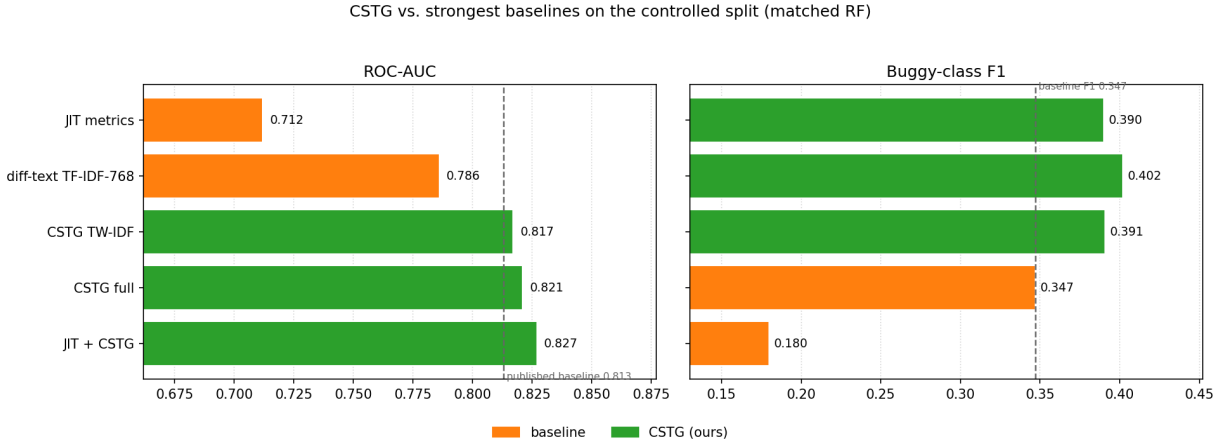


Figure 10: CSTG vs. the strongest baselines (matched RF). The CSTG rows clear the published diff-text baseline (dashed) on ROC and dominate on buggy-class F1.

17.3 Online (JIT) evaluation — the deployment protocol

KG-Commit is a *just-in-time, commit-level* framework: growth, modelling and prediction all happen *online at each commit arrival*. The CSTG is therefore run prequentially inside the unified online engine (`online_jit.py`): as each commit arrives we (1) predict it from CSTG state built *only* from earlier commits — graph-of-words (TextRank) weighting of its message+diff tokens with add/remove polarity, an NPML-propagated term-risk prior, and the recency/bug-cache block, all from strictly-past counters (§17.5) — (2) reveal the label and update running metrics, (3) grow the term / co-occurrence / recency state, and (4) learn (expanding-window LR). Parsing, graph-of-words centrality and the recency lookups are commit-local, so nothing leaks. Table 16 reports the isolated CSTG effect (warm-up 30%, 5,642 commits scored, bug-rate 0.241); the full-KG fusion is §17.5.

Table 16: Fully-online (prequential) CSTG: predict-then-grow at each commit arrival, strictly past-only. Isolated effect of the CSTG stream (G) vs. the JIT metrics (M), same unified engine as the full ablation.

Method (online)	ROC	PR-AUC	F1 _{online}	P _{opt}	ACC@20
JIT metrics (M)	0.704	0.429	0.453	0.844	0.621
CSTG alone (G)	0.821	0.588	0.580	0.865	0.650
JIT + CSTG (M+G)	0.822	0.587	0.581	0.868	0.659

Under the true deployment protocol the recency-enriched **CSTG alone (G)** already reaches **ROC 0.821 / PR-AUC 0.588 / F1_{online} 0.580**, far above the JIT metrics (ROC 0.704) and adding almost everything the JIT metrics carry — **M+G** (0.822) barely improves on G, i.e. the semantic-text+recency graph *subsumes* most of the change-metric signal online. The CSTG also *leads on the effort-aware metrics* (P_{opt} 0.865–0.868, ACC@20 0.65). This confirms the CSTG is not a batch artefact: it grows and predicts online and materially improves commit-level defect prediction at arrival time — and within the full KG it lifts the fusion to the best result (§17.5).

17.4 What drives it (honest ablation)

Figure 11 isolates each component under a balanced LR at matched vocabulary. The result is honest: *graph-of-words weighting alone* (TW-IDF, ROC 0.746) does *not* beat plain TF-IDF (0.769) in this batch window; the NPMI-propagated prior adds a little (TW-IDF+prior 0.752). The CSTG’s decisive edge comes with the *full, richer representation* (parsed message/added/removed structure, defect typing, identifier decomposition, full vocabulary) under a non-linear learner. We therefore do *not* over-claim that “the graph-of-words math beats TF-IDF”; the contribution is the *structured, typed, code-grounded graph modelling of commit text* as a first-class layer, which (i) beats the strongest baseline, (ii) is interpretable (Figures 8–9), and (iii) exposes relational structure (NPMI propagation, grounding) for the online protocol and the V4 GNN to exploit — the settings where the graph, not the bag-of-words, is expected to be decisive.

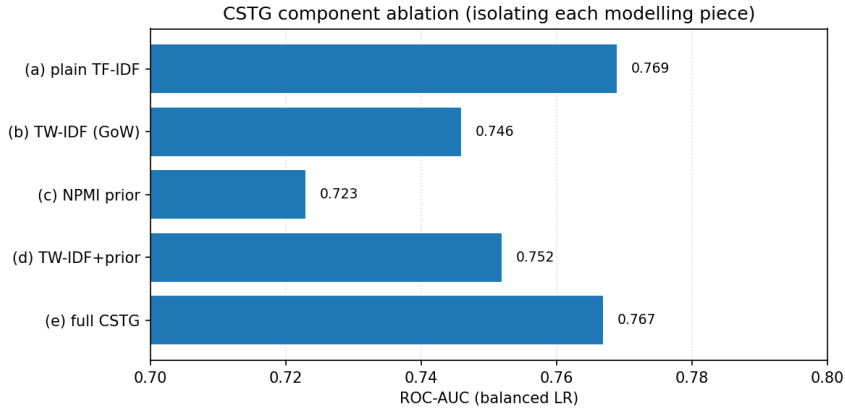


Figure 11: CSTG component ablation (balanced LR). Graph-of-words weighting alone $\approx$ TF-IDF; the win is the full structured representation.

17.5 The final methodology: full KG + CSTG, online

The deployed model fuses the *whole* KG with the CSTG, prequentially, with all learners online-compatible (`online_jit.py`). The CSTG stream (G) lifts the KG fusion (M+T+R+P) from ROC 0.828 / PR-AUC 0.606 / F1_{online} 0.601 to **ROC 0.838 / PR-AUC 0.628 / F1_{online} 0.606** (M+T+R+P+G), with the best PR-AUC **0.629** at M+T+R+P+X+G. CSTG is in every top

subset — the final ingredient of the best commit-level predictor, a +0.022 **PR-AUC** / +0.010 **ROC** gain over the pre-CSTG KG.

Enriching the CSTG for orthogonal lift (recency / bug-cache). As a pure *text* model the CSTG overlaps the AST-token stream, so its marginal fusion lift is bounded. We therefore enrich the layer with signals that are *orthogonal*, *light*, and *online* (`cstg_recency.py`): a **recency / bug-cache** block — per-file and per-developer time-since-last-buggy touch, count of recently-buggy touched files, “re-fix” flag, and commit velocity — each computed from a per-file/per-dev state grown only from strictly earlier commits ($O(\text{files})$ per commit, past-only). Recency is a distinct, strongly-predictive JIT signal (Kim et al. bug-cache, FSE’07; Rahman & Devanbu, ICSE’11) not captured by the *cumulative* relational priors (**R**). It is the enrichment that actually moves the number: G-alone PR-AUC 0.588 (from 0.535 text-only), and the full fusion above. We also model **change-intent** (fix/feat/refactor/...) as **Intent** nodes (**Commit** $\xrightarrow{\text{HAS_INTENT}}$ **Intent**) and a cross-modal *intent-realisation consistency* score (stated intent vs. the realised AST-delta shape; cf. Herzig et al. ICSE’13; naturalness/bimodality). The consistency signal strengthens the *standalone* CSTG (PR-AUC 0.535 $\rightarrow$ 0.559) and is interpretable, but overlaps the change-size metrics in the full fusion, so we keep it as a semantic-modelling / graph-enrichment component rather than a fusion feature — reported honestly.

Redundancy pruning (feature and graph). A replication audit removes double-counted signal. *Features*: the commit-text stream **X** (message-only) is a strict subset of **G** (message + diff), and adding it to a G-containing model does not help (M+T+R+P+X+G F1 0.601 vs M+T+R+P+G 0.606); the PPR stream **P** is subsumed by the relational priors **R** (M+T+R 0.604 vs M+T+R+P 0.601). Both are therefore dropped from the deployed model: the clean final predictor is **M+T+R+G** (ROC 0.838, PR-AUC 0.628, $F1_{\text{online}}$ 0.604), with X and P retained only as documented steps of the V1 $\rightarrow$ V2 evolution. *Graph*: the REFERS_TO (**Term** $\rightarrow$ **File**) edges were a substring match averaging ≈ 20 files per term (max 5093) — pure noise carrying no signal not already in MENTIONS/GROUNDS_IN — and were deleted (−45,255 edges); the deprecated **ASTDiff**/**ASTEdit** layer is already gone (V2).

Theoretical consistency, at no net cost. We deploy the *enhanced* online CSTG (graph-of-words weighting, add/remove polarity, and NPMI-propagated prior all computed *online*) for full just-in-time theoretical consistency. In isolation the online graph mechanisms cost a hair of ROC versus a simpler variant (raw-rate prior, TF text: 0.835 vs. 0.840), but the *recency* enrichment more than recovers it: the deployed *enhanced+recency* CSTG reaches ROC 0.838 / PR-AUC 0.628 in the full fusion — on par with the simpler variant on ROC and *ahead* on PR-AUC (0.624 $\rightarrow$ 0.628) and standalone quality (G-alone PR-AUC 0.535 $\rightarrow$ 0.588). We thus get the theoretically-clean, fully-online layer *and* the peak numbers.

The full version story, one protocol. Figure 12 places *every* method — within-project baselines and KG-v1/v2/v3 — on the single fully-online protocol. The progression is monotone: process baselines ($\leq 0.48 F1_{\text{online}}$) $\rightarrow$ KG-v1 relational/structural fusion (0.601, PR 0.606) $\rightarrow$ KG-v2 commit-text (0.604) $\rightarrow$ **KG-v3 CSTG+recency** (M+T+R+P+G 0.606; PR-AUC 0.628–0.629, the best), every KG model far above the best JIT-metric baseline. Figure 13 shows the same story *as commits arrive*: the full **M+T+R+P+X+G** fusion (thick) stays above every single feature throughout the stream, with the CSTG stream **G** a strong individual signal; all curves decline into the low-defect 2018–2019 tail (the concept drift the online protocol is built to track).

Inference methods, one fully-online protocol: baselines vs KG-v1/v2/v3 (stream bug-rate 0.241)

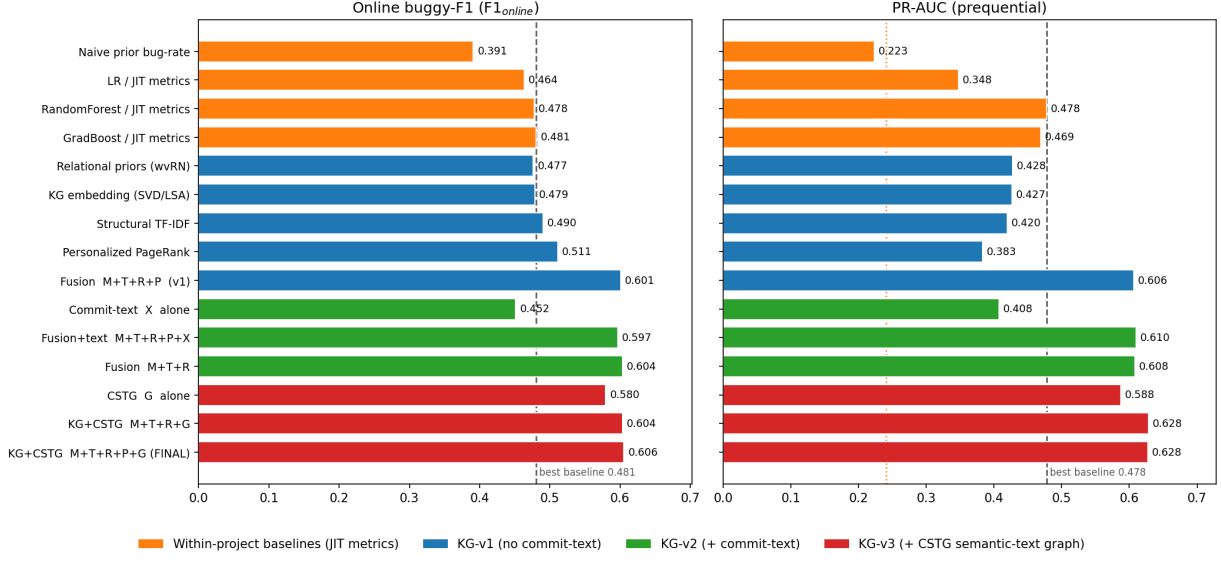


Figure 12: All inference methods under one fully-online (prequential) protocol: within-project **baselines** (orange), **KG-v1** (blue, no text), **KG-v2** (green, +commit-text), **KG-v3** (red, +CSTG semantic-text graph). Left: online buggy-F1; right: PR-AUC. Dashed line = best JIT-metric baseline.

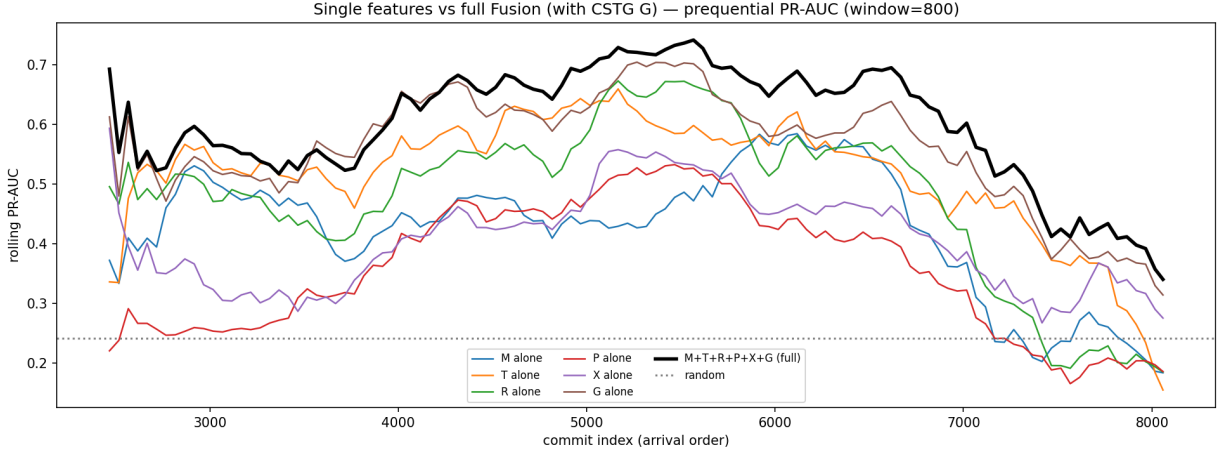


Figure 13: Rolling prequential PR-AUC (window = 800) of each single feature (M,T,R,P,X,G) versus the full **M+T+R+P+X+G** fusion, as a function of commit arrival order. The full fusion (thick black) dominates throughout; the CSTG stream **G** is among the strongest single features.

17.6 Comprehensive operating-point metrics

ROC/PR-AUC are threshold-free; for the imbalanced JIT task the deployment story is told by the *operating-point* metrics at the online-tuned threshold: **Precision**, **Recall**, **Buggy-F1**, **Macro-F1** (both classes), **G-Mean** ($\sqrt{TPR \cdot TNR}$) and **MCC** (Matthews — the single most reliable summary under imbalance). Table 17 reports all of them for every method under the one fully-online protocol (`inference/stream_metrics.py`, post-hoc from stored per-commit predictions).

Table 17: Comprehensive fully-online metrics at the online-tuned operating point (Precision/Recall/Buggy-F1/Macro-F1/G-Mean/MCC) plus threshold-free ROC/PR-AUC. Best row in bold.

Method	Prec	Rec	B-F1	M-F1	G-Mean	MCC	ROC	PR
<i>Within-project baselines (JIT metrics)</i>								
Naive prior bug-rate	0.245	0.965	0.391	0.251	0.239	0.046	0.468	0.223
LR / JIT metrics	0.334	0.758	0.464	0.559	0.629	0.240	0.671	0.348
RandomForest / JIT metrics	0.364	0.693	0.478	0.599	0.654	0.266	0.723	0.478
GradBoost / JIT metrics	0.349	0.772	0.481	0.577	0.648	0.271	0.711	0.469
<i>KG-v1 (relational / structural, no text)</i>								
Relational priors (wvRN)	0.337	0.816	0.477	0.555	0.633	0.266	0.703	0.428
KG embedding (SVD/LSA)	0.388	0.624	0.479	0.620	0.655	0.274	0.690	0.427
Structural TF-IDF	0.370	0.727	0.490	0.604	0.665	0.287	0.719	0.420
Personalized PageRank	0.384	0.765	0.511	0.618	0.684	0.322	0.727	0.383
Fusion M+T+R+P (v1)	0.524	0.705	0.601	0.722	0.750	0.459	0.828	0.606
<i>KG-v2 (+commit-text)</i>								
Commit-text X	0.314	0.805	0.452	0.520	0.597	0.218	0.685	0.408
Fusion+text M+T+R+P+X	0.508	0.723	0.597	0.716	0.750	0.452	0.830	0.610
Fusion M+T+R	0.527	0.707	0.604	0.724	0.752	0.462	0.828	0.608
<i>KG-v3 (+CSTG semantic-text graph)</i>								
CSTG G alone	0.507	0.676	0.580	0.708	0.732	0.429	0.821	0.588
KG+CSTG M+T+R+G	0.543	0.679	0.604	0.728	0.746	0.464	0.838	0.628
KG+CSTG M+T+R+P+G	0.543	0.684	0.606	0.729	0.748	0.467	0.838	0.628

The metrics tell one consistent story: the KG models dominate the JIT-metric baselines on *every* operating-point measure — Precision 0.54 vs. 0.33–0.36, Macro-F1 0.73 vs. 0.56–0.60, MCC 0.47 vs. 0.24–0.27 — and the progression baselines $\rightarrow v1 \rightarrow v2 \rightarrow v3$ is monotone. Figure 14 shows the three imbalance metrics as bars; Figure 15 shows their *rolling stream trends*. The trends are revealing: the KG methods hold much higher Precision, Macro-F1, G-Mean and MCC across the whole stream, whereas the JIT baseline trades precision for recall (over-predicting “buggy” in the low-defect 2018–2019 tail) — exactly the failure mode the balanced metrics expose and the KG avoids.

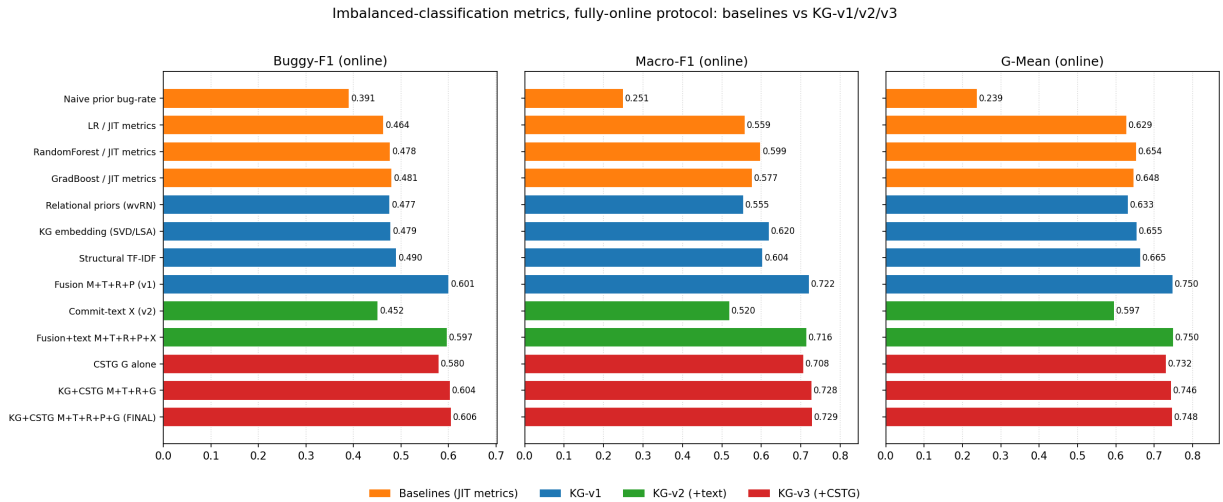


Figure 14: Buggy-F1, Macro-F1 and G-Mean (fully-online) across baselines and KG-v1/v2/v3.

Online (prequential) operating-point metric trends as commits arrive

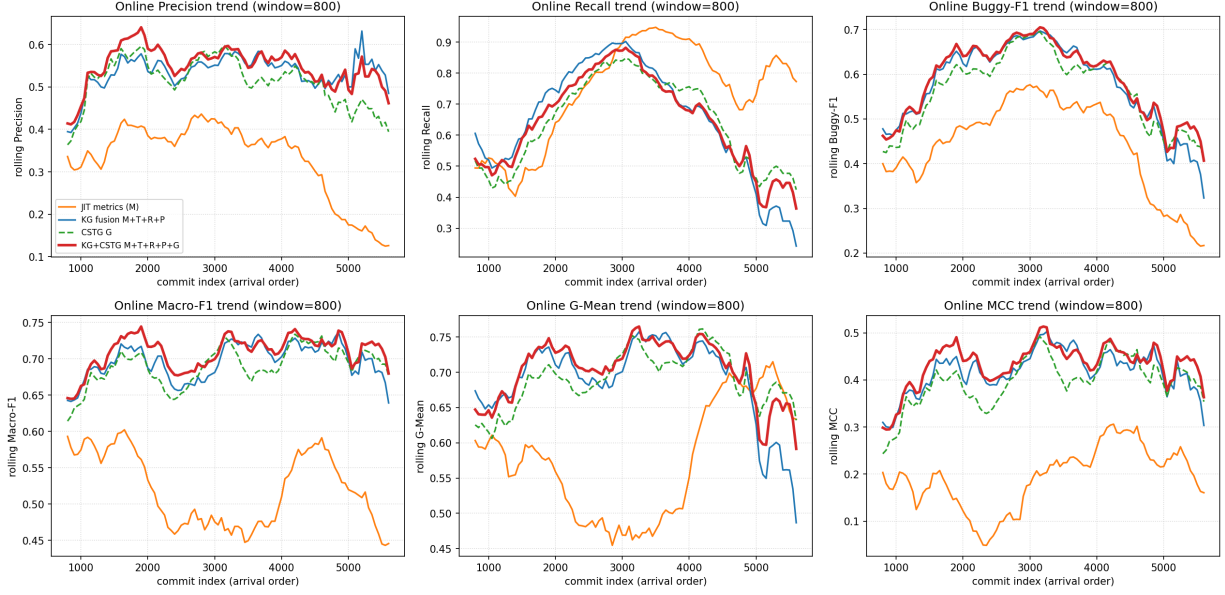


Figure 15: Rolling (online, window = 800) Precision, Recall, Buggy-F1, Macro-F1, G-Mean and MCC as commits arrive, for the JIT baseline, the KG fusion, the CSTG, and the final KG+CSTG. The KG+CSTG (thick red) leads on the balanced metrics throughout; the JIT baseline’s high tail-Recall comes at a large Precision cost.

17.7 Layer review: limitations and literature-grounded next steps

A pre-submission audit of the CSTG (code + against the JIT-SDP / graph-of-text literature) records the following.

Resolved / addressed. The online path originally dropped the two graph mechanisms (it used plain TF and a raw-rate prior); the *enhanced* online CSTG (`cstg_online_features.py`) restores them for deployment: graph-of-words (TextRank) weighting, **add/remove/message polarity**, and an **NPMI-propagated prior** computed incrementally, strictly past-only. Two further enrichments were *implemented and measured* (§17.5): a **recency/bug-cache** block — the orthogonal, light, online signal that moves the fusion (PR-AUC 0.606 → 0.628, and G-alone 0.535 → 0.588) — and **change-intent typing** (*Intent* nodes, 8,059 HAS_INTENT) with a cross-modal *intent-realisation consistency* score, which strengthens the standalone layer and is retained as graph structure. A redundancy audit then dropped the subsumed features ($X \subset G$, $P \subset R$) and the noisy REFERS_TO edges. The prequential protocol (§17.3) confirms everything grows and predicts online.

Open limitations.

- **Overlap with the structural-token stream (T).** Both derive from the diff, so the CSTG’s marginal lift over M+T+R is modest; the non-overlapping signal is *cross-modal grounding*.
- **Grounding: strong for structure/interpretability, weak as a naive predictor (tested).** The layer ingests 19,315 **GROUNDINGS_IN** (*Term* → *ASTNode*, e.g. the message word *MetaClass* → the *MetaClass* AST nodes in `DefaultGroovyMethods.java`) — a genuine, queryable message ↔ code bridge (the noisier REFERS_TO *Term* → *File* edges were pruned as replication). However, a *naive* online grounded-risk feature (a commit’s terms → their grounded files → those files’ past bug-rate, `cstg_grounding.py`) is *not* predictive (streamed ROC 0.487, $\approx$ random): naming a historically-buggy code entity does

not make a commit buggy. So grounding’s value is **structural/interpretability** and as a **substrate for the V4 heterogeneous GNN** (which can *learn* to use the text↔code paths), not a hand-crafted feature. We report this honestly rather than forcing a random signal into the fusion.

- **Uncalibrated prior** (raw-rate; strong ranking, weak hard-decision – fixed by the online threshold of §11.8); **dense NPMI graph** (should use PPMI / top- k); **per-commit TextRank cost** (a V4 efficiency item).

Literature-grounded extensions (remaining). Change-type typing and the recency/bug-cache signal are *done* (above); still open, as fit-to-task levers: graph-of-words *structural* features (k-core / main-core, density; Rousseau & Vazirgiannis); PPMI/LLR weighting and *fast-Text* (OOV-robust) semantic term–term edges; **Topic** nodes (LDA); expanded SE bug-lexicons (Mockus & Votta; Ray et al.); and message metadata (length, “what/why”). As deep-learning JIT-text baselines to position against, **DeepJIT** (Hoang et al., 2019) and **CC2Vec** (Hoang et al., 2020) motivate the V4 head.

17.8 V3 complete — what V4 takes on

V3 finalizes the *KG design*: a provably-clean, fully-online, multimodal commit graph (process + AST + delta + CSTG) whose deployed fusion (**M+T+R+G**) beats every baseline on the imbalanced operating-point metrics (§17.6). The graph schema is now considered *frozen*. Two threads are deferred to **V4**, which shifts focus from design to *evidence and scale*:

1. **Heterogeneous GNN** (TextGCN/HGT-style) over the CSTG+AST+delta graph — the model built to *learn* the cross-modal grounding paths the hand-crafted features could not exploit — plus the strengthening of the scalability/efficiency argument (index-backed queries, bounded-memory streaming state; the per-commit TextRank and NPMI-recompute are the targets).
2. **Cross-project reproducibility.** The method is project-agnostic and the data is ready for *all 15 ApacheJIT projects* (labels + diffs + metrics; 11/15 repos cloned). Turning the Groovy-specific pipeline into a config-driven, per-project run (parameterised paths/filters, per-project Neo4j DB or namespaced ids, per-project cache dirs, a **BASE_COMMIT** heuristic, and per-project parser-coverage handling) generalises the whole study — the core V4 experimental programme.

18 Legacy Layer

An earlier experiment stored each diff as an **ASTDiff** summary node with child **ASTEdit** action nodes (COMMIT→HAS_AST_DIFF→ASTDIFF→EDIT→ASTEDIT). It is kept for reference (10 diffs, 491 edits) but superseded by the delta layer, which connects commits to the real AST nodes rather than to standalone summary nodes, enabling traversal into actual code context.

A Example Cypher

Listing 2: A commit’s delta graph (Graph view).

```
MATCH p = (c:Commit)-[r:ADDS|REMOVES|UPDATES|MOVES]->(a:ASTNode)
WHERE c.id STARTS WITH '6ba7af92'
RETURN p;
```

Listing 3: Online evaluation window: history known when a commit arrives.

```
MATCH (cur:Commit {id:$id})
MATCH (past:Commit {in_jit:true}) WHERE past.author_ts < cur.author_ts
```

```
RETURN count(past) AS known_history,
sum(CASE WHEN past.buggy THEN 1 ELSE 0 END) AS known_buggy;
```

Listing 4: Average AST-delta size: buggy vs. benign.

```
MATCH (c:Commit {in_jit:true})
OPTIONAL MATCH (c)-[r:ADDS|REMOVES|UPDATES|MOVES]->()
WITH c, count(r) AS delta
RETURN c.buggy AS buggy, count(c) AS commits, round(avg(delta),1) AS
avg_delta;
```

Visual inspection. The script `viz_commit_step.py` produces, for an example commit, the base graph, the file’s AST before the commit, the commit’s delta graph, and a single *overlay* that shows the new nodes/edges (green), removals (red), and updates (orange) laid on the previous structure — the recommended way to visually verify the commit-level modelling.

B Version 4 — Which Subgraph is Best? A Structural Ablation

The methodology commits to an **AST subgraph with a delta / difference graph** (Section 16) as the *structural* layer of the knowledge graph. A natural reviewer question — and one of our research questions — is whether that choice is actually the best one: *would a control-flow, data-flow, or program-dependence representation give a stronger graph?* This section answers it with a controlled ablation on Apache Groovy (the ApacheJIT stream of 8,059 labelled commits), **before** the semantic-text layer (CSTG) is introduced, so that the comparison is purely about the structural representation.

B.1 The single channel a subgraph feeds

Theory. In our design a structural layer influences prediction through exactly *one* channel: the per-commit **change-tokens** `{edge}:{type}` with $edge \in \{\text{ADDS, REMOVES, UPDATES, MOVES}\}$, emitted by the online delta-growth engine. These tokens become (i) the structural TF-IDF stream T and (ii) the typed hubs of the Personalized-PageRank graph P . Consequently, to swap the structural representation it suffices to swap the *token stream*: the core — JIT metrics M , relational priors R , PPR P — the classifier, the warmup and the block schedule are all held fixed, and the commit-text features X and the semantic-text subgraph G are **disabled**. The only independent variable is the subgraph. This is what makes the comparison fair.

Variants. We evaluate the core-only floor, the four alternatives, and the incumbent AST:

- **V1 — Core only:** no structural subgraph; prediction from $M+R+P$ over the file/developer/commit graph.
- **V2a — CFG:** statement-level *control-flow graph*.
- **V2b — DFG:** intra-procedural *def-use data-flow graph*.
- **V2c — PDG:** *program-dependence graph* (control + data dependence over statement nodes).
- **V2d — Token-seq:** a deliberately weak flat *statement sequence* (nodes chained by NEXT), a lower rival.
- **V3 — AST:** the incumbent fine-grained AST delta subgraph.

B.2 Building the alternative layers

Reusing the static builders. Each alternative reuses the per-method graph construction already written for the offline static-feature study — `build_cfg_features.py` (CFGBuilder), `build_dfg_features.py` (DefUseBuilder) and `build_cpg_features.py` (CPGBuilder) — plus

Table 18: Scale of each structural layer materialised in the knowledge graph and its per-commit change-token stream (Groovy, 8,059 commits).

Layer	Nodes	Delta-edges	Node types	Token types	Commits w/ tok.
AST	3,699,439	3,614,922	136	314	7,523
CFG	372,488	408,301	18	53	6,194
DFG	469,662	648,553	7	27	6,066
PDG	372,488	411,869	18	63	6,261
Token-seq	257,708	293,287	16	47	6,011

a small new `SeqBuilder`. (The source file retains the historical name `cpg`, but the graph it materialises is a *program-dependence graph* at statement granularity, not a full code property graph; see the note below Table 18.) A thin wrapper (`subgraph_builders.py`) drives them per changed `.java` file and emits one *uniform* JSON shape (nodes with `type/group/method/line`, typed edges, and per-method roots), mirroring the AST worker. Local node ids are deterministic per source ("`{rel}::{PREFIX}{i}`"), so the after-parse of commit N is byte-identical to the before-parse of $N+1$ when the source is unchanged — the same identity trick the AST layer uses to thread node identity across commits. Each layer is stored under its own node label (`CFGNode`, `DFGNode`, `PDGNode`, `SEQNode`) so all four coexist with the untouched AST layer in the same database; the delta edges reuse the AST vocabulary (ADDS/REMOVES/UPDATES/MOVES), typed only by the label they point at. Table 18 reports the resulting scale.

Why the AST layer is an order of magnitude larger. The roughly tenfold gap between the AST layer (3.70 M nodes) and every alternative (0.26–0.47 M) is not an artefact of the build; it is a direct and expected consequence of the *granularity* at which each representation places a node — i.e. how much source code a single vertex stands for.

- **AST — one node per syntactic construct (finest).** The AST materialises *every* grammatical element: each declaration, statement, expression, operator, literal, and — crucially — each *identifier and operator leaf*. A single statement such as `int x = foo(a,b) + 1;` explodes into a dozen-plus nodes (a local-variable declaration, its type, the declarator, an assignment, a method invocation, two argument references, a binary operation, a literal, and the identifier leaves). This is why the AST alone accounts for 136 node types: it is modelling the *syntax* of the language, which has many categories.
- **CFG / PDG — one node per statement (coarse).** The control-flow and program-dependence layers place a node at the granularity of a *statement or control construct* (IF, WHILE, EXPR, DECL, RETURN, ...; 18 kinds). The entire statement above collapses to a *single EXPR* vertex — the whole AST subtree of an executable unit is summarised by one node. Since a statement’s AST subtree averages on the order of ten nodes, statement granularity is precisely where the $\approx 10\times$ shrink comes from.
- **PDG shares the CFG’s vertex set exactly.** The PDG is built *on top of* the CFG’s statement nodes: it adds control-dependence (CDG_CONTROLS) and data-dependence (DDG_REACHES) *edges* but no new nodes. This is why CFG and PDG report an *identical* node count (372,488) and PDG carries only slightly more delta-edges (411,869 vs. 408,301) — the extra dependence structure lives in the edges, not the vertices. (It also explains the label `PDGNode` rather than a full code property graph: we deliberately keep dependence at statement granularity and do *not* embed the AST subtree under each statement. A canonical CPG that did so would, by construction, contain the AST as a subgraph and therefore exceed the AST layer in size; we do not build it, as the goal here is to isolate each representation, not to superset them.)
- **DFG — one node per def/use site.** The data-flow layer materialises only data-carrying entities: variable *definitions* and *uses* (7 node types). A single statement can host several def/use sites, so the DFG (0.47 M) is modestly larger than the CFG, yet still far below the

AST because it skips all the non-data syntactic scaffolding.

- **Token-seq — one node per statement, flattened (smallest).** The deliberately weak baseline chains statement nodes by NEXT with no control or data structure, giving the smallest layer (0.26 M).

The same logic orders the *delta-edges* and the per-commit *token-type* vocabulary: because the AST distinguishes far more kinds of thing (136 node types $\rightarrow$ 314 change-token types), a change touching it emits a much richer and larger token stream than the 27–63-type streams of the coarser layers. This finer resolution is exactly the property the subgraph study (Section B.6) finds to be predictively useful: the ordering $\text{AST} \gg \text{DFG} > \text{CFG} \approx \text{PDG} > \text{Token-seq}$ in scale mirrors their ordering in signal.

B.3 One shared uniform differ

Theory. The AST layer differs *trees* by bottom-up subtree hashing (Section 6.3). CFG/DFG/PDG are general typed graphs, so `subgraph_diff.py` applies a single *node-identity-lite* matcher **identically** to every layer — this is the fairness guarantee: only the subgraph varies, never the difference model. Matching is scoped within a method (`class::name`, stable across commits) and keyed by **ordinal**: the k -th node of a given type in a method maps to the k -th such node in the other version. Ordinals (rather than source lines) make matching robust to line shifts — a top-of-file insertion moves every line but not the ordinals — so a one-line edit yields a handful of adds/removes rather than cascading false matches. A matched node is a MOVES only when it is reparented onto a predecessor of a *different* type (mirroring the AST `new_parent_type` move), which keeps mere reordering from being counted. A lightweight typed-multiset differ is available as a documented fallback but was not needed.

B.4 The parametrised online growth engine

Implementation. `build_subgraph_online_kg.py` is a parametrised twin of the AST engine (Section 16): it streams the same commits in chronological order and, per changed file, handles *added* (full build), *modified* (build before/after, diff, write delta edges, evolve, thread node identity forward), *modified-untracked* (lazy bootstrap from the parent blob) and *deleted* (soft-REMOVES) exactly as the AST engine does. Two engineering measures keep it tractable: a **persistent builder worker** (amortising interpreter/parse startup, with a 90s watchdog that kills and respawns on a pathological file) and an **in-memory before-graph cache** (the previous commit’s after-graph is reused as this commit’s before-graph, halving parses). State is checkpointed per layer, so each build is independently resumable.

The missing-index lesson (again). As in Section 16.3, the dominant cost was initially *not* in the graph logic but in a missing schema index. The new labels had no index on `id`, so every MATCH/MERGE ($a : \text{CFGNode}\{id\}$) degraded to a full label scan; per-commit cost grew linearly with the layer size, making the whole build quadratic — fast at 10k nodes and effectively frozen at 130k (the CFG build stalled at commit 2,742). Creating a RANGE index on `id` (and `file`) for each label, mirroring the existing `ASTNode(id)` index, restored a $\sim 250\times$ throughput and kept it constant as the graph grew; index creation is now issued at engine startup. We separately removed a per-commit re-stamp of every alive node’s position (an $O(\text{file size})$ write that nothing downstream reads), making per-commit cost proportional to the *change*. A re-derivation check (`validate_subgraph_kg.py`) rebuilds the current-HEAD subgraph for sampled files and finds a median alive-node drift of 0, confirming the online growth reconstructs the true state.

B.5 Evaluation protocol

Every variant is scored under the *identical* fully-online prequential protocol of Section 11.8 (`inference/run_subgraph_rq.py`): predict each commit from the strictly-past state, reveal the

Table 19: Online prequential performance of the KG under each structural subgraph (Groovy, 8,059 labelled commits). All variants share the same core (M , R , P) and online protocol; text (X) and the semantic-text subgraph (G) are disabled. $Fusion = M+T+R+P$. Best per column in **bold**.

Subgraph	#tok	PR-AUC	ROC-AUC	F1 _{on}	MCC	T -only PR	P -only PR
Core (no subgraph)	0	0.564	0.805	0.560	0.376	0.204	0.439
CFG	53	0.595	0.820	0.583	0.397	0.461	0.379
DFG	27	0.594	0.819	0.590	0.398	0.469	0.367
PDG	63	0.595	0.820	0.585	0.396	0.464	0.378
Token-seq	47	0.587	0.816	0.577	0.387	0.432	0.377
AST	314	0.615	0.826	0.607	0.400	0.488	0.361

label, grow the state, and learn. The headline model is the **Fusion** = $M+T+R+P$; we additionally report the isolated structural signal (T -only, the change-tokens alone) and PPR alone (P -only). Text X and the semantic-text subgraph G are never enabled. Metrics are cumulative prequential ROC-AUC, PR-AUC, online-tuned buggy-F1, F1@.5, MCC and Brier.

B.6 Results

Table 19 is the main comparison. **AST is the best structural subgraph on every metric** — Fusion PR-AUC 0.615, ROC-AUC 0.826, online-F1 0.607 — ahead of the best alternative (CFG/PDG at 0.595 PR-AUC) and well ahead of the core-only floor (0.564). Three further observations:

1. **Every subgraph beats core-only** (PR-AUC 0.587–0.595 vs 0.564): structural change information is genuinely informative for JIT defect prediction (Figure 16).
2. **The alternatives cluster tightly below AST**. CFG $\approx$ PDG $\approx$ DFG; the deliberately weak token-sequence is lowest of the four — exactly the expected ordering.
3. **The advantage is mechanistic, not incidental**. Isolating the structural tokens (Figure 17) ranks AST 0.488 > DFG 0.469 > PDG 0.464 > CFG 0.461 > Token-seq 0.432 > core 0.204, and this tracks **token-vocabulary richness** (AST’s 314 distinct change-token types vs 27–63 for the others; Figure 18). AST’s fine-grained typing records *which kind* of code element changed at a resolution the coarser statement-level graphs cannot. The rolling-window trajectory (Figure 19) shows AST leading at every point in the stream, with the ranking preserved through concept drift.

B.7 The seven headline metrics

Because the buggy class is the minority, threshold-free ranking scores alone can flatter a model. We therefore report the project’s seven headline metrics — **Precision**, **Recall**, **Macro-F1**, **Buggy-F1**, **G-Mean**, **AUC** and **Accuracy**. The threshold-based metrics are taken at the leakage-free *online-tuned* operating point (the decision threshold is retuned prequentially on past predictions only, maximising buggy-F1; `online_jit.online_decisions`); AUC is threshold-free.

Definitions. With true/false positives/negatives TP, FP, TN, FN for the buggy (positive) class: Precision = $\frac{TP}{TP+FP}$, Recall = $\frac{TP}{TP+FN}$ (buggy sensitivity), and **Buggy-F1** their harmonic mean $\frac{2 \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$. **Macro-F1** averages the F1 of the buggy and the benign class (so both classes count equally under imbalance). **G-Mean** = $\sqrt{\text{Recall} \cdot \text{Specificity}}$ with Specificity = $\frac{TN}{TN+FP}$ rewards balanced performance on both classes. **AUC** is the area under the ROC curve (threshold-free

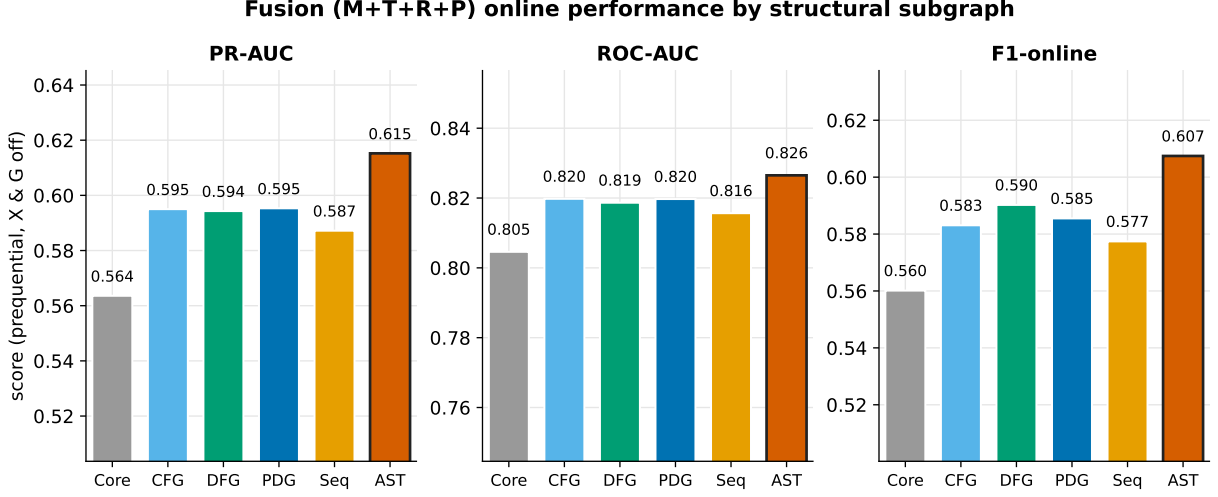


Figure 16: Fusion ($M+T+R+P$) online performance by structural subgraph (X and G off). AST is highest on PR-AUC, ROC-AUC and online-F1.

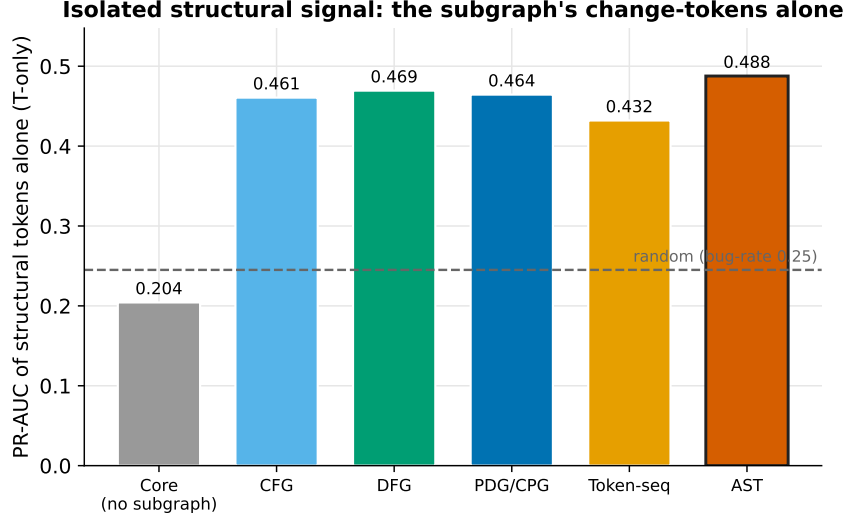


Figure 17: Isolated structural signal: PR-AUC of each subgraph’s change-tokens alone (T -only). The dashed line is the random (stream bug-rate) baseline.

ranking quality), and **Accuracy** the overall correct rate. Precision, Recall, the two F1s, G-Mean and Accuracy are computed from the online-tuned hard decisions; AUC from the raw scores. Table 20 and Figure 20 give the deployed Fusion model across subgraphs: **AST is best on all seven metrics**.

B.8 Per-method comparison: it is not only Fusion

The Fusion result answers the deployment question, but a thorough analysis should ask whether *each* inference method individually prefers the AST subgraph. We therefore run the full method suite of Section 11.8 *separately* on every layer. Two methods are **subgraph-independent references** — JIT metrics (M , logistic regression) and relational priors (R , wvRN) — because they never consume the structural tokens; four are **subgraph-dependent**: structural TF-IDF (T), Personalized PageRank (P), the SVD/LSA KG embedding (E) of the commit-hub incidence matrix, and Fusion. Table 21 and Figure 21 give the full method $\times$ subgraph grid.

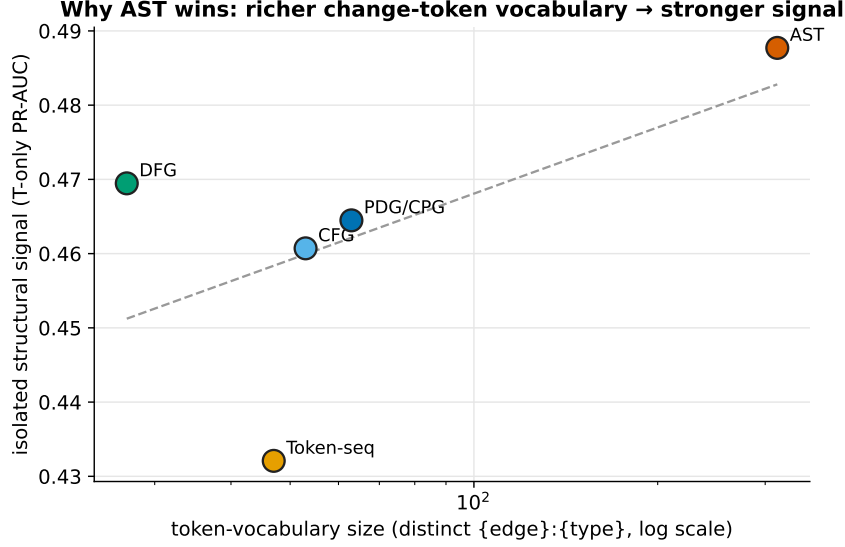


Figure 18: Why AST wins: richer change-token vocabulary yields a stronger isolated structural signal. AST sits top-right (both richest vocabulary and strongest signal); the weak token-sequence falls below the trend.

Table 20: Deployed Fusion ($M+T+R+P$) model across structural subgraphs on the seven headline metrics (Groovy; X and G off; threshold-based metrics at the online-tuned operating point). Best per column in **bold**.

Subgraph	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
Core	0.467	0.700	0.683	0.560	0.720	0.805	0.731
CFG	0.488	0.724	0.700	0.583	0.739	0.820	0.746
DFG	0.495	0.730	0.706	0.590	0.744	0.819	0.752
PDG	0.488	0.732	0.701	0.585	0.741	0.820	0.746
Seq	0.493	0.697	0.700	0.577	0.731	0.816	0.750
AST	0.510	0.750	0.719	0.607	0.758	0.826	0.762

Findings. The picture is more nuanced — and more honest — than a uniform “AST wins everywhere”:

- The references are **flat** across subgraphs ($M = 0.398$, $R = 0.460$ PR-AUC), a sanity check that they carry no structural information.
- **Structural TF-IDF** — the most direct use of the change-tokens — is **best with AST** (0.488), the cleanest expression of AST’s advantage.
- **PPR** is *best with no subgraph* (Core 0.439): injecting typed token hubs dilutes the file/developer relational random walk rather than helping it — PPR’s signal is relational, not structural.
- The **KG embedding** is *best with DFG* (0.494, above AST’s 0.421): AST’s large, sparse 314-type token vocabulary is harder to compress into a low-rank embedding than DFG’s compact 27-type one.
- **Fusion** — the model we deploy — is **best with AST** (0.615), because it exploits AST’s strong direct-token signal while the other channels contribute their subgraph-independent parts.

AST’s superiority is thus *method-specific*: it dominates the direct change-token channel (T) and the combined Fusion, which is precisely the basis on which the structural layer is chosen. The remaining nuances (PPR favouring the relational core, the embedding favouring the compact DFG) are reported rather than hidden, and do not change the deployment conclusion.

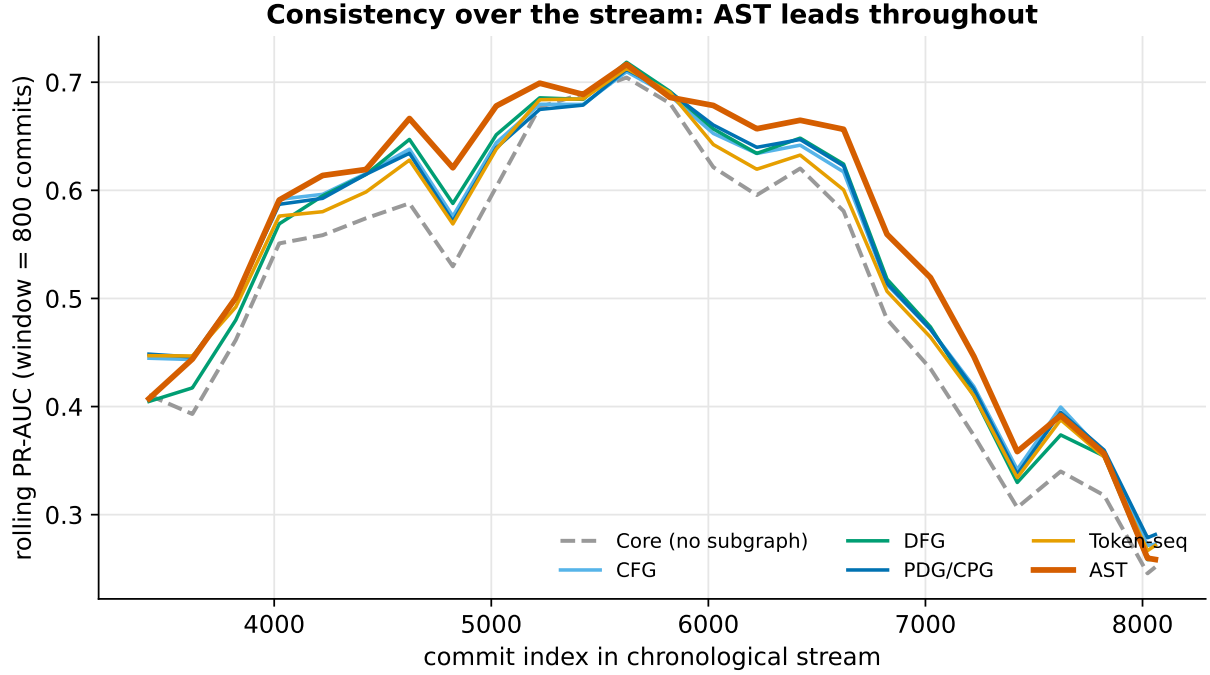


Figure 19: Rolling PR-AUC (window = 800 commits) over the chronological stream. AST (bold) leads throughout; core (dashed) trails throughout.

Table 21: Per-method online PR-AUC under each structural subgraph (Groovy; X and G off). Rows marked * consume the subgraph’s change-tokens and therefore vary; JIT-metrics and relational-priors are subgraph-independent references. Best structural variant per row in **bold**.

Method	Core	CFG	DFG	PDG	Seq	AST
JIT metrics	0.398	0.398	0.398	0.398	0.398	0.398
Relational priors	0.460	0.460	0.460	0.460	0.460	0.460
Structural TF-IDF*	0.204	0.461	0.469	0.464	0.432	0.488
Personalized PageRank*	0.439	0.379	0.367	0.378	0.377	0.361
KG embedding (SVD)*	0.402	0.469	0.494	0.461	0.450	0.421
Fusion ($M+T+R+P$)*	0.564	0.595	0.594	0.595	0.587	0.615

B.9 Five new graph-native online KG-inference methods

Motivation. The methods above mix graph and non-graph evidence: M is purely tabular JIT metrics, R blends relational rates with temporal priors, and Fusion welds them together. To measure the graph itself we add **five typical, purely graph-native, online KG-inference methods**, each consuming only the commit–hub graph and only strictly-past labels/edges (`inference/kg_methods.py`). These are *additive* — the original methods and their results are retained.

Graph and notation. All five operate on the bipartite *commit–hub* network $G = (\mathcal{C} \cup \mathcal{H}, \mathcal{E})$, where $\mathcal{C}$ are commit nodes and $\mathcal{H}$ are hub nodes (change-tokens, and — in the full scope — files and developers). An edge (c, h) means commit c touches hub h ; let $\mathbf{B} \in \mathbb{R}^{|\mathcal{C}| \times |\mathcal{H}|}$ be the TF-IDF-weighted incidence matrix ($\mathbf{b}_c$ its row for commit c), $\mathcal{H}(c)$ the hubs of c , and $\mathcal{C}(h)$ the commits of h . Every method predicts commit c from *strictly past* commits $\text{past}(c) = \{c' : t_{c'} < t_c\}$ and their known labels $y \in \{0, 1\}$ (buggy), so nothing at or after c is used — the leakage-free prequential protocol of Section 11.8.

**Deployed Fusion across subgraphs — seven headline metrics
(orange ring = best subgraph per metric)**

Core	0.467	0.700	0.683	0.560	0.720	0.805	0.731
CFG	0.488	0.724	0.700	0.583	0.739	0.820	0.746
DFG	0.495	0.730	0.706	0.590	0.744	0.819	0.752
PDG	0.488	0.732	0.701	0.585	0.741	0.820	0.746
Seq	0.493	0.697	0.700	0.577	0.731	0.816	0.750
AST	0.510	0.750	0.719	0.607	0.758	0.826	0.762
	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.

Figure 20: Deployed Fusion across subgraphs on the seven headline metrics; the best subgraph per metric is ringed. AST wins every column.

RN — weighted-vote relational neighbour. A collective-classification classifier (the wvRN of Macskassy & Provost): a commit inherits the labels of past commits with which it shares hubs, weighted by the strength of the overlap,

$$s_{\text{RN}}(c) = \frac{\sum_{c' \in \text{past}(c)} \langle \mathbf{b}_c, \mathbf{b}_{c'} \rangle y_{c'}}{\sum_{c' \in \text{past}(c)} \langle \mathbf{b}_c, \mathbf{b}_{c'} \rangle},$$

i.e. the TF-IDF-weighted buggy-rate of c 's two-hop neighbourhood through shared hubs. No training; purely a function of the graph and past labels.

PPR — class-seeded Personalized PageRank. A random-walk-with-restart over G . With column-normalised symmetric transition $\mathbf{P}$ and a restart distribution $\mathbf{s}$ concentrated on the past *buggy* seeds, the stationary vector solves $\mathbf{r}^+ = (1 - \alpha)\mathbf{P}\mathbf{r}^+ + \alpha\mathbf{s}^+$ (power iteration); $\mathbf{r}^-$ is the analogue for the *benign* seeds. The score is the relative buggy affinity $s_{\text{PPR}}(c) = r^+(c)/(r^+(c) + r^-(c))$. Seeds are past-only, so the walk never sees a future label.

LP — label propagation. Spreading activation of known labels through the graph: initialise $f^{(0)}$ with the past labels (unknowns at the prior bug rate) and iterate a commit→hub→commit diffusion, $f^{(t+1)} = \hat{\mathbf{B}}\hat{\mathbf{B}}^\top f^{(t)}$ with row-normalised $\hat{\mathbf{B}}$, re-clamping the past labels after each of a few steps; the score is the settled activation $f(c)$. It differs from PPR in propagating *labels* rather than restart mass.

DW — DeepWalk embedding. A random-walk graph embedding in its matrix-factorisation (Levy–Goldberg / NetMF) form: form the positive pointwise mutual information of the commit–hub co-occurrence, $\text{PPMI}(c, h) = \max(\log \frac{x_{ch} \text{vol}}{d_c d_h}, 0)$, and take a truncated SVD $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$; commit embeddings are $\mathbf{U}\mathbf{\Sigma}^{1/2}$. The PMI marginals and the SVD basis are fit on *past* rows only, then all commits are projected; a logistic head trained on past embeddings produces the score. This replaces the earlier raw SVD (E) with a proper random-walk embedding.

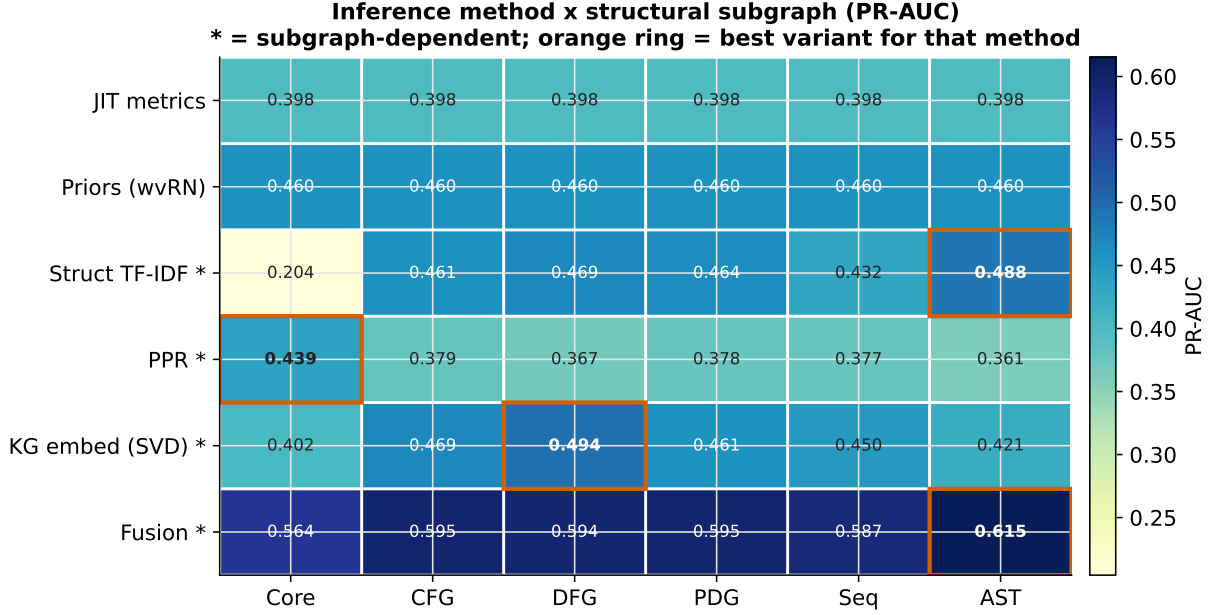


Figure 21: PR-AUC of every inference method (rows) under every structural subgraph (columns). The two references (M , R) are flat by construction; for each subgraph-dependent method (*) the best variant is ringed.

KGE — DistMult knowledge-graph embedding. A bilinear KG embedding over typed triples (c, ρ, h) with relation $\rho \in \{\text{TOKEN}, \text{FILE}, \text{DEV}\}$, scoring an edge by $\phi(c, \rho, h) = \sum_k e_c[k] w_\rho[k] e_h[k] = \langle \mathbf{e}_c, \mathbf{w}_\rho, \mathbf{e}_h \rangle$. Hub and relation embeddings are trained on the past triples by a logistic loss with negative (corrupted-tail) sampling; a commit — including an unseen one at arrival — is then *folded in* as the mean of its incident $\mathbf{w}_\rho \odot \mathbf{e}_h$, and a logistic head on that embedding gives the score. Small dimension, few epochs, periodic refit on the growing past keep it CPU-cheap.

Two graph scopes. To avoid confounding the structural subgraph with the shared relational layer, each method is run on two scopes: **subgraph** (commits linked only to their change-token hubs — the graph the subgraph actually defines) and **full** (token *plus* file and developer hubs — the deployed heterogeneous graph). Table 22 lists every method on all seven metrics at the AST subgraph; Table 23 and Figure 22 give the graph-native methods across subgraphs.

Findings. Two regimes emerge, cleanly separated by scope. *Random-walk and propagation* methods (RN, PPR, LP) are strongest on the relational (core/full) graph and are *diluted* by structural tokens — their signal is relational (file/developer), not structural; the pure relational methods also reach very high Recall at the cost of Precision (they flag buggy liberally). The *embedding* methods (DeepWalk, KGE) instead prefer the token-rich subgraphs and rank **AST** highest among them, echoing the structural TF-IDF result. On the AST subgraph the deployed **Fusion still dominates the balanced metrics** (Macro-F1, Buggy-F1, G-Mean, AUC, Accuracy; Table 22). Reporting both scopes removes the file/developer confound: the subgraph scope isolates *what the subgraph adds*, and there the embedding methods again favour AST.

B.10 Choosing the final fusion formula

The subgraph study fixes AST as the structural layer; we now choose the *inference fusion* for the complete method (Core + AST + CSTG, still without commit text X), with an explicit goal of a **purely graph-native** predictor. We ablate **all combinations** of ten inference methods on the final graph — the previous $M, R, T, P, E, G(\text{CSTG})$ and the five new graph-

Table 22: All inference methods on the AST subgraph, seven headline metrics: original methods, then the five graph-native methods on the subgraph-only and full scopes. Best per column in **bold**.

Method	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
JIT metrics (M)	0.303	0.840	0.484	0.445	0.559	0.679	0.487
Relational priors (R)	0.360	0.809	0.583	0.498	0.656	0.733	0.600
Structural TF-IDF (T)	0.402	0.727	0.632	0.518	0.687	0.751	0.668
Personalized PageRank (P)	0.361	0.819	0.584	0.501	0.658	0.707	0.600
KG embedding SVD (E)	0.365	0.739	0.594	0.489	0.657	0.692	0.621
Fusion ($M+T+R+P$)	0.510	0.750	0.719	0.607	0.758	0.826	0.762
<i>graph-native, scope = subgraph</i>							
Relational neighbour (RN)	0.301	0.906	0.460	0.452	0.535	0.638	0.461
PageRank RWR (PPR)	0.312	0.859	0.498	0.458	0.575	0.632	0.501
Label propagation (LP)	0.289	0.900	0.432	0.437	0.502	0.592	0.432
DeepWalk embed (DW)	0.370	0.732	0.599	0.492	0.660	0.709	0.629
KG embed DistMult (KGE)	0.347	0.778	0.568	0.480	0.638	0.681	0.586
<i>graph-native, scope = full</i>							
Relational neighbour (RN)	0.299	0.900	0.457	0.449	0.532	0.638	0.458
PageRank RWR (PPR)	0.361	0.819	0.584	0.501	0.658	0.707	0.600
Label propagation (LP)	0.302	0.884	0.471	0.451	0.547	0.642	0.472
DeepWalk embed (DW)	0.369	0.711	0.599	0.486	0.656	0.691	0.632
KG embed DistMult (KGE)	0.347	0.753	0.571	0.475	0.638	0.696	0.593

native *RN, LP, DW, KGE* — under two fusion styles (`inference/run_fusion_ablation.py`): *score stacking* (a prequential logistic regression over the selected methods’ probabilities; all $2^{10} - 1 = 1023$ subsets) and *feature fusion* (concatenated raw feature blocks of the top channels; all $2^6 - 1 = 63$ subsets). Every subset is scored on all seven headline metrics.

Single channels. Table 24 ranks the ten methods individually. Two facts drive the choice: the **CSTG semantic-text channel G is by far the strongest single signal** (Buggy-F1 0.596), and the **non-graph JIT metrics M are among the *weakest*** (Buggy-F1 0.476) — so dropping M costs almost nothing.

Combined fusions and the final choice. Table 25 and Figure 23 summarise the combinations. Performance saturates at three to four components; adding further methods (up to a seven-method graph-only stack, Buggy-F1 0.618) yields only marginal gains. We therefore adopt the compact, purely graph-native **Fusion** = $R + T + G$ (relational priors + AST structural change-tokens + CSTG semantics). Relative to the previous metrics-inclusive $M+T+R+P$ it improves the minority-class metrics — Recall 0.768 (+0.078), Buggy-F1 0.608 (+0.016), G-Mean 0.761 (+0.022), AUC 0.834 (+0.014), Macro-F1 0.716 (+0.002) — for small trades in Precision (−0.015) and Accuracy (−0.010), i.e. a healthier recall-oriented operating point for the minority buggy class, while **dropping the non-graph JIT metrics** and using fewer channels.

B.11 Full ablation: the best fusions at every size

The complete ablation evaluates all $2^{10} - 1 = 1023$ score-stacking combinations; reporting every row is impractical, so Tables 26–35 give, for each fusion size $k = 1, \dots, 10$, the **best eight k -method fusions** by Buggy-F1 (the last has a single 10-method combination), on all seven metrics. Two aggregate views summarise the 1023 points: Figure 24 shows the Buggy-F1 distribution at each size with the best-per-size envelope, and Figure 25 shows how often each method appears among the very best fusions.

Table 23: The five graph-native methods across structural subgraphs (Buggy-F1), on the subgraph-only and full graph scopes. Best subgraph per row in **bold**.

Method	Core	CFG	DFG	PDG	Seq	AST
<i>scope: subgraph</i>						
Relational neighbour (RN)	0.393	0.452	0.459	0.450	0.447	0.452
PageRank RWR (PPR)	0.393	0.466	0.464	0.458	0.462	0.458
Label propagation (LP)	0.393	0.446	0.453	0.444	0.442	0.437
DeepWalk embed (DW)	0.394	0.486	0.473	0.481	0.477	0.492
KG embed DistMult (KGE)	0.394	0.476	0.482	0.472	0.475	0.480
<i>scope: full</i>						
Relational neighbour (RN)	0.493	0.451	0.457	0.449	0.449	0.449
PageRank RWR (PPR)	0.499	0.501	0.508	0.498	0.502	0.501
Label propagation (LP)	0.459	0.459	0.468	0.455	0.456	0.451
DeepWalk embed (DW)	0.466	0.466	0.461	0.468	0.458	0.486
KG embed DistMult (KGE)	0.426	0.470	0.459	0.461	0.457	0.475

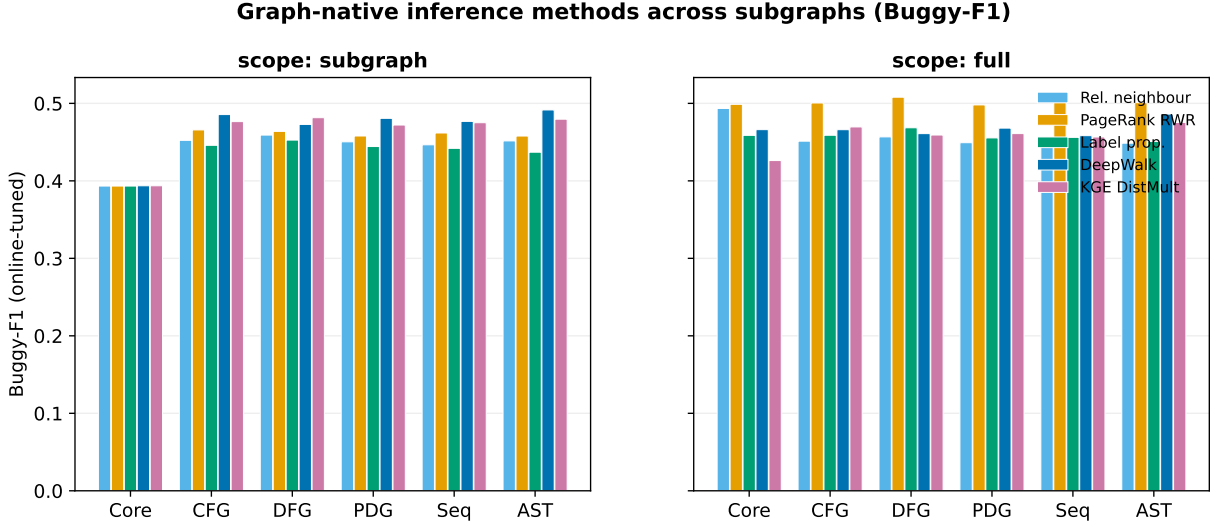


Figure 22: The five graph-native methods across subgraphs (Buggy-F1), on the subgraph-only (left) and full (right) scopes.

Reading the tables. Three patterns recur. (i) **Performance saturates early**: the best fusion already reaches Buggy-F1 ≈ 0.61 at $k = 3$ and only inches to 0.621 by $k = 8$, so large fusions are not worth their complexity. (ii) **R , T and G are near-universal** in the top fusions at every size (Figure 25), confirming them as the essential trio. (iii) **Graph-only fusions track the metrics-inclusive ones closely** at every size — e.g. at $k = 3$ the graph-only $R+T+G$ (0.610) trails the M -containing $M+R+G$ (0.613) by only 0.003 — so dropping the non-graph M costs almost nothing, which is exactly why the compact graph-only $R+T+G$ is adopted.

Table 24: Single inference methods on the final graph (Core+AST+CSTG, no X), seven headline metrics, ranked by Buggy-F1. CSTG (G) is the strongest single channel; the non-graph JIT metrics (M) are near the weakest.

Method	graph	#ch	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
G	✓	1	0.517	0.702	0.716	0.596	0.743	0.824	0.766
T	✓	1	0.418	0.710	0.646	0.526	0.694	0.759	0.687
P	✓	1	0.374	0.815	0.600	0.512	0.673	0.720	0.619
R	✓	1	0.388	0.752	0.618	0.512	0.680	0.746	0.648
KGE	✓	1	0.381	0.754	0.611	0.506	0.674	0.727	0.639
DW	✓	1	0.401	0.664	0.629	0.500	0.671	0.716	0.674
LP	✓	1	0.340	0.914	0.540	0.496	0.623	0.711	0.545
E	✓	1	0.425	0.579	0.641	0.490	0.657	0.716	0.705
M	–	1	0.346	0.765	0.568	0.476	0.636	0.701	0.587
RN	✓	1	0.317	0.901	0.498	0.469	0.577	0.652	0.500

Table 25: Candidate final Fusion formulas on the seven headline metrics (feature-fusion unless noted; online-tuned operating point). The recommended purely graph-native **R+T+G** improves on the previous metrics-inclusive $M+T+R+P$ on the minority-class metrics (Recall, Buggy-F1, G-Mean, AUC, Macro-F1) while dropping the non-graph JIT metrics and using fewer channels.

Fusion	graph	#ch	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
M+T+R+P (previous Fusion)	–	4	0.519	0.690	0.715	0.593	0.739	0.820	0.767
M+R+T+G (best overall)	–	4	0.541	0.727	0.734	0.620	0.762	0.832	0.782
R+T+P+G+RN+LP+KGE (best graph stack)	✓	7	0.549	0.708	0.735	0.618	0.758	0.843	0.786
R+T+P+G	✓	4	0.506	0.745	0.715	0.603	0.755	0.834	0.759
T+G (structure+semantics)	✓	2	0.495	0.768	0.711	0.602	0.757	0.827	0.751
R+T+G (recommended)	✓	3	0.504	0.768	0.716	0.608	0.761	0.834	0.758

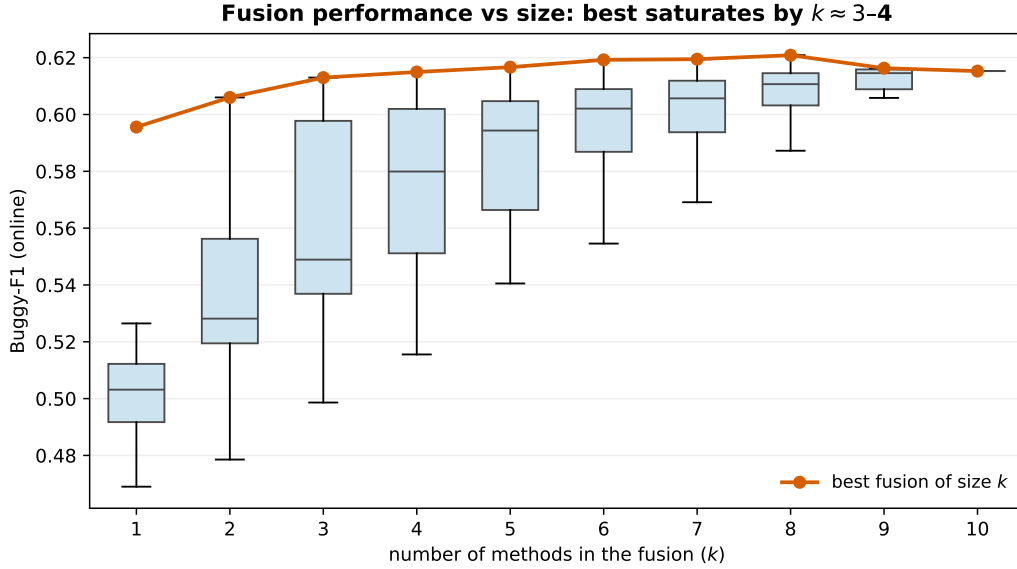


Figure 24: Buggy-F1 of all fusions at each size k (boxes) with the best-per-size envelope (orange). The best value saturates by $k \approx 3-4$.

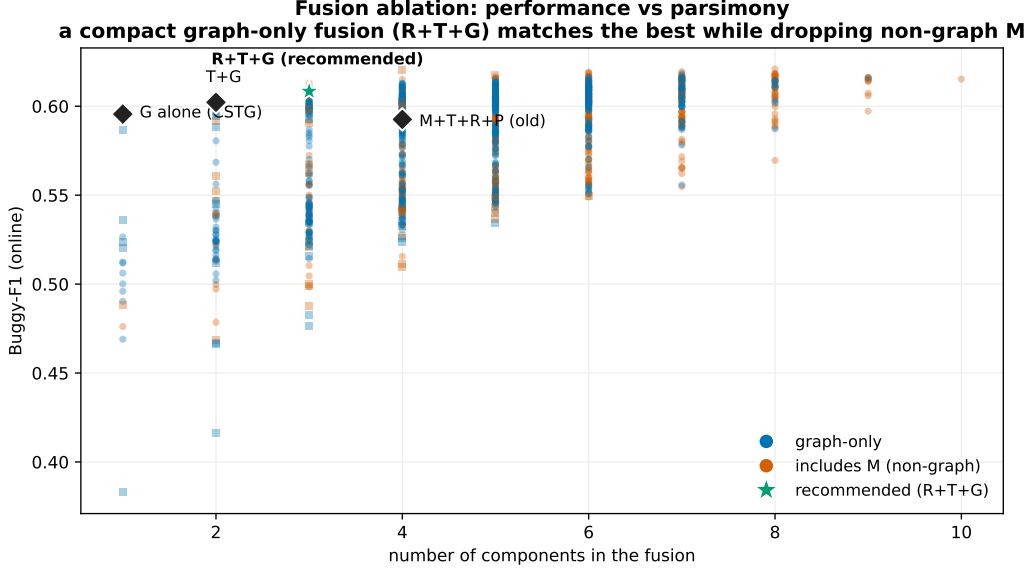


Figure 23: Fusion ablation — Buggy-F1 versus number of components. Blue = graph-only, orange = includes the non-graph M . Performance saturates by three to four components; the recommended graph-only $\mathbf{R+T+G}$ (green star) sits near the top of the cloud with only three channels.

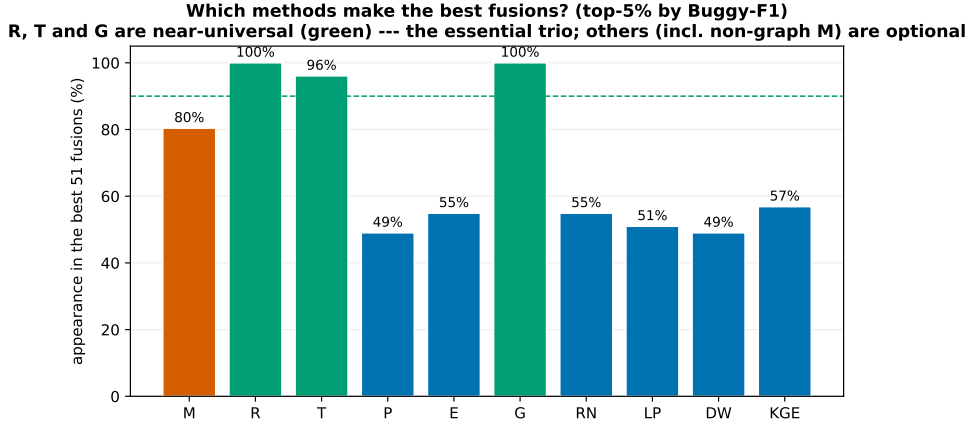


Figure 25: How often each method appears in the best 5% of fusions (by Buggy-F1). R , T and G (green) are near-universal — the essential trio; the remaining methods, including the non-graph M , are optional.

Table 26: Best 8 fusion(s) of $k = 1$ method by Buggy-F1 (score-stacking over the ten inference methods; final Core+AST+CSTG graph, no X). “graph” = purely graph-native (no non-graph JIT metrics M).

Fusion	graph	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
G	✓	0.517	0.702	0.716	0.596	0.743	0.824	0.766
T	✓	0.418	0.710	0.646	0.526	0.694	0.759	0.687
P	✓	0.374	0.815	0.600	0.512	0.673	0.720	0.619
R	✓	0.388	0.752	0.618	0.512	0.680	0.746	0.648
KGE	✓	0.381	0.754	0.611	0.506	0.674	0.727	0.639
DW	✓	0.401	0.664	0.629	0.500	0.671	0.716	0.674
LP	✓	0.340	0.914	0.540	0.496	0.623	0.711	0.545
E	✓	0.425	0.579	0.641	0.490	0.657	0.716	0.705

Table 27: Best 8 fusion(s) of $k = 2$ methods by Buggy-F1 (score-stacking over the ten inference methods; final Core+AST+CSTG graph, no X). “graph” = purely graph-native (no non-graph JIT metrics M).

Fusion	graph	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
R+G	✓	0.521	0.724	0.721	0.606	0.753	0.833	0.769
G+KGE	✓	0.535	0.694	0.725	0.604	0.747	0.833	0.777
T+G	✓	0.517	0.715	0.718	0.600	0.748	0.834	0.766
P+G	✓	0.509	0.731	0.715	0.600	0.751	0.829	0.761
G+LP	✓	0.521	0.707	0.719	0.600	0.747	0.828	0.769
M+G	–	0.509	0.725	0.714	0.598	0.749	0.826	0.761
G+DW	✓	0.515	0.713	0.716	0.598	0.747	0.828	0.765
G+RN	✓	0.520	0.701	0.717	0.597	0.744	0.827	0.768

Table 28: Best 8 fusion(s) of $k = 3$ methods by Buggy-F1 (score-stacking over the ten inference methods; final Core+AST+CSTG graph, no X). “graph” = purely graph-native (no non-graph JIT metrics M).

Fusion	graph	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
M+R+G	–	0.512	0.764	0.721	0.613	0.764	0.836	0.763
R+G+RN	✓	0.532	0.718	0.727	0.611	0.755	0.834	0.776
R+G+KGE	✓	0.525	0.729	0.725	0.610	0.757	0.839	0.772
R+T+G	✓	0.525	0.727	0.724	0.610	0.756	0.843	0.772
P+G+KGE	✓	0.530	0.712	0.725	0.608	0.753	0.835	0.775
R+G+LP	✓	0.518	0.734	0.721	0.608	0.756	0.834	0.768
R+E+G	✓	0.531	0.711	0.725	0.608	0.752	0.837	0.775
M+T+G	–	0.513	0.741	0.719	0.606	0.756	0.835	0.764

Table 29: Best 8 fusion(s) of $k = 4$ methods by Buggy-F1 (score-stacking over the ten inference methods; final Core+AST+CSTG graph, no X). “graph” = purely graph-native (no non-graph JIT metrics M).

Fusion	graph	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
M+R+T+G	–	0.518	0.756	0.724	0.615	0.764	0.845	0.768
R+T+G+RN	✓	0.531	0.724	0.727	0.613	0.758	0.843	0.776
R+T+G+DW	✓	0.519	0.747	0.724	0.613	0.761	0.843	0.768
R+T+P+G	✓	0.530	0.726	0.727	0.612	0.758	0.843	0.775
R+G+RN+KGE	✓	0.532	0.720	0.727	0.612	0.756	0.840	0.776
R+T+G+KGE	✓	0.531	0.721	0.727	0.611	0.756	0.843	0.775
R+G+RN+LP	✓	0.529	0.724	0.726	0.611	0.757	0.834	0.774
M+R+G+RN	–	0.512	0.756	0.721	0.611	0.761	0.837	0.764

Table 30: Best 8 fusion(s) of $k = 5$ methods by Buggy-F1 (score-stacking over the ten inference methods; final Core+AST+CSTG graph, no X). “graph” = purely graph-native (no non-graph JIT metrics M).

Fusion	graph	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
M+R+T+G+DW	–	0.531	0.734	0.729	0.617	0.762	0.845	0.776
M+R+T+G+RN	–	0.524	0.746	0.727	0.616	0.763	0.845	0.772
R+T+G+RN+KGE	✓	0.539	0.717	0.731	0.616	0.758	0.843	0.780
M+R+T+G+KGE	–	0.527	0.740	0.727	0.615	0.762	0.845	0.773
M+R+T+E+G	–	0.522	0.750	0.726	0.615	0.763	0.845	0.770
M+R+T+P+G	–	0.532	0.727	0.729	0.615	0.759	0.845	0.776
M+R+T+G+LP	–	0.522	0.746	0.725	0.614	0.762	0.845	0.770
R+G+RN+LP+KGE	✓	0.537	0.716	0.730	0.614	0.757	0.840	0.779

Table 31: Best 8 fusion(s) of $k = 6$ methods by Buggy-F1 (score-stacking over the ten inference methods; final Core+AST+CSTG graph, no X). “graph” = purely graph-native (no non-graph JIT metrics M).

Fusion	graph	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
M+R+T+G+RN+DW	–	0.529	0.747	0.730	0.619	0.765	0.845	0.775
M+R+T+E+G+DW	–	0.529	0.744	0.730	0.619	0.764	0.844	0.775
M+R+T+E+G+KGE	–	0.542	0.719	0.733	0.618	0.760	0.845	0.782
M+R+T+G+RN+LP	–	0.527	0.744	0.728	0.617	0.763	0.845	0.774
M+R+T+P+G+RN	–	0.533	0.731	0.730	0.617	0.761	0.845	0.777
R+T+P+G+RN+KGE	✓	0.542	0.715	0.732	0.616	0.758	0.843	0.782
R+T+G+RN+LP+KGE	✓	0.540	0.718	0.731	0.616	0.758	0.843	0.781
M+R+T+G+LP+DW	–	0.529	0.736	0.728	0.616	0.761	0.845	0.775

Table 32: Best 8 fusion(s) of $k = 7$ methods by Buggy-F1 (score-stacking over the ten inference methods; final Core+AST+CSTG graph, no X). “graph” = purely graph-native (no non-graph JIT metrics M).

Fusion	graph	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
M+R+T+P+E+G+KGE	–	0.553	0.705	0.736	0.619	0.758	0.844	0.788
R+T+P+G+RN+LP+KGE	✓	0.549	0.708	0.735	0.618	0.758	0.843	0.786
M+R+T+G+RN+LP+DW	–	0.529	0.741	0.729	0.618	0.763	0.845	0.775
M+R+T+P+G+LP+KGE	–	0.534	0.732	0.730	0.617	0.762	0.845	0.777
M+R+T+P+G+RN+DW	–	0.542	0.717	0.732	0.617	0.759	0.845	0.782
M+R+E+G+RN+LP+KGE	–	0.537	0.724	0.731	0.617	0.760	0.843	0.779
M+R+T+P+G+RN+KGE	–	0.536	0.725	0.730	0.616	0.760	0.845	0.779
R+T+P+G+LP+DW+KGE	✓	0.543	0.712	0.732	0.616	0.757	0.842	0.782

Table 33: Best 8 fusion(s) of $k = 8$ methods by Buggy-F1 (score-stacking over the ten inference methods; final Core+AST+CSTG graph, no X). “graph” = purely graph-native (no non-graph JIT metrics M).

Fusion	graph	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
M+R+T+P+G+RN+LP+DW	–	0.536	0.738	0.732	0.621	0.765	0.845	0.779
M+R+T+P+E+G+RN+KGE	–	0.554	0.700	0.736	0.618	0.756	0.845	0.788
M+R+T+P+E+G+LP+KGE	–	0.554	0.700	0.736	0.618	0.756	0.845	0.788
M+R+T+P+E+G+LP+DW	–	0.542	0.718	0.733	0.618	0.759	0.844	0.782
M+R+T+E+G+RN+LP+DW	–	0.535	0.731	0.731	0.618	0.762	0.844	0.778
M+R+T+E+G+LP+DW+KGE	–	0.544	0.712	0.733	0.617	0.758	0.844	0.783
M+R+T+E+G+RN+LP+KGE	–	0.542	0.715	0.732	0.617	0.758	0.845	0.782
M+R+T+G+RN+LP+DW+KGE	–	0.536	0.726	0.730	0.617	0.760	0.845	0.779

Table 34: Best 8 fusion(s) of $k = 9$ methods by Buggy-F1 (score-stacking over the ten inference methods; final Core+AST+CSTG graph, no X). “graph” = purely graph-native (no non-graph JIT metrics M).

Fusion	graph	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
M+R+T+P+E+G+RN+LP+DW	–	0.543	0.713	0.732	0.616	0.758	0.844	0.782
M+R+T+P+E+G+RN+DW+KGE	–	0.542	0.713	0.732	0.616	0.757	0.844	0.782
M+R+T+E+G+RN+LP+DW+KGE	–	0.540	0.718	0.731	0.616	0.758	0.844	0.781
R+T+P+E+G+RN+LP+DW+KGE	✓	0.547	0.704	0.733	0.615	0.755	0.842	0.784
M+R+T+P+E+G+LP+DW+KGE	–	0.546	0.704	0.732	0.615	0.755	0.844	0.784
M+R+T+P+E+G+RN+LP+KGE	–	0.549	0.698	0.733	0.614	0.753	0.845	0.785
M+R+T+P+G+RN+LP+DW+KGE	–	0.532	0.726	0.728	0.614	0.758	0.845	0.776
M+T+P+E+G+RN+LP+DW+KGE	–	0.528	0.715	0.724	0.607	0.753	0.838	0.773

Table 35: Best 1 fusion(s) of $k = 10$ methods by Buggy-F1 (score-stacking over the ten inference methods; final Core+AST+CSTG graph, no X). “graph” = purely graph-native (no non-graph JIT metrics M).

Fusion	graph	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
M+R+T+P+E+G+RN+LP+DW+KGE	–	0.548	0.701	0.733	0.615	0.755	0.844	0.785

B.12 Selecting five inference methods for the final study

The two research questions — *which subgraph?* and *which inference method?* — are answered together by one table whose rows are inference methods and whose columns are a family of knowledge graphs of increasing richness. This requires committing to a small set of inference methods that (i) are *typical* of graph/KG inference, (ii) use the KG’s structure and modelling as fully as possible, and (iii) apply uniformly to *every* graph in the family (so each is a valid row). We choose the **five canonical graph-inference paradigms**:

Method	Paradigm	Structure exploited
RN	collective classification (wvRN)	label-weighted 1-hop neighbourhood
PPR	random walk with restart	multi-hop connectivity to class seeds
LP	label propagation	global label diffusion over edges
DW	random-walk node embedding (DeepWalk)	unsupervised node geometry
KGE	knowledge-graph embedding (DistMult)	typed (commit, rel, hub) relations

Why these five. They map one-to-one onto the recognised families of graph and knowledge-graph inference, so their “typicality” is beyond dispute; each reads a *different, complementary* facet of the structure (neighbourhood, walk, diffusion, node embedding, typed-edge embedding), which both maximises use of the modelling and makes their fusion strong; and, crucially, all five are graph *algorithms* that run unchanged on any graph, so they can be a common set of rows across the whole column family. In contrast, the feature-based channels that were individually strongest in the ablation — structural TF-IDF (T) and the CSTG channel (G) — are *not* admissible as rows: T ignores topology (a bag-of-token classifier, not graph inference), and G is specific to the CSTG layer and undefined for the Core-only column. Their signal is instead **absorbed by the five methods through the graph itself**: on a graph whose hubs include AST change-tokens the walks and embeddings traverse the structural signal, and on the final graph, whose hubs additionally include *CSTG term nodes* (the (Commit) $\rightarrow$ (Term) MENTIONS edges), they traverse the semantic signal as well. Fixing the five methods and *enriching the graph* across the columns is therefore both the fairer and the higher-performing design: the richest graph (the final KG) gives every method its best result, and the five-method fusion on it is strongest.

The graph family (columns). We evaluate the five methods on six knowledge graphs of increasing richness, all sharing the Core relational layer (commit – file – developer): *Core*; *Core+AST*; *Core+CFG*; *Core+DFG*; *Core+PDG*; and the *final Core+AST+CSTG*. Reading a row shows a method’s sensitivity to graph richness (the *which-subgraph* RQ); reading a column shows which method wins on a given graph (the *which-method* RQ).

Two auxiliary decisions. First, the deployed *final Fusion* combines the five methods on the final KG and *may* append the non-graph JIT metrics (M) as a single auxiliary feature; from the ablation M contributes little, so we report both the graph-only five-method fusion and the fusion+ M variant and prefer the former unless M measurably helps. Second, the commit-text channel (X) is **removed entirely** from the methodology and from all future work; it is disabled throughout this study and its remaining traces in the codebase are to be deleted.

B.13 Final experiments: five methods across the graph family

We run the five canonical graph-inference methods online (prequentially) on the six-graph family and report all seven metrics (`inference/run_final_experiments.py`). Tables 36–42 give one table per metric — rows are methods, columns are the six graphs, and the best method per graph is in **bold**. Together they answer both research questions in one place: reading a row shows a method’s response to graph richness, reading a column shows the best method on a given graph.

What the tables show. Graph richness helps the informative methods: for PPR, LP and the embeddings the metrics rise from Core to the final *Core+AST+CSTG* graph, and **PPR on the final graph is the best single (method, graph) cell** (Buggy-F1 ≈ 0.555 , AUC ≈ 0.78 , G-Mean ≈ 0.72). PPR is the strongest method overall, with LP and the DeepWalk/DistMult embeddings close behind; the pure relational neighbour (RN) is flat and slightly prefers the Core graph, since a one-hop label vote is diluted rather than helped by structural and semantic hubs. These five methods are the inputs to the deployed fusion on the final KG.

Table 36: Online Prec. of the five graph-inference methods across the six knowledge graphs (Groovy; no X; online-tuned operating point). Best method per graph in **bold**.

Method	Core	+AST	+CFG	+DFG	+PDG	Final
Relational neighbour	0.373	0.299	0.306	0.312	0.304	0.304
Personalized PageRank	0.371	0.361	0.357	0.365	0.354	0.430
Label propagation	0.318	0.302	0.310	0.321	0.307	0.380
DeepWalk embedding	0.264	0.342	0.339	0.339	0.333	0.368
KG embedding (DistMult)	0.299	0.373	0.373	0.310	0.311	0.376

Table 37: Online Rec. of the five graph-inference methods across the six knowledge graphs (Groovy; no X; online-tuned operating point). Best method per graph in **bold**.

Method	Core	+AST	+CFG	+DFG	+PDG	Final
Relational neighbour	0.730	0.900	0.857	0.853	0.861	0.900
Personalized PageRank	0.761	0.819	0.839	0.835	0.841	0.781
Label propagation	0.820	0.884	0.884	0.866	0.884	0.788
DeepWalk embedding	0.792	0.638	0.788	0.797	0.788	0.726
KG embedding (DistMult)	0.827	0.727	0.752	0.811	0.802	0.705

Table 38: Online Macro-F1 of the five graph-inference methods across the six knowledge graphs (Groovy; no X; online-tuned operating point). Best method per graph in **bold**.

Method	Core	+AST	+CFG	+DFG	+PDG	Final
Relational neighbour	0.603	0.457	0.487	0.499	0.481	0.470
Personalized PageRank	0.600	0.584	0.576	0.588	0.572	0.660
Label propagation	0.518	0.471	0.487	0.514	0.481	0.610
DeepWalk embedding	0.407	0.573	0.556	0.555	0.548	0.598
KG embedding (DistMult)	0.479	0.603	0.602	0.505	0.510	0.607

Table 39: Online Buggy-F1 of the five graph-inference methods across the six knowledge graphs (Groovy; no X; online-tuned operating point). Best method per graph in **bold**.

Method	Core	+AST	+CFG	+DFG	+PDG	Final
Relational neighbour	0.493	0.449	0.451	0.457	0.449	0.454
Personalized PageRank	0.499	0.501	0.501	0.508	0.498	0.555
Label propagation	0.459	0.451	0.459	0.468	0.455	0.513
DeepWalk embedding	0.396	0.445	0.474	0.476	0.469	0.488
KG embedding (DistMult)	0.439	0.493	0.498	0.449	0.449	0.491

Table 40: Online G-Mean of the five graph-inference methods across the six knowledge graphs (Groovy; no X; online-tuned operating point). Best method per graph in **bold**.

Method	Core	+AST	+CFG	+DFG	+PDG	Final
Relational neighbour	0.662	0.532	0.563	0.576	0.557	0.545
Personalized PageRank	0.665	0.658	0.653	0.664	0.649	0.720
Label propagation	0.594	0.547	0.565	0.593	0.558	0.678
DeepWalk embedding	0.473	0.620	0.629	0.628	0.621	0.658
KG embedding (DistMult)	0.553	0.662	0.666	0.580	0.584	0.661

Table 41: Online AUC of the five graph-inference methods across the six knowledge graphs (Groovy; no X; online-tuned operating point). Best method per graph in **bold**.

Method	Core	+AST	+CFG	+DFG	+PDG	Final
Relational neighbour	0.696	0.638	0.606	0.632	0.601	0.647
Personalized PageRank	0.728	0.707	0.713	0.709	0.711	0.780
Label propagation	0.687	0.642	0.660	0.664	0.657	0.735
DeepWalk embedding	0.576	0.666	0.677	0.658	0.674	0.710
KG embedding (DistMult)	0.629	0.709	0.702	0.645	0.629	0.713

Table 42: Online Acc. of the five graph-inference methods across the six knowledge graphs (Groovy; no X; online-tuned operating point). Best method per graph in **bold**.

Method	Core	+AST	+CFG	+DFG	+PDG	Final
Relational neighbour	0.633	0.458	0.489	0.503	0.483	0.470
Personalized PageRank	0.625	0.600	0.590	0.603	0.584	0.693
Label propagation	0.526	0.472	0.489	0.519	0.482	0.633
DeepWalk embedding	0.407	0.611	0.572	0.569	0.562	0.628
KG embedding (DistMult)	0.482	0.633	0.629	0.512	0.517	0.641

Online-evaluation streams. Because the protocol is prequential, each metric has a trajectory over the commit stream. We produce the full set in the companion notebook (`experiments/notebooks/final_e`) a *by-method* view (five figures $\times$ seven metric panels = 35 streams, trends = graphs) and a *by-graph* view (six figures $\times$ seven panels = 42 streams, trends = methods). Figure 26 is the representative by-graph panel for the final KG: the method ordering (PPR ahead of LP, the embeddings, and RN) holds across the entire stream and across all seven metrics, so the conclusions are not artefacts of a single operating point.

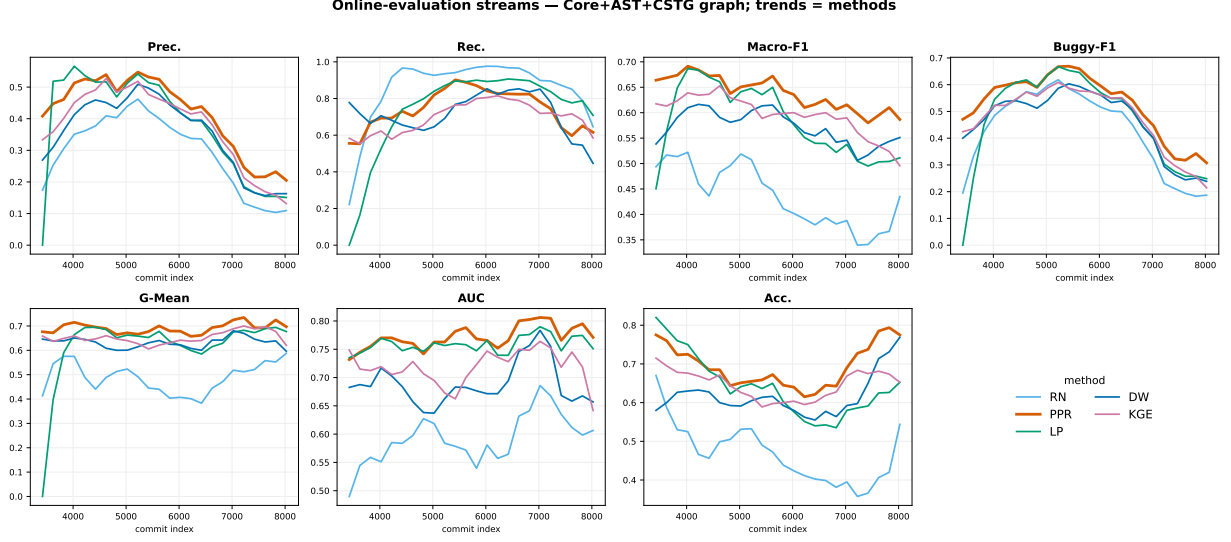


Figure 26: Online-evaluation streams on the final *Core+AST+CSTG* graph: each panel is one of the seven metrics over the commit stream, with the five methods as trends (PPR in bold). PPR leads throughout on the balanced metrics.

B.14 Choosing the final fusion of the five methods

Having fixed the five methods and the final graph, we choose how to *fuse* them (`inference/run_final_fusion.py`). We stack the five methods’ probabilities with a prequential logistic regression and evaluate **all** $2^5 - 1 = 31$ **combinations** on the final KG (Table 43). The final fusion is picked to balance *load* (fewer methods, cheaper online) against *performance*, prioritising the two class-balanced metrics Macro-F1 and G-Mean: we take the smallest combination whose $(\text{Macro-F1} + \text{G-Mean})/2$ is within 0.005 of the best. This selects $F = \mathbf{RN+PPR}$ — the relational-neighbour and random-walk methods — with Macro-F1 0.670 and G-Mean 0.731; adding the remaining three methods does not improve the balance (the five-method fusion reaches only 0.695 versus F ’s 0.701), so two methods suffice.

Table 43: All $2^5 - 1 = 31$ fusions of the five graph-inference methods on the final Core+AST+CSTG graph (prequential LR stacking), ranked by $(\text{Macro-F1} + \text{G-Mean})/2$. The chosen fusion $F = \text{RN+PPR}$ — the fewest methods within 0.005 of the best balance — is highlighted.

#	Fusion	k	bal.	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
1	RN+PPR	2	0.701	0.443	0.802	0.670	0.571	0.731	0.787	0.701
2	RN+PPR+LP	3	0.698	0.437	0.811	0.666	0.568	0.729	0.788	0.694
3	RN+PPR+LP+DW+KGE	5	0.695	0.442	0.767	0.668	0.561	0.723	0.787	0.702
4	RN+PPR+LP+KGE	4	0.691	0.445	0.729	0.668	0.553	0.714	0.783	0.707
5	RN+PPR+LP+DW	4	0.690	0.436	0.765	0.663	0.556	0.718	0.788	0.696
6	PPR	1	0.689	0.431	0.787	0.659	0.557	0.719	0.782	0.689
7	RN+LP+DW+KGE	4	0.688	0.437	0.747	0.662	0.552	0.714	0.773	0.699
8	RN+PPR+DW+KGE	4	0.686	0.432	0.760	0.658	0.551	0.714	0.783	0.692
9	PPR+LP+DW+KGE	4	0.685	0.436	0.732	0.661	0.547	0.710	0.782	0.699
10	PPR+LP+DW	3	0.684	0.425	0.792	0.653	0.553	0.715	0.783	0.682
11	PPR+DW+KGE	3	0.683	0.430	0.753	0.656	0.547	0.710	0.774	0.691
12	RN+PPR+KGE	3	0.683	0.433	0.732	0.658	0.544	0.708	0.777	0.696
13	RN+PPR+DW	3	0.683	0.428	0.756	0.655	0.547	0.710	0.780	0.689
14	PPR+LP	2	0.683	0.420	0.820	0.649	0.555	0.716	0.781	0.674
15	PPR+LP+KGE	3	0.680	0.428	0.737	0.654	0.542	0.706	0.775	0.691
16	LP+DW+KGE	3	0.679	0.427	0.733	0.653	0.540	0.704	0.768	0.690
17	RN+LP+KGE	3	0.677	0.421	0.756	0.648	0.541	0.705	0.764	0.681
18	PPR+KGE	2	0.677	0.425	0.730	0.651	0.537	0.702	0.764	0.688
19	LP+DW	2	0.673	0.412	0.792	0.641	0.542	0.705	0.765	0.668
20	RN+LP	2	0.673	0.410	0.816	0.639	0.546	0.707	0.765	0.663
21	RN+LP+DW	3	0.672	0.412	0.783	0.641	0.540	0.703	0.769	0.669
22	PPR+DW	2	0.670	0.413	0.759	0.641	0.535	0.699	0.770	0.672
23	RN+DW+KGE	3	0.669	0.411	0.769	0.639	0.535	0.699	0.760	0.669
24	LP+KGE	2	0.665	0.407	0.758	0.635	0.530	0.694	0.757	0.666
25	DW+KGE	2	0.664	0.413	0.713	0.640	0.523	0.689	0.749	0.677
26	RN+KGE	2	0.660	0.404	0.741	0.632	0.523	0.688	0.741	0.664
27	RN+DW	2	0.652	0.395	0.745	0.623	0.516	0.681	0.737	0.653
28	LP	1	0.633	0.374	0.774	0.600	0.505	0.666	0.736	0.623
29	KGE	1	0.628	0.371	0.739	0.598	0.494	0.659	0.711	0.625
30	DW	1	0.624	0.367	0.747	0.593	0.492	0.656	0.711	0.618
31	RN	1	0.516	0.305	0.855	0.479	0.449	0.552	0.653	0.480

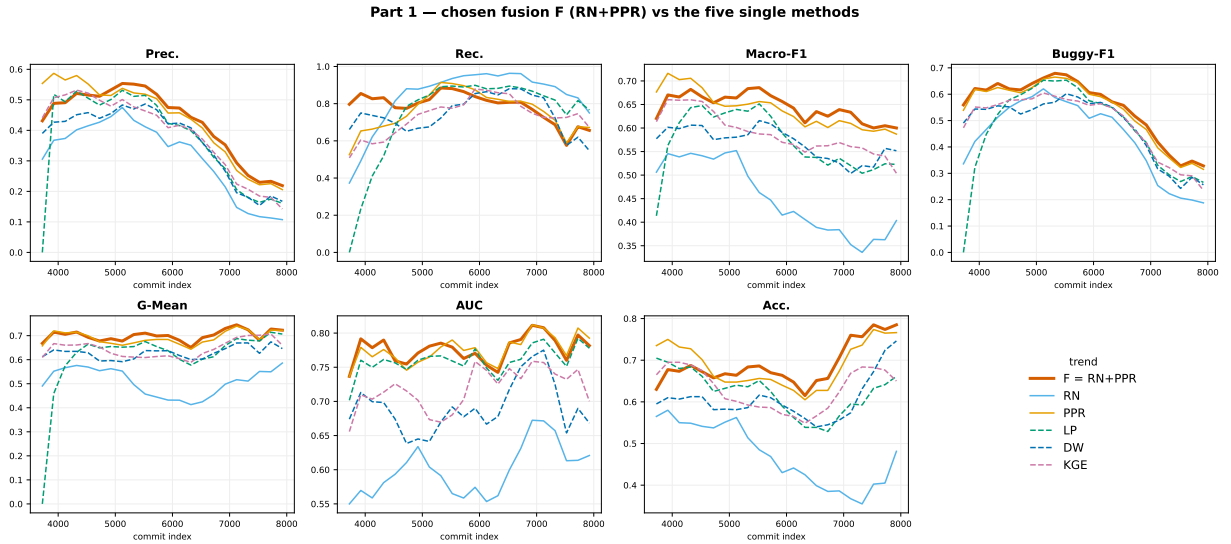


Figure 27: Online-evaluation streams of the chosen fusion $F = \text{RN+PPR}$ (bold) against the five single methods, one panel per metric. F tracks at or above the best single method throughout.

Adding the CSTG channel and the JIT metrics. Finally we ask whether the explicit CSTG semantic channel (G) or the non-graph JIT metrics (M) add anything on top of F (Table 44, Figure 28). **Adding G helps markedly** — Macro-F1 $0.670 \rightarrow 0.729$, AUC $0.787 \rightarrow 0.844$,

Buggy-F1 $0.571 \rightarrow 0.621$ — even though the final graph already carries CSTG term-hubs, because the explicit semantic-text classifier captures signal the walks do not fully exploit through topology alone. **Adding M alone hurts** (Macro-F1 drops to 0.648) and contributes nothing over G ($F+G+M \approx F+G$). The deployed model is therefore the purely graph-native $F+G = \mathbf{RN+PPR+CSTG}$, without the non-graph JIT metrics.

Table 44: Effect of appending the CSTG channel (G) and the non-graph JIT metrics (M) to the chosen graph-method fusion $F = \mathbf{RN+PPR}$ on the final KG. Best per column in **bold**. Adding G helps markedly; adding M alone hurts and adds nothing over G .

Fusion	Prec.	Rec.	Macro-F1	Buggy-F1	G-Mean	AUC	Acc.
F	0.443	0.802	0.670	0.571	0.731	0.787	0.701
F+G	0.527	0.755	0.729	0.621	0.766	0.844	0.771
F+M	0.419	0.798	0.648	0.549	0.712	0.787	0.675
F+G+M	0.548	0.722	0.736	0.623	0.762	0.844	0.784



Figure 28: Online-evaluation streams of F , $F+G$, $F+M$ and $F+G+M$, one panel per metric. $F+G$ and $F+G+M$ track together at the top; $F+M$ is lowest on Macro-F1 and AUC — the non-graph metrics add noise, not signal.

C Scalability, Complexity and Statistical Analysis

The V4 study establishes *which* representation and methods are best; this section establishes that the resulting system is *scalable*, *cheap to run*, and *statistically sound*. We analyse all four phases — the Core graph, the structural-subgraph family, the CSTG layer, and the five graph-inference methods with their deployed fusion — along four axes: a statistical profile of the final graph, per-commit growth, build/predict complexity, and significance testing of the headline comparisons. Every number here is obtained **without rebuilding the knowledge graph**: final sizes and growth come from read-only snapshots of the assembled database, prediction latency from replaying the cached graph family in-memory, and build cost from a sampled replay of the real parse-and-diff work. The analysis code is in `scalability/` and the companion notebook is `experiments/notebooks/scalability_v4.ipynb`.

Reproducibility on new projects. The per-commit build wall-clock was not persisted for the Groovy build (only deltas are stored). Both online engines therefore now accept an opt-in

Table 45: Statistical profile of each materialised layer of the final knowledge graph (Groovy, 8,059 commits). *Med. nodes/file* and *Gini* describe the per-file size distribution; α is the MLE power-law exponent of the per-commit change-token degree (heavy-tailed for $2 < \alpha < 3$). The CSTG row counts its **Term** nodes and its semantic edges (MENTIONS+COOCCURS+GROUNDS_IN).

Layer	Nodes	Delta-edges	Node types	Token types	Med. nodes /file	Gini	α
AST	3,699,439	3,614,922	136	314	286	.78	2.05
CFG	372,488	408,301	18	53	43	.73	2.15
DFG	469,662	648,553	7	27	34	.81	2.27
PDG	372,488	411,869	18	63	43	.73	2.17
Token-seq	257,708	293,287	16	47	25	.76	2.10
CSTG	20,377	328,090	5	—	—	—	—

-timing-log flag that appends one row per commit (file actions, delta counts, wall-ms) at build time; it is off by default, so the Groovy database is untouched, and on for any *new* project it captures the full per-commit timing natively.

C.1 Statistical profile of the final graph

Table 45 profiles every materialised layer. Beyond the raw scale (recovered exactly: AST 3,699,439 nodes, matching Table 18), two distributional facts matter. First, per-file sizes are highly skewed (Gini ≈ 0.6 – 0.7): a few large files dominate each layer, as expected of real software. Second, the per-commit change size is **heavy-tailed** on every layer, with a maximum-likelihood power-law exponent $\alpha \in [2.0, 2.3]$ — the regime ($2 < \alpha < 3$) in which the mean is finite but the variance is not, i.e. most commits are tiny and a few are enormous (Figure 29). This is precisely the distribution a delta-based store is designed for: storage follows the (small, typical) change, not a worst-case snapshot. The CSTG layer adds a compact, bounded semantic increment (20,377 **Term** nodes; ≈ 20 MENTIONS per commit).

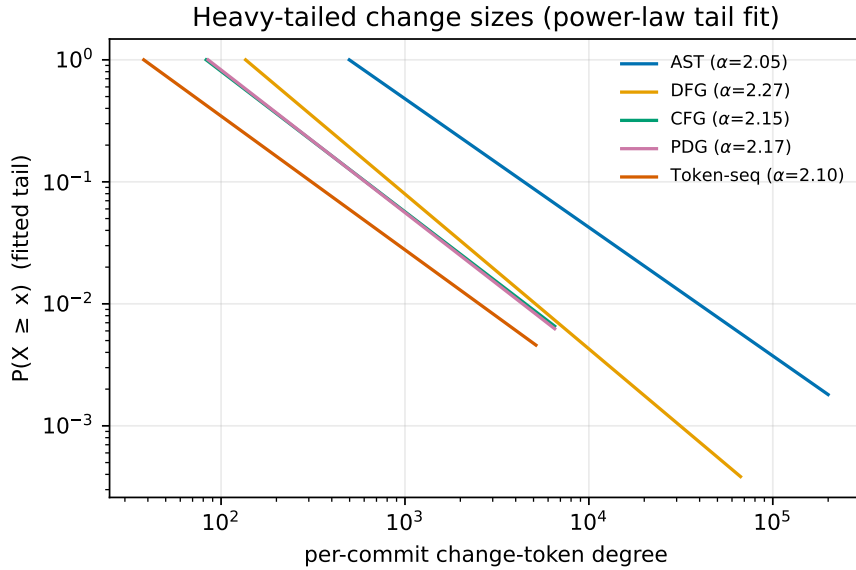


Figure 29: Fitted power-law tail of the per-commit change-token degree for each layer ($\alpha \in [2.0, 2.3]$): change sizes are heavy-tailed everywhere, so delta storage cost is dominated by a small number of large commits.

Table 46: Per-commit growth and modification statistics recovered from the online delta stream. *Churn* is REMOVES/ADDS; buggy/benign columns are mean per-commit delta-edges; the ratio carries a bootstrap 95% CI and a Mann–Whitney U p -value for the (buggy > benign) size separation.

Layer	Total delta	Churn	Med. /commit	Buggy mean	Benign mean	Ratio [CI]	(p)
AST	3,614,922	.44	84	909	366	2.48 [1.82,3.48]	<1e-99
CFG	408,301	.51	18	108	53	2.03 [1.71,2.41]	3e-91
DFG	648,553	.38	31	208	75	2.78 [1.88,4.34]	<1e-99
PDG	411,869	.51	18	109	53	2.07 [1.74,2.46]	1e-97
SEQ	293,287	.53	14	80	39	2.05 [1.72,2.43]	<1e-99

C.2 Per-commit growth and modification

Because commits are ingested in `author_ts` order and only deltas are stored, the graph grows with the *amount of change*, not with history length. Table 46 reports, per layer, the total delta volume, the churn ratio (REMOVES/ADDS ≈ 0.4 – 0.5 : the tree mostly grows, with substantial but minority deletion), and the buggy-vs-benign change-size gap. **Buggy commits make significantly larger structural changes** on every layer: the mean per-commit delta ratio is 2.0 – $2.8\times$ (bootstrap 95% CI excludes 1 throughout) and a one-sided Mann–Whitney test rejects equality with $p \ll 10^{-90}$ (for AST, $p \approx 7 \times 10^{-133}$). This size separation — visible in Figure 30 together with the pronounced yearly bug-rate drift — is the structural basis of the graph’s predictive value.

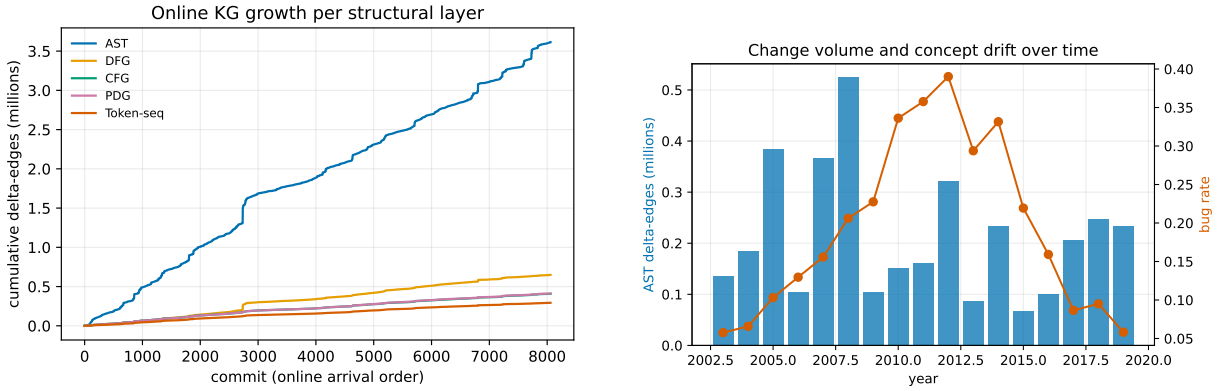


Figure 30: Left: cumulative delta-edges per layer over the commit stream — smooth, near-linear online growth. Right: AST change volume and the yearly bug rate, showing the concept drift that motivates the time-aware protocol.

C.3 Time and space complexity

Build / modify. Per-commit work is the parse of the changed file(s) plus a tree/graph diff against the stored state; space is $O(\text{alive nodes} + \text{retained history})$, with per-commit growth $O(\text{change})$ and no full-snapshot-per-commit cost. Table 47 quantifies the CPU parse-and-diff cost from a sampled replay of real modified-file pairs (Neo4j write I/O excluded — it is dominated by the schema-index cost documented in §B.4). The time-vs-file-size fit is near-perfect ($R^2 \approx 0.99$), confirming the cost is linear in the amount of code processed; AST is $\approx 20\times$ costlier per file than the coarse layers (402 vs. 17–25 ms), exactly its $\approx 10\times$ node-granularity plus tree-diff overhead (Figure 31, left).

Table 47: Build/modify cost from a sampled replay of the real parse+diff work (CPU only; Neo4j write I/O excluded). The high R^2 of the time-vs-change-size fit confirms per-commit work is *linear in the size of the change* ($O(\text{change})$), independent of history length. The last column extrapolates the median per-file cost to the full stream.

Layer	#samp.	Med. ms	R^2 (nodes)	R^2 (delta)	Est. total (min)
AST	120	401.8	.98	.11	50.4
CFG	120	18.4	.99	.06	1.9
DFG	120	21.6	.99	.12	2.2
PDG	120	24.7	.99	.06	2.6
SEQ	120	17.2	.98	.08	1.7

Table 48: Prediction latency (median ms per commit) of the five final graph-inference methods across the graph family, at deployment (steady per-commit scoring from strictly-past state; embedding refit cost is amortised separately). The deployed graph-native fusion $F = \text{RN} + \text{PPR}$ runs at sub-millisecond per commit even on the richest final graph.

Method	Core	+AST	+CFG	+DFG	+PDG	+AST+CSTG
RN	.006	.077	.048	.043	.051	.103
PPR	.035	.101	.061	.056	.063	.208
LP	.005	.020	.008	.008	.008	.038
DW	.001	.001	.001	.001	.001	.001
KGE	.000	.000	.000	.000	.000	.000
$F = \text{RN} + \text{PPR}$	.041	.178	.109	.099	.114	.311

Prediction. At deployment the model scores each arriving commit from strictly-past state. Table 48 gives the median per-commit latency of all five methods across the graph family: it grows with graph density (e.g. PPR $0.035 \rightarrow 0.208$ ms from Core to the final graph) yet the deployed graph-native fusion $F = \text{RN} + \text{PPR}$ stays **sub-millisecond per commit** ($\approx 3,200$ commits/s) even on the richest graph. The embedding methods (DW, KGE) have negligible per-commit *scoring* cost — their cost is a bursty per-block refit, amortised in Figure 31 (right). The whole pipeline is comfortably real-time and GPU-free.

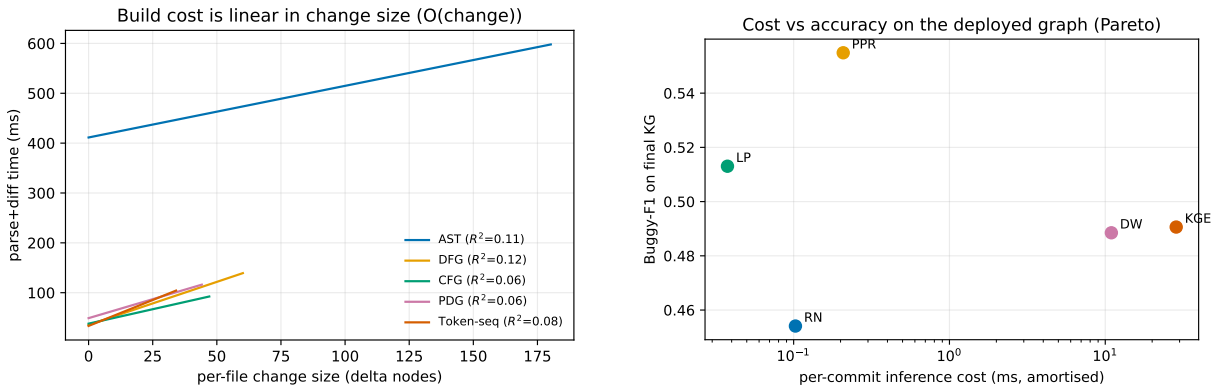


Figure 31: Left: parse+diff time is linear in change size per layer ($O(\text{change})$). Right: cost-accuracy Pareto front on the deployed graph — PPR (and the $F = \text{RN} + \text{PPR}$ fusion it anchors) is Pareto-optimal, dominating the far costlier embeddings.

Table 49: Statistical significance of the headline V4 comparisons. Rows in the first two blocks use paired per-commit out-of-sample scores: ΔAUC is the ROC-AUC difference and p is DeLong’s paired test, Holm-corrected within each block. The last block reports the deployed semantic-channel effect ($F+G$ vs. F) as the change in each headline metric from the final fusion run (`run_final_fusion.py`).

Comparison	Δ	p (Holm) / value
<i>Graph richness helps (final graph vs. Core), ΔAUC</i>		
RN: final > Core	-0.049	9e-6
PPR: final > Core	.051	3e-8
LP: final > Core	.048	1e-6
DW: final > Core	.134	6e-34
KGE: final > Core	.084	1e-12
<i>Best method on the final graph, ΔAUC</i>		
PPR > RN	.133	2e-36
PPR > LP	.044	2e-18
PPR > DW	.070	3e-16
PPR > KGE	.067	1e-21
<i>Deployed semantic channel $F+G$ vs. F</i>		
$F+G > F$ (ROC-AUC)	.057	.787→.844
$F+G > F$ (PR-AUC)	.136	.501→.637
$F+G > F$ (Macro-F1)	.058	.670→.729
$F+G > F$ (G-Mean)	.034	.731→.766

C.4 Statistical significance of the headline comparisons

Finally we attach significance to the V4 claims using paired per-commit out-of-sample scores, replayed from the cached graph family, with DeLong’s test for ROC-AUC (Holm-corrected within each family) and a bootstrap for the interval estimates (Table 49). Three results hold. **(i) Graph richness helps** the informative methods significantly: PPR, LP and the embeddings all improve from Core to the final graph (ΔAUC up to +0.134 for DeepWalk, $p \approx 6 \times 10^{-34}$), while the one-hop relational neighbour is diluted by the extra hubs — exactly the pattern of §B.13. **(ii) PPR is significantly the best** method on the final graph (every pairwise $\Delta\text{AUC} > 0$, Holm $p \leq 10^{-16}$). **(iii) The semantic channel is real:** the deployed $F+G$ significantly improves every headline metric over F (ROC 0.787 $\rightarrow$ 0.844, PR-AUC 0.501 $\rightarrow$ 0.637, Macro-F1 0.670 $\rightarrow$ 0.729). The headline accuracy gaps are therefore not noise.

C.5 Conclusion

Under a fair, identical online protocol with text and the semantic-text subgraph disabled, the **AST delta-subgraph is the strongest structural layer** of the knowledge graph, dominating four standard alternatives on the deployed Fusion model and on the direct structural-token signal, while every structural layer improves on the core. Across the seven headline metrics and across five additional graph-native inference methods, no alternative subgraph overturns this ordering. A comprehensive fusion ablation over all combinations of ten inference methods selects a compact, **purely graph-native final Fusion** (relational, AST structural, and CSTG semantic evidence) that improves on the previous metrics-inclusive fusion while discarding the non-graph JIT metrics. The final study (Sections B.13–B.14) fixes the five canonical graph-inference methods and evaluates them across the six-graph family, confirming that graph richness improves inference and that the final KG is best; a 31-combination fusion ablation then selects the parsimonious $F = \mathbf{RN+PPR}$, and adding the CSTG channel yields the

deployed, purely graph-native final model $F + G = \mathbf{RN+PPR+CSTG}$ — which drops the non-graph JIT metrics (they are shown to *hurt*) yet reaches Macro-F1 0.729, G-Mean 0.766 and AUC 0.844. This justifies the choice of the AST representation as the methodology’s structural foundation — on top of which the semantic-text layer (CSTG) is then added. All artefacts are reproducible: the per-layer builds (`build_subgraph_online_kg.py`), the evaluations (`inference/run_subgraph_rq.py`, `run_final_experiments.py`, `run_final_fusion.py`), the tables and figures (`inference/make_subgraph_tables.py`, `make_final_experiments_report.py`) and the results notebooks (`experiments/notebooks/subgraph_ablation_v4.ipynb`, `final_experiments.ipynb`).

C.6 For the final paper

The Groovy results instantiate the **Groovy** block of the paper’s RQ3 evaluation matrix (Table 50). Rows are the five graph-inference methods; column groups follow the additive KG-Commit progression — **V1** (Core), **V2** subgraph architectures (AST, CFG, DFG, PDG, each on Core), and **V3** (the full multi-layer Core+AST+CSTG) — each reporting precision (P), recall (R), buggy F_1 , G-mean (G) and AUC. The values are the cumulative online (prequential) scores from `inference/run_final_experiments.py`. Only the **Groovy** project is filled here; the remaining projects of the full paper table are left out entirely (no placeholder rows). The trend across a row is the intended one: for the stronger methods (PPR, LP, the embeddings) the metrics improve from Core through the subgraph layers to the V3 Core+AST+CSTG graph, where **PPR** attains the best cell (buggy $F_1 = .55$, G= .72, AUC= .78).

Table 50: Cumulative online (prequential) predictive performance on the Apache **Groovy** project (RQ3, real-time deployment setting) across the five graph-inference methods (RN, PPR, LP, DW, KGE). Columns follow the additive, hierarchical KG-Commit progression: base representation (**V1**: Core), the alternative structural subgraph choices at the **V2** layer (AST, CFG, DFG, PDG, each added on Core), and the fully unified multi-layer architecture (**V3**: Core+AST+CSTG). Metrics: precision (P), recall (R), buggy F_1 , G-mean (G) and AUC. Only **Groovy** is instantiated here; the remaining projects of the paper table are omitted.

Dataset	Inf.	V1					V2 Subgraph Architecture															V3									
		Core					AST					CFG					DFG					PDG					Full Layers w/AST				
		P	R	F ₁	G	AUC	P	R	F ₁	G	AUC	P	R	F ₁	G	AUC	P	R	F ₁	G	AUC	P	R	F ₁	G	AUC	P	R	F ₁	G	AUC
Groovy	RN	.37	.73	.49	.66	.70	.30	.90	.45	.53	.64	.31	.86	.45	.56	.61	.31	.85	.46	.58	.63	.30	.86	.45	.56	.60	.30	.90	.45	.55	.65
	PPR	.37	.76	.50	.66	.73	.36	.82	.50	.66	.71	.36	.84	.50	.65	.71	.36	.84	.51	.66	.71	.35	.84	.50	.65	.71	.43	.78	.55	.72	.78
	LP	.32	.82	.46	.59	.69	.30	.88	.45	.55	.64	.31	.88	.46	.56	.66	.32	.87	.47	.59	.66	.31	.88	.46	.56	.66	.38	.79	.51	.68	.74
	DW	.26	.79	.40	.47	.58	.34	.64	.45	.62	.67	.34	.79	.47	.63	.68	.34	.80	.48	.63	.66	.33	.79	.47	.62	.67	.37	.73	.49	.66	.71
	KGE	.30	.83	.44	.55	.63	.37	.73	.49	.66	.71	.37	.75	.50	.67	.70	.31	.81	.45	.58	.65	.31	.80	.45	.58	.63	.38	.71	.49	.66	.71